Machine Learning & Deep Learning

The Complete Guide - From Beginner to Expert

Foundations · Classical ML · Deep Learning · Transformers · Generative Models · Quantization & Pruning · On-Device Training · Reinforcement Learning · MLOps

Contents	35 chapters in 5 parts, plus 3 appendices
Level	Absolute beginner to research practitioner
Style	Intuition first, then the mathematics, then working code
Worked examples	Every core algorithm is computed by hand on real numbers you can verify with a calculator
Code	Python / NumPy / PyTorch (framework-agnostic where possible)

How To Use This Book

This book is written to be read front to back by someone who has never trained a model, and to be used as a reference by someone who trains them for a living. Every chapter follows the same three-beat rhythm: first the **intuition** in plain language, then the **mathematics** written out step by step with no skipped algebra, then **code** that you can run.

Nothing is assumed beyond high-school algebra. Everything else - vectors, matrices, derivatives, probability - is built up in Chapter 2 and used consistently afterwards.

The five parts

Part	Chapters	What you get out of it
I - Foundations	1-4	Vocabulary, the mathematics you actually need, what 'learning' means formally, and how to prepare data.
II - Classical ML	5-13	Linear and logistic regression, trees, SVMs, boosting, clustering, PCA, and how to measure a model honestly. Still the right answer for most tabular problems.
III - Deep Learning Core	14-20	Neurons, backpropagation derived by hand, activations, optimizers, normalization, regularization, and a practical debugging playbook.
IV - Architectures	21-27	CNNs, RNN/LSTM, attention and Transformers, LLMs, VAEs/GANs/diffusion, graph networks, and self-supervised learning.
V - Expert Topics	28-35	Quantization, pruning and sparsity, distillation, on-device and federated training, reinforcement learning, uncertainty and robustness, MLOps, and how to do research.

Reading paths

- **Complete beginner (about 6 months, part-time):** Chapters 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 13 -> 9 -> 11, then Part III in order. Do the exercises at the end of every chapter before moving on.
- **Programmer who wants deep learning fast:** skim 1-3, read 4, then jump to 14-20, then pick the architecture chapter that matches your data (21 for images, 22-23 for sequences, 26 for graphs).
- **Practitioner shipping to devices:** Part III as refresher, then 28-31 (quantization, pruning, distillation, on-device training) and 34 (MLOps).
- **Interview preparation:** 3, 5, 6, 13, 15, 17, 21, 23, plus the glossary in Appendix B.

Conventions used throughout

Symbol / style	Meaning
x, y	Scalars in italics in the maths; monospace in the code.
x (bold)	A vector; by default a column vector of shape (d, 1).
X (bold capital)	A matrix; the design matrix has shape (n, d): n rows = samples, d columns = features.
theta, w, b	Learnable parameters: generic parameters, weights, bias.
L, J	Per-sample loss L, and full objective / cost J averaged over data.
eta (or lr)	Learning rate.
Coloured boxes	KEY IDEA = the one thing to remember. MATH = a derivation. INTUITION = the mental picture. PRACTICAL TIP = what to actually do. PITFALL = a mistake people really make. EXPERT CORNER = depth you can skip on a first read.

Worked examples

Explanations are cheap; arithmetic is not. Wherever an algorithm has a core computation, this book performs it on real numbers and shows every intermediate value, so you can check the claim rather than accept it. Among

them: least squares solved by hand on five houses; a gradient-descent trace step by step; a logistic regression trained on four points; an exhaustive decision-tree split search; three rounds of gradient boosting on six numbers; k-means and PCA computed in full; every classification metric derived from one confusion matrix; a complete forward and backward pass through a small network with a numerical gradient check; one Adam update; convolution and receptive-field arithmetic; attention computed on three tokens; and eight weights quantized to INT8 with the exact error.

LEARN BY REBUILDING

Every algorithm in Parts II and III is presented so that you can reimplement it in NumPy in under 100 lines. Do that at least once for linear regression, logistic regression, a decision tree, and a two-layer network trained with your own backpropagation. Nothing else produces the same level of understanding - libraries hide exactly the parts that matter.

Table of Contents

PART I

Foundations

What learning from data means, the mathematics behind it, and how to set up a problem so that the answer is trustworthy.

1. What Machine Learning Really Is

A traditional program is a list of rules written by a person. You want to detect spam, so you write: *if the subject contains 'FREE MONEY', mark it as spam*. That works until the spammers write 'F R E E M0NEY'. You add another rule. They adapt again. After two years you have 4,000 brittle rules and no one understands them.

Machine learning inverts the arrow. Instead of writing the rules, you collect **examples** - 50,000 emails, each labelled spam or not-spam - and you write a program that **searches for the rules by itself**. The output of that search is a **model**: a function with numbers inside it that turns an input into a prediction.

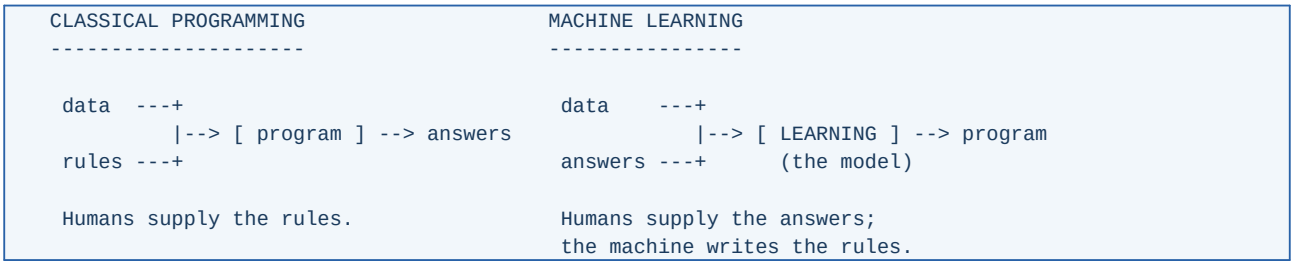

Figure 1.1 - The defining inversion of machine learning.

THE ONE-SENTENCE DEFINITION

Machine learning is the practice of fitting a parameterised function to data by minimising a measure of error, in the hope that the fitted function also works on data it has never seen. Everything else in this book - trees, transformers, diffusion models - is a choice about the shape of that function and the way you search for its parameters.

1.1 A worked micro-example before any theory

Suppose you have five houses and you want to predict price from size.

Size (m^2)	50	70	90	110	130
Price (k EUR)	150	195	260	300	355

You guess the relationship is a straight line, `price = w * size + b`. The pair `(w, b)` are the **parameters**. Learning here means: try many values of `w` and `b`, and keep the pair whose predictions are closest to the five real prices. 'Closest' has to be defined numerically - that definition is the **loss function**. The usual choice is mean squared error:

```
J(w, b) = (1/n) * SUM_i ( w * size_i + b - price_i )^2
```

For this data the best fit is roughly `w = 2.55`, `b = 20`. That model now predicts a 100 m^2 house at about 275k. You never wrote the rule 'each square metre costs 2,550 EUR' - the data implied it and the optimiser found it. That is the entire idea, and every later chapter is a richer version of this loop:

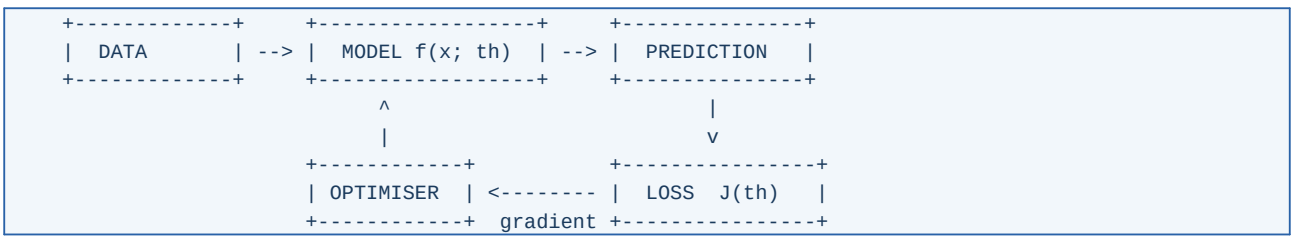

Figure 1.2 - The universal training loop: predict, score, adjust, repeat.

1.2 The vocabulary, defined once and used forever

Term	Meaning	In the house example
Sample / instance	One row of data, one thing you make a prediction about.	One house.
Feature	One measured input variable. d features per sample.	Size in m^2.
Feature vector x	All features of one sample stacked together.	[50]
Label / target y	The correct answer you want to predict.	150k
Design matrix X	All samples stacked: shape (n, d).	5 x 1 matrix
Model f(x; th)	The parameterised function producing predictions.	w*x + b
Parameters th	Numbers learned from data.	w and b
Hyperparameters	Numbers you choose before training, not learned from the training loss.	Learning rate, degree of polynomial
Loss L	Error on one sample.	(pred - price)^2
Cost / objective J	Average loss over a dataset, plus any penalties.	MSE over 5 houses
Training	Searching parameter space to minimise J.	Fitting w, b
Inference	Running the trained model on new inputs.	Pricing a new listing
Generalisation	Accuracy on data not used for training.	Pricing houses you never saw

1.3 The four learning paradigms

1. Supervised learning - you have the answers

Every training sample carries a label. The model learns a mapping `x -> y`. Two sub-types dominate:

- **Regression:** y is a continuous number. Price, temperature, remaining battery life, time-to-failure. Measured with MSE, MAE, R^2.
- **Classification:** y is one of K discrete classes. Spam / not spam, which of 1,000 objects is in this photo, which of 5 activities a wearable sensor is recording. Measured with accuracy, precision, recall, F1, AUC.

Supervised learning is by far the most reliable paradigm, and also the most expensive: someone has to produce the labels. A rule of thumb from industry is that 60-80% of the effort on a real supervised project is spent obtaining, cleaning and auditing labels, not on modelling.

2. Unsupervised learning - structure without answers

Only `x` is available. The goal is to find structure: **clusters** of similar samples (k-means, GMM), a **low-dimensional** description (PCA, autoencoders), a **density** model of where data lives (which also gives you anomaly detection), or **associations** between items.

3. Self-supervised learning - the answers hide in the data

Take unlabelled data and invent a task whose label is part of the data itself: hide a word and predict it, hide a patch of an image and reconstruct it, ask whether two augmented crops came from the same photo. This is how every modern large model - GPT-class language models, CLIP, DINO, wav2vec - is pretrained. It is technically supervised learning with free labels, and it is the single biggest reason deep learning scaled: it removed the labelling bottleneck. Chapter 27 covers it in depth.

4. Reinforcement learning - learning from consequences

An **agent** takes **actions** in an **environment**, receives a scalar **reward**, and must learn a **policy** that maximises cumulative reward. There is no labelled correct action, only delayed and noisy feedback, and the agent's own behaviour determines the data it sees. Robotics, game playing, and the alignment stage of large language models (RLHF) all live here. Chapter 32.

WHICH PARADIGM IS MY PROBLEM?

Ask what you can actually collect. If you can collect input-output pairs, use supervised learning - it is the most sample-efficient and the easiest to evaluate. If you can collect only inputs, use self-supervised pretraining and then fine-tune on the few labels you can afford. Use reinforcement learning only when the decision changes the future data (control, sequential decisions); otherwise it makes an easy problem hard.

1.4 Where deep learning fits

Deep learning is not a separate field from machine learning; it is the subset in which the model is a neural network with many layers, and in which the **features are learned rather than designed**. That is the whole distinction, and it is a big one:

```
CLASSICAL ML PIPELINE
  raw data -> [ hand-designed features ] -> [ learned classifier ] -> y
               ^ engineered by a human expert over months

DEEP LEARNING PIPELINE
  raw data -> [ layer1 -> layer2 -> ... -> layerN -> classifier ] -> y
               ^ every stage learned jointly from the same loss
```

Figure 1.3 - Feature engineering versus representation learning.

In a convolutional network trained on photographs, the first layer ends up detecting edges, the next combines edges into corners and textures, the next into object parts, and the last into whole objects. Nobody programmed that hierarchy; it is what minimising the classification loss produces. This is called **representation learning**, and it is why deep learning wins on images, audio, text and video, where useful features are hard for humans to write down.

DEEP LEARNING IS NOT ALWAYS THE ANSWER

On small or medium tabular datasets - the most common kind of data in industry - gradient-boosted trees (Chapter 11) usually beat neural networks, train in seconds instead of hours, need almost no tuning, and are easier to explain to a regulator. Reach for deep learning when the input is perceptual (pixels, waveforms, tokens), when you have a lot of data, or when you can start from a pretrained model.

1.5 The taxonomy at a glance

Paradigm	Input	Typical models	Typical use
Supervised - regression	x, y in R	Linear/ridge, gradient boosting, MLP	Price, demand, RUL
Supervised - classification	x, y in {1..K}	Logistic regression, trees, CNN, Transformer	Spam, diagnosis, vision
Unsupervised - clustering	x only	k-means, GMM, DBSCAN, HDBSCAN	Segmentation, grouping
Unsupervised - dim. reduction	x only	PCA, t-SNE, UMAP, autoencoder	Visualisation, compression
Unsupervised - density	x only	GMM, normalising flow, diffusion	Anomaly detection, generation
Self-supervised	x + invented task	BERT/GPT objectives, SimCLR, MAE	Pretraining foundation models
Reinforcement	state, action, reward	DQN, PPO, SAC, AlphaZero	Control, games, RLHF

1.6 A first end-to-end program

Here is a complete, honest supervised-learning workflow in 25 lines. Read it now even if the details are unfamiliar; every line is explained in the next three chapters.

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, roc_auc_score

X, y = load_breast_cancer(return_X_y=True)      # 569 samples, 30 features

# 1. Hold out a test set FIRST and do not look at it again until the end.
X_tr, X_te, y_tr, y_te = train_test_split(
        X, y, test_size=0.2, stratify=y, random_state=0)

# 2. Scaling belongs INSIDE the pipeline so it is refit on each CV fold.
model = make_pipeline(StandardScaler(),
                      LogisticRegression(max_iter=5000, C=1.0))

# 3. Estimate generalisation using cross-validation on the training set.
cv = cross_val_score(model, X_tr, y_tr, cv=5, scoring='roc_auc')
print(f'CV AUC: {cv.mean():.3f} +/- {cv.std():.3f}')

# 4. Refit on all training data, then evaluate ONCE on the test set.
model.fit(X_tr, y_tr)
proba = model.predict_proba(X_te)[:, 1]
print('test AUC:', round(roc_auc_score(y_te, proba), 3))
print(classification_report(y_te, (proba > 0.5).astype(int)))
```

Listing 1.1 - The shape of every honest supervised experiment.

Four habits in that listing separate practitioners from beginners: the test set is split off **before** anything else happens; preprocessing is inside the pipeline so it cannot leak information across folds; the model is selected using cross-validation, not the test set; and the test set is touched exactly once.

1.7 Why machine learning works now and not in 1990

The core algorithms are old: least squares is from 1805, the perceptron from 1958, backpropagation was popularised in 1986, and convolutional networks recognised digits commercially in the early 1990s. Nothing conceptual was missing. Three practical things changed, and it is worth knowing which one you are short of when a project stalls.

Ingredient	Then	Now	Why it mattered
Data	Thousands of hand-collected samples	Billions of images, trillions of text tokens	Large models need large data; the internet supplied it
Compute	A workstation doing millions of operations per second	A GPU doing 10^14 operations per second	Training that took a year now takes an hour, so you get hundreds of attempts instead of one
Method	Sigmoid units, random initialisation, plain SGD	ReLU, He initialisation, batch norm, residual connections, Adam	These made deep networks trainable at all - before them, depth beyond a few layers simply did not converge

WHICH OF THE THREE IS YOUR BOTTLENECK?

If more data reliably improves your validation score, you are data-bound: buy labels or use self-supervision. If your model has already fitted the training set perfectly and you cannot afford a bigger one, you are compute-bound. If training is unstable, diverging, or plateauing far above a reasonable loss, you are method-bound - and Part III is the chapter list for that.

1.8 A day in the life of a machine-learning project

Textbooks present modelling as the main activity. It is not. Here is where the time actually goes on a typical supervised project, and the chapter that covers each stage.

Stage	Share of effort	What it involves	Chapter
Framing the problem	5%	Deciding what to predict, what a prediction is worth, and what a mistake costs	1, 13
Getting and labelling data	35%	Collection, joins, consent, labelling guidelines, adjudicating disagreements	4
Cleaning and features	25%	Missing values, outliers, encodings, aggregations, leakage audits	4
Modelling	10%	Baseline, then a stronger model, then tuning	5-27
Evaluation	10%	Metrics, slices, calibration, error analysis	3, 13
Deployment and monitoring	15%	Serving, drift, retraining, incident response	34

THE MOST COMMON BEGINNER MISTAKE

Spending three weeks on architectures before spending three days looking at the data. Print fifty random rows. Look at fifty random images with their labels. Find the ten samples your baseline gets most wrong and read them one by one. Almost every project has a data problem hiding in plain sight, and no architecture fixes a wrong label.

1.9 Reading the training loop as a sentence

Every training run, in every framework, in every architecture in this book, is the same five-line sentence. If you can narrate these five lines you can read any deep-learning codebase.

```
for xb, yb in loader:          # 1. take a batch of examples
    pred = model(xb)           # 2. FORWARD:  guess the answers
    loss = lossf(pred, yb)     # 3. SCORE:    how wrong were the guesses?
    loss.backward()            # 4. BACKWARD: how should each weight change?
    opt.step()                 # 5. UPDATE:   change every weight a little
    opt.zero_grad()            #    then forget the old gradients
```

Listing 1.2 - The five verbs: batch, forward, score, backward, update.

Chapters 14-17 explain each verb in full: forward is a stack of matrix multiplications and non-linearities; score is the negative log-likelihood of your data under an assumed noise model; backward is the chain rule applied right to left; and update is a small step downhill with a per-parameter step size. Everything else - convolutions, attention, diffusion - changes only what happens inside step 2.

1.10 What can go wrong (a preview of Chapter 3)

- **Overfitting:** the model memorises the training data, including its noise, and fails on new data. The classic symptom is 99% training accuracy and 62% test accuracy.
- **Underfitting:** the model is too simple to capture the pattern - a straight line through a curved relationship. Both training and test error are high.
- **Data leakage:** information about the answer sneaks into the features. Scaling before splitting, using a future value as a feature, or having the same patient in both train and test all inflate your score and produce a model that fails in production.
- **Distribution shift:** the world changes after you train. Prices move, sensors drift, users behave differently. A model is a photograph of the past.
- **Optimising the wrong metric:** 99% accuracy on a dataset where 99% of samples are negative means your model learned to say 'no'.

Exercises

1. Fit `price = w*size + b` on the five houses above by hand: compute J for (w, b) = (2.0, 50), (2.55, 20), (3.0, 0) and say which is best.
2. For each of these, name the paradigm and the metric you would use: predicting tomorrow's electricity demand; grouping customers with no labels; teaching a drone to land; filling in masked words in Wikipedia.
3. Run Listing 1.1. Then deliberately break it by scaling `X` before the split and observe how much the test AUC changes - this is leakage in miniature.

2. The Mathematical Toolkit You Actually Need

You do not need a mathematics degree. You need fluency with four things: **linear algebra** (how data and parameters are stored and multiplied), **calculus** (how a loss changes when a parameter moves), **probability** (how to talk about uncertainty and where loss functions come from), and **optimisation** (how to move parameters downhill). This chapter builds all four from scratch, with machine-learning meaning attached to every object.

2.1 How to read the mathematics in this book

Mathematical notation is compressed English. This section decompresses the five symbols that do most of the work, so that no equation later in the book is opaque.

Symbol	Read it aloud as	Example	Meaning of the example
SUM_i x_i	the sum over i of x sub i	SUM_i x_i	Add up every element of the list x
PROD_i x_i	the product over i	PROD_i p_i	Multiply all the probabilities together
argmin_w f(w)	the w that makes f smallest	argmin_w J(w)	The parameters with the lowest loss - not the loss value itself, the PARAMETERS
E[X]	the expected value of X	E[loss]	The average loss if you could see infinite data
x ~ D	x is drawn from the distribution D	eps ~ N(0, 1)	The noise is a random draw from a standard normal
dJ/dw	the derivative of J with respect to w	dJ/dw = 3	If w increases by 0.01, J increases by about 0.03
||x||	the norm, i.e. the length of x	||w||^2	The squared length of the weight vector
a := b or a <- b	a is defined as / set to b	w <- w - eta g	Overwrite w with the new value

THE TRICK THAT MAKES EQUATIONS READABLE

Whenever you meet an unfamiliar equation, do two things. First, say out loud what each symbol IS - a scalar, a vector, a matrix, and of what shape. Second, substitute the smallest possible concrete case: one sample, two features, one output. Almost every equation in machine learning becomes obvious at n = 1, d = 2, and the general case is only bookkeeping on top of that.

2.2 Linear algebra: the language of data

Scalars, vectors, matrices, tensors

Object	Notation	Shape	In ML
Scalar	x	()	A learning rate, a single loss value
Vector	x (bold)	(d,)	One sample's features; one layer's biases
Matrix	X (bold cap)	(n, d)	A batch of n samples; a weight matrix
3-tensor	T	(N, H, W)	A batch of greyscale images
4-tensor	T	(N, C, H, W)	A batch of colour images in PyTorch order

A **tensor** in deep-learning software is simply an n-dimensional array with an attached gradient machinery. The word carries no physics meaning here. What matters in practice is **shape discipline**: most bugs in deep learning are shape bugs, and the fastest debugging habit you can build is printing `tensor.shape` at every step.

The three products you must never confuse

```
Dot product      a . b  = SUM_i a_i b_i          -> a scalar
Matrix-vector    (A x)_i = SUM_j A_ij x_j        -> a vector
Elementwise      (a * b)_i = a_i b_i             -> same shape
```

Matrix multiplication `C = A B` with A of shape (m, k) and B of shape (k, n) gives C of shape (m, n), where `C_ij = SUM_k A_ik B_kj`. The inner dimensions must match; the outer ones survive. Reading that rule as *(m, k) x (k, n) -> (m, n)* and checking it mentally before every line you write will save you hours.

A NEURAL NETWORK LAYER IS ONE MATRIX MULTIPLY

A fully connected layer computing z = W x + b for a single sample becomes Z = X W^T + b for a whole batch, where X is (n, d) and W is (units, d). The reason GPUs transformed this field is that this single operation - dense matrix multiplication - is exactly what they do thousands of times faster than a CPU.

Useful identities you will meet again in the backpropagation chapter:

```
(A B)^T = B^T A^T

(A B) C = A (B C)              (associative)

A (B + C) = A B + A C          (distributive)

A B != B A                     (NOT commutative)
```

Norms - how big is this vector?

```
L1 norm     ||x||_1 = SUM_i |x_i|            -> promotes sparsity
L2 norm     ||x||_2 = sqrt( SUM_i x_i^2 )    -> ordinary length
L-inf norm  ||x||_inf = max_i |x_i|          -> worst single component
```

L2 is the default: it is smooth, differentiable everywhere, and gives Euclidean distance `||a - b||_2`. L1 is not differentiable at zero, and that corner is precisely why L1 regularisation drives coefficients exactly to zero (Chapter 7). L-infinity appears in adversarial robustness, where an attacker may perturb every pixel by at most epsilon (Chapter 33).

Geometry: angles, projections, similarity

```
a . b = ||a|| ||b|| cos(theta)      =>      cos_sim(a, b) = (a . b) / (||a|| ||b||)
```

Cosine similarity ignores magnitude and compares direction only; it is the standard way to compare embeddings, and it is the numerator inside attention (Chapter 23) and inside every vector database. Two vectors are **orthogonal** when their dot product is zero - they share no information in the linear sense.

Matrix decompositions that carry real meaning

- **Eigendecomposition** `A = Q L Q^-1`: for a symmetric matrix, Q is orthogonal and L diagonal. Eigenvectors are directions the matrix only stretches; eigenvalues are the stretch factors. The eigenvalues of the **Hessian** of your loss describe the curvature of the optimisation landscape, and their ratio (the **condition number**) predicts how badly plain gradient descent will zig-zag.
- **Singular value decomposition** `A = U S V^T`: exists for every matrix. Keeping only the top k singular values gives the best possible rank-k approximation (Eckart-Young theorem). This one fact powers PCA (Chapter 12), latent semantic analysis, recommender systems, and LoRA fine-tuning of large models (Chapter 24).
- **Rank**: the number of linearly independent directions a matrix actually uses. Low-rank structure is what makes compression possible; 'low-rank adaptation' means updating a big weight matrix with a product of two thin ones.

```
import numpy as np
A = np.random.randn(200, 50)
U, s, Vt = np.linalg.svd(A, full_matrices=False)
k = 10
A_k = (U[:, :k] * s[:k]) @ Vt[:k]          # best rank-10 approximation
err = np.linalg.norm(A - A_k) / np.linalg.norm(A)
print(k, 'components keep', round(100*(1-err), 1), '% of the energy')
# Storage: 200*50 = 10,000 numbers  ->  (200+50)*10 = 2,500 numbers
```

Listing 2.1 - Low-rank approximation: the mathematical core of PCA and LoRA.

2.3 Calculus: how the loss reacts to a parameter

Derivative, partial derivative, gradient

The derivative `df/dx` is the rate of change of f at a point: move x by a tiny amount h and f moves by about `h * df/dx`. With many inputs, the **partial derivative** `dJ/dw_i` measures the effect of one parameter with all others frozen. Collecting all partials gives the **gradient**:

```
grad J(w) = [ dJ/dw_1 , dJ/dw_2 , ... , dJ/dw_d ]
```

THE GRADIENT IS THE UPHILL DIRECTION

Stand on a hillside in fog. The gradient points in the direction of steepest ascent, and its length says how steep. To minimise a loss you therefore step in the direction of the NEGATIVE gradient. That single sentence is the whole of gradient descent, and by extension most of modern machine learning.

The chain rule - the engine of deep learning

If `y = f(u)` and `u = g(x)`, then `dy/dx = (dy/du) * (du/dx)`. For a network that computes a composition of L layers, the derivative of the loss with respect to an early weight is a product of L local derivatives. Backpropagation (Chapter 15) is nothing more than an efficient, right-to-left evaluation of that product, reusing shared subexpressions.

```
J = L(a_L),   a_L = f_L(a_L-1),  ... ,  a_1 = f_1(x)

dJ/dW_k = dJ/da_L * da_L/da_L-1 * ... * da_k+1/da_k * da_k/dW_k
```

Two immediate consequences, both central to Part III: if the local factors are consistently smaller than 1, the product shrinks exponentially with depth (**vanishing gradients**); if consistently larger than 1, it explodes (**exploding gradients**). Residual connections, careful initialisation and normalisation layers all exist to keep that product near 1.

Jacobians, Hessians and what curvature buys you

- **Jacobian** J: the matrix of all first partials of a vector-valued function, shape (outputs, inputs). Automatic differentiation never builds it explicitly; it computes Jacobian-vector products instead, which is why backprop costs about the same as a forward pass.
- **Hessian** H: the matrix of second partials, shape (d, d). Positive definite H means a local minimum; mixed signs mean a saddle point. In high dimensions saddles vastly outnumber local minima, which is the modern explanation of why gradient descent works so well on non-convex networks.
- **Second-order methods** (Newton, L-BFGS, K-FAC) use curvature to choose the step size per direction. They converge in fewer iterations but cost O(d^2) or O(d^3) per step; with d in the billions they are impractical, which is why Adam - a cheap diagonal approximation - dominates.

2.4 Probability: reasoning under uncertainty

The rules

```
Sum rule          P(A) = SUM_B P(A, B)
Product rule      P(A, B) = P(A | B) P(B)
Bayes rule        P(H | D) = P(D | H) P(H) / P(D)
Independence      P(A, B) = P(A) P(B)
```

Bayes' rule is how you turn a **likelihood** (how probable is this data if the hypothesis holds) into a **posterior** (how probable is the hypothesis given the data). Read it as: *posterior is proportional to likelihood times prior*.

THE CLASSIC MEDICAL-TEST CALCULATION

A disease affects 1 in 1,000 people. A test has 99% sensitivity and 99% specificity. You test positive - what is the probability you are ill? P(D)=0.001, P(+|D)=0.99, P(+|not D)=0.01. Then P(+) = 0.99*0.001 + 0.01*0.999 = 0.01098, so P(D|+) = 0.00099 / 0.01098 = 9.0%. Ninety-one percent of positives are false. This is the same arithmetic as precision on an imbalanced classification problem (Chapter 13) - and the same reason a 99%-accurate fraud detector can be useless.

Distributions you will meet

Distribution	Support	Parameters	Where it appears in ML
Bernoulli	{0,1}	p	Binary labels; the output of a sigmoid
Categorical	{1..K}	p_1..p_K	Multiclass labels; softmax output
Binomial	0..n	n, p	Counts of successes; A/B tests
Gaussian	R	mu, sigma^2	Noise models, weight init, VAEs, diffusion
Laplace	R	mu, b	The prior behind L1 regularisation
Poisson	0,1,2..	lambda	Event counts; click and arrival models
Exponential	R+	lambda	Waiting times, survival analysis
Uniform	[a,b]	a, b	Random search, dropout masks, init ranges
Beta / Dirichlet	simplex	alpha	Priors over probabilities; topic models

Expectation, variance, covariance

```
E[X] = SUM_x x P(x)                          (mean)
Var[X] = E[(X - E[X])^2] = E[X^2] - E[X]^2
Cov[X, Y] = E[(X - E X)(Y - E Y)]
Corr[X, Y] = Cov[X, Y] / (sd(X) sd(Y))    in [-1, 1]
```

Linearity of expectation, `E[aX + bY] = aE[X] + bE[Y]`, holds even when X and Y are dependent, and it is used constantly - for instance to show that a minibatch gradient is an unbiased estimate of the full-dataset gradient, which is the entire justification for stochastic gradient descent.

Maximum likelihood: where loss functions come from

Assume your data was generated by a model with parameters theta. The **likelihood** is the probability of the observed data under that model. Maximum-likelihood estimation picks theta to maximise it; equivalently it minimises the negative log-likelihood, because logs turn products into sums and are monotone:

```
theta_MLE = argmax_theta PROD_i p(y_i | x_i; theta)
          = argmin_theta  -SUM_i log p(y_i | x_i; theta)
```

TWO DERIVATIONS YOU SHOULD BE ABLE TO DO FROM MEMORY

Assume Gaussian noise, y = f(x) + eps with eps ~ N(0, sigma^2). The negative log-likelihood becomes SUM (y_i - f(x_i))^2 / (2 sigma^2) plus a constant - that is MEAN SQUARED ERROR. Assume a Bernoulli label with p = sigmoid(z). The negative log-likelihood becomes -SUM [y log p + (1-y) log(1-p)] - that is CROSS-ENTROPY. Loss functions are not arbitrary; each one encodes an assumption about the noise in your labels.

Information theory in one page

```
Entropy            H(p) = -SUM_x p(x) log p(x)

Cross-entropy      H(p, q) = -SUM_x p(x) log q(x)

KL divergence      KL(p || q) = SUM_x p(x) log( p(x)/q(x) ) >= 0

Relationship       H(p, q) = H(p) + KL(p || q)
```

Entropy measures average surprise, in bits if the log is base 2. Cross-entropy measures the cost of encoding data from p using a code built for q. Since H(p) is fixed by the data, **minimising cross-entropy is exactly minimising KL divergence** between the true label distribution and your model - which is why it is the default classification loss. KL is not symmetric, and that asymmetry is the difference between mode-seeking and mode-covering behaviour in variational inference and GAN training (Chapter 25).

2.5 Worked example: matrix shapes in a real layer

Suppose a batch of 4 samples, each with 3 features, entering a layer with 2 output units. Track the shapes:

```
X  (4, 3)     four samples, three features each

W  (2, 3)     two units, each with three weights

b  (2,)       one bias per unit

Z = X W^T + b        (4,3) x (3,2) -> (4,2),  b broadcasts over rows

A = phi(Z)           (4,2)  elementwise, shape unchanged
```

Read `X W^T` as: for each of the 4 rows of X, take the dot product with each of the 2 rows of W. That is 4 x 2 = 8 dot products, each of length 3, which is exactly what a GPU does in one fused operation. If you ever see a shape error in deep-learning code, write the three shapes down in this form and the mismatch becomes visible immediately.

2.6 Worked example: a derivative you can check by hand

Take `J(w) = (w - 3)^2`, the simplest possible loss. Its derivative is `dJ/dw = 2(w - 3)`. Start at w = 0 with a learning rate of 0.1 and turn the crank:

Step t	w	J(w)	dJ/dw	New w = w - 0.1 * dJ/dw
0	0.000	9.000	-6.000	0.600
1	0.600	5.760	-4.800	1.080
2	1.080	3.686	-3.840	1.464
3	1.464	2.359	-3.072	1.771
10	2.678	0.104	-0.644	2.742
30	2.996	0.000	-0.007	2.997

Three lessons live in that table, and all three carry over unchanged to a billion-parameter network. The steps are large when the gradient is large and shrink automatically as you approach the minimum. The loss falls quickly at first and then slowly - which is why loss curves are shaped the way they are. And the process never

quite arrives: it converges geometrically towards w = 3, which is why 'train until it stops improving' is a practical rule rather than a mathematical one.

WHAT HAPPENS IF THE LEARNING RATE IS WRONG

With eta = 0.1 the update is w <- w - 0.2(w - 3), so the distance to the optimum is multiplied by 0.8 each step - smooth convergence. With eta = 0.5 the factor is 0, and you land exactly on the optimum in one step. With eta = 0.9 the factor is -0.8: you overshoot and oscillate, but still converge. With eta = 1.1 the factor is -1.2, and the distance GROWS every step - divergence. For this loss the exact threshold is eta < 1, which is 2/L with curvature L = 2. That is the general rule from Chapter 17 in miniature.

2.7 Worked example: Bayes' rule as counting

Probability is easier when you count people instead of manipulating fractions. Take 100,000 people, a disease affecting 1 in 1,000, and a test with 99% sensitivity and 99% specificity:

```
100,000 people
  |
  +-- 100 ill        --> 99 test positive     (true positives)
  |                      1 tests negative     (false negative)
  |
  +-- 99,900 healthy --> 999 test positive    (false positives)
                         98,901 test negative (true negatives)

positives = 99 + 999 = 1,098
P(ill | positive) = 99 / 1,098 = 9.0%
```

Figure 2.2 - Bayes' rule done by counting people rather than by algebra.

The result is identical to the algebra in the box above, but the counting version makes the cause visible: there are simply far more healthy people than ill ones, so even a small false-positive RATE produces a large false-positive COUNT. Keep this picture in mind for Chapter 13, where the same arithmetic explains why a 99%-accurate fraud detector can still be wrong nine times out of ten.

2.8 Optimisation: moving downhill

Gradient descent, stated precisely

```
repeat:   theta <- theta - eta * grad J(theta)
```

Convergence depends on the learning rate eta. For a convex quadratic with largest Hessian eigenvalue L, gradient descent converges only if `eta < 2/L`. Too small and you crawl; too large and you oscillate or diverge. This one hyperparameter is still, in 2020s practice, the most important knob in deep learning.

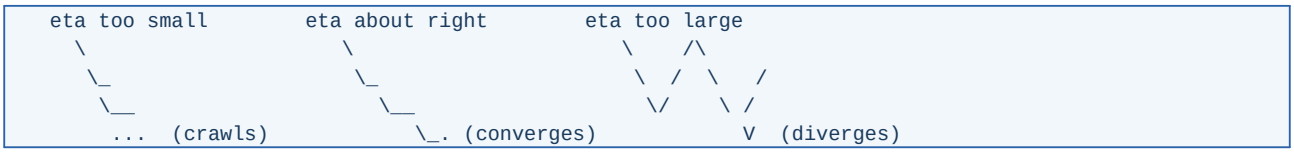

Figure 2.1 - The effect of the learning rate on the same loss surface.

Batch, stochastic and mini-batch

Variant	Gradient uses	Cost per step	Behaviour
Batch GD	all n samples	O(n)	Smooth, exact, far too slow for large n
SGD	1 sample	O(1)	Very noisy, can escape shallow minima
Mini-batch	B samples (32-8192)	O(B)	The universal default: parallel-friendly and stable

Mini-batching wins for a hardware reason as much as a statistical one: a GPU computes a 256-sample batch in barely more wall-clock time than a single sample, because the bottleneck is memory movement, not arithmetic.

Convexity, and why non-convexity is survivable

A function is **convex** if the line segment between any two points on the graph lies above the graph; then any local minimum is global. Linear regression, ridge, logistic regression and linear SVMs are convex - you can trust the optimum. Neural networks are decidedly non-convex, yet gradient descent finds excellent solutions. The current understanding: in very high dimensions, most critical points are saddles rather than poor local minima, and over-parameterised networks have wide, connected basins of good solutions.

Constrained optimisation and Lagrange multipliers

To minimise f(x) subject to g(x) = 0, form `L(x, lam) = f(x) + lam g(x)` and set all partial derivatives to zero. This machinery yields the dual formulation of SVMs (Chapter 10), the eigenvalue problem in PCA (Chapter 12), and the equivalence between constrained-norm and penalised-norm regularisation (Chapter 7).

NUMERICAL CARE THAT SAVES REAL EXPERIMENTS

Never compute `log(softmax(z))` directly - subtract max(z) first and use the log-sum-exp trick, or call `log_softmax`. Never compute `log(sigmoid(z))` - use `logsigmoid` or a loss that takes raw logits (`BCEWithLogitsLoss`). Float32 has about 7 decimal digits of precision and float16 about 3, with a maximum of 65,504 - which is why mixed precision training needs loss scaling (Chapter 20). Catastrophic cancellation when subtracting nearly equal numbers is the source of most mysterious NaNs.

Exercises

1. Show that for f(w) = ||Xw - y||^2 the gradient is 2 X^T (Xw - y). Do it componentwise first, then in matrix form.
2. Derive cross-entropy from the Bernoulli likelihood, then differentiate it with respect to the logit z where p = sigmoid(z). You should get the famously clean result p - y.
3. Compute the condition number of [[1, 0], [0, 100]] and sketch what gradient descent does on the quadratic it defines.
4. Implement log-sum-exp in NumPy and compare it against a naive implementation on the input [1000, 1001, 1002].

3. The Learning Problem: Generalization, Bias and Variance

Training error is easy to drive to zero - store the training set in a lookup table. The entire difficulty of machine learning is **test** error. This chapter gives you the formal statement of that difficulty and the practical tools for managing it. If you read only one chapter of Part I, read this one; nearly every real-world failure traces back to something here.

3.1 The formal setup

Assume samples (x, y) are drawn independently from an unknown distribution D. The quantity you care about is the **risk**, the expected loss on a fresh sample:

```
R(f)      = E_(x,y)~D [ L(f(x), y) ]          <- what you want
R_emp(f)  = (1/n) SUM_i L(f(x_i), y_i)        <- what you can measure
```

You minimise R_emp and hope R is close to it. The gap `R - R_emp` is the **generalisation gap**. Statistical learning theory bounds it, roughly, by a term that grows with the capacity of your model class and shrinks with the square root of the number of samples:

```
R(f) <= R_emp(f) + O( sqrt( capacity / n ) )
```

The classical capacity measures are **VC dimension** (the largest set of points the class can label in all possible ways) and **Rademacher complexity** (how well the class can fit pure noise). These bounds are loose for deep networks - a ResNet can memorise random labels, so its VC dimension is enormous, yet it still generalises. Explaining that is an active research area (implicit regularisation of SGD, flat minima, margin-based and compression-based bounds). The bounds are still worth knowing because their **shape** is right: more capacity hurts, more data helps, and the trade-off is what you tune.

3.2 Overfitting and underfitting, seen on a curve

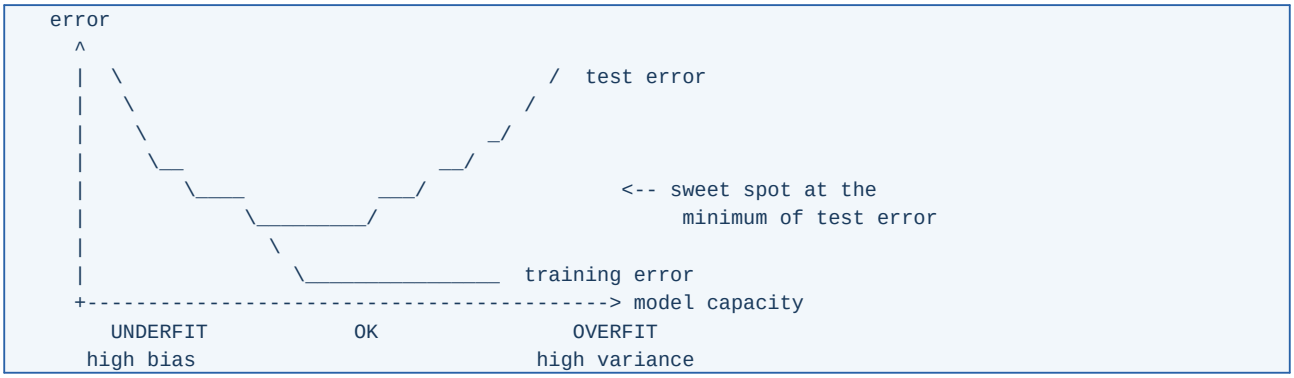

Figure 3.1 - The classical capacity/error picture.

Symptom	Diagnosis	What to do
Train error high, test error high (similar)	Underfitting / high bias	Bigger model, more features, train longer, reduce regularisation, check the learning rate
Train error low, test error much higher	Overfitting / high variance	More data, augmentation, stronger regularisation, smaller model, early stopping, ensembling
Train error low, test low, production bad	Distribution shift or leakage	Audit the features for leakage, re-split by time or group, monitor drift, retrain regularly
Test error lower than train error	Usually a bug, or dropout/augmentation active only at train time	Verify the split; check eval() mode

3.3 The bias-variance decomposition, derived

For squared loss and a target `y = f*(x) + eps` with `E[eps] = 0` and `Var[eps] = sigma^2`, consider the prediction `f_hat(x)` produced by training on a random dataset. Taking the expectation over datasets:

```
E[ (y - f_hat(x))^2 ]  =  ( E[f_hat(x)] - f*(x) )^2      <- BIAS^2
                       +  E[ (f_hat(x) - E[f_hat(x)])^2 ] <- VARIANCE
                       +  sigma^2                          <- IRREDUCIBLE
```

WHERE EACH TERM COMES FROM

Add and subtract the average prediction E[f_hat(x)] inside the square, expand, and observe that the cross term vanishes because E[f_hat - E f_hat] = 0. Noise eps is independent of the model, so it separates out. Bias is systematic error - the model class cannot represent the truth. Variance is sensitivity to which particular training set you happened to draw. Irreducible noise is the floor: no model, however large, can go below sigma^2.

- **High bias, low variance:** linear regression on a curved relationship; a depth-2 decision tree; heavy L2 regularisation.
- **Low bias, high variance:** a fully grown decision tree; 1-nearest neighbour; a huge network trained without regularisation on a small dataset.
- **Bagging** (Chapter 11) attacks variance by averaging many high-variance models. **Boosting** attacks bias by adding many high-bias models in sequence. Knowing which one you need is the point of the decomposition.

DOUBLE DESCENT - THE MODERN AMENDMENT

Push capacity past the point where the model exactly interpolates the training data and test error often falls AGAIN, sometimes below its classical minimum. The test-error curve is therefore not U-shaped but U-then-down. This is observed for random-feature models, boosting and deep networks, and it is why 'the model is too big, it will overfit' is no longer sound advice for neural networks. In the over-parameterised regime the useful lever is not smaller capacity but better implicit and explicit regularisation.

3.4 Splitting data honestly

Train / validation / test

Split	Typical size	Used for	How often you may look
Training	60-80%	Fitting parameters	Continuously
Validation	10-20%	Choosing hyperparameters, early stopping, model selection	Many times - it is being consumed
Test	10-20%	One final unbiased estimate	Once. Genuinely once.

THE VALIDATION SET DECAYS WITH USE

Every hyperparameter choice made on the validation set leaks a little information into your model. After a hundred experiments, validation performance is optimistic - you have partially fitted the validation set through your own decisions. This is why a untouched test set exists, and why leaderboards eventually go stale. If you must run hundreds of experiments, refresh the validation split or hold back a second test set.

Cross-validation

```
  5-fold cross-validation (shaded = validation fold)
    fold 1:  [VVVV][    ][    ][    ][    ]
    fold 2:  [    ][VVVV][    ][    ][    ]
    fold 3:  [    ][    ][VVVV][    ][    ]
    fold 4:  [    ][    ][    ][VVVV][    ]
    fold 5:  [    ][    ][    ][    ][VVVV]
    score = mean of the five validation scores  (+/- std)
```

Figure 3.2 - k-fold cross-validation uses every sample for both roles.

- **k-fold (k = 5 or 10):** the default. Cost is k trainings.
- **Stratified k-fold:** preserves class proportions in every fold - always use it for classification, especially when classes are imbalanced.
- **Leave-one-out:** k = n. Nearly unbiased but high variance and very expensive; useful only for tiny datasets.
- **Group k-fold:** keeps all samples from one patient, user or device in the same fold. Without it, a model can recognise the individual instead of the condition.
- **Time-series split:** train on the past, validate on the future, never shuffle. Rolling or expanding windows only.
- **Nested CV:** an inner loop selects hyperparameters, an outer loop estimates performance. This is the statistically correct way to report a number when you also tuned - and it is what reviewers ask for.

```python
from sklearn.model_selection import (StratifiedKFold, GroupKFold,
                                     TimeSeriesSplit, cross_val_score)

# classification, keeps class balance in each fold
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)

# one subject must never appear in both train and validation
cv = GroupKFold(n_splits=5)          # pass groups=subject_ids to the split

# temporal data: fold i trains on [0..t_i] and validates on (t_i..t_i+1]
cv = TimeSeriesSplit(n_splits=5)

scores = cross_val_score(model, X, y, cv=cv, scoring='f1_macro')
print(scores.mean(), '+/-', scores.std())
```

Listing 3.1 - Choosing the splitter that matches your data's structure.

3.5 Data leakage: the silent score inflator

Leakage is any situation where information that will not be available at prediction time influences training. It is the single most common reason a model that scored 0.97 offline scores 0.61 in production.

Leak	How it happens	Fix
Preprocessing leak	Scaler, imputer or feature selector fitted on the whole dataset before splitting	Fit inside a Pipeline, on training folds only
Target leak	A feature computed from the label, e.g. `total_paid` when predicting default	Audit every feature: could I compute it at prediction time?
Temporal leak	Training on rows from after the prediction timestamp	Split by time; build features with explicit as-of joins
Group leak	The same patient, user or device in train and test	GroupKFold, or split by entity
Duplicate leak	Near-duplicate rows or images across splits	Deduplicate, including perceptual near-duplicates
Tuning leak	Feature selection or hyperparameter search done before the split	Everything data-dependent goes inside the CV loop

3.6 Learning curves: the diagnostic you should always plot

Plot training and validation error against the number of training samples. The shape tells you what to buy next:

Shape	Interpretation	Action
Both curves high and converged	High bias; more data will not help	Increase capacity or improve features
Wide gap, validation still falling	High variance; more data will help	Collect or synthesise data; augment; regularise
Validation rises after a point	Overfitting with training duration	Early stopping; stronger regularisation
Both curves noisy	Batch too small, learning rate too high, or the validation set is too small	Enlarge the validation set; average over seeds

3.7 Overfitting made concrete: one dataset, three models

Twenty-five points were generated from `y = sin(x) + noise`. Three polynomials were fitted to the same 20 training points and evaluated on the same 5 held-out points. Nothing differs except the degree.

Degree	Train RMSE	Test RMSE	Diagnosis	What the curve does
1	0.42	0.45	Underfitting - high bias	A straight line through a wave: wrong everywhere, equally wrong on new data
3	0.11	0.13	About right	Follows the wave, ignores the noise
15	0.01	1.87	Overfitting - high variance	Passes through every training point and swings wildly between them

Notice the signature of overfitting in the numbers: training error goes **down** while test error goes **up**. That divergence, not the absolute value of either, is the thing to watch. A model with 5% training error and 6% test error is healthier than one with 0.1% training error and 4% test error, even though the second has a better test score - the second one is telling you it would improve with regularisation or more data.

MEMORISING VERSUS UNDERSTANDING

A student who memorises the answers to last year's exam scores 100% on last year's exam and 40% on this year's. A student who understands the material scores 85% on both. Training error is last year's exam. It is not a measure of learning; it is a measure of memory capacity, and every model has more of that than you think.

3.8 Cross-validation, step by step

The mechanics of 5-fold cross-validation, spelled out, because the order of operations is exactly where mistakes happen:

1. Shuffle the training data once (stratified by class if classifying) and cut it into 5 equal blocks.
2. For fold 1: hold out block 1. **Fit the scaler, the imputer, any feature selection and the model on blocks 2-5 only.** Transform block 1 with those fitted objects and score it.
3. Repeat for folds 2 to 5, each time refitting everything from scratch.
4. You now have 5 scores. Report their mean and standard deviation. The standard deviation is not decoration - it tells you whether a 0.3-point difference between two models is real.
5. Finally, refit the whole pipeline on all 5 blocks and use that as your model. The cross-validation was an estimate of how well this final model will do, not the model itself.

THE SINGLE MOST COMMON LEAK, IN ONE LINE OF CODE

`X = scaler.fit_transform(X)` written BEFORE the split. The scaler's mean and standard deviation now contain information from the test rows, so every model you evaluate afterwards has peeked. The score inflates by a little on large datasets and by a lot on small ones, and the inflation is invisible - it looks like a good result. Putting the scaler inside a Pipeline makes this class of bug structurally impossible.

3.9 How much data do I need?

There is no universal answer, but there are usable anchors, and a learning curve settles the question empirically for your problem in an afternoon:

Situation	Rough requirement
Linear or logistic model, d features	At least 10-20 samples per feature; more if classes are imbalanced
Gradient boosting on tabular data	A few thousand rows is often enough; it degrades gracefully below that
Small CNN trained from scratch	1,000-10,000 labelled images per class
Fine-tuning a pretrained backbone	50-500 images per class - two orders of magnitude less, which is why transfer learning dominates
Fine-tuning an LLM for style or format	500-5,000 high-quality instruction pairs
Training a foundation model from scratch	Do not; the cost is in the millions and the result is available for download

The reliable procedure: train on 10%, 25%, 50% and 100% of what you have, and plot validation error against sample count. If the curve is still descending steeply at 100%, more data is the cheapest improvement available. If it has flattened, more data will not help and you should spend the budget on features, capacity or better labels instead.

3.10 No free lunch, and what it means for you

The **No Free Lunch theorem** says that averaged over all possible problems, every learning algorithm performs identically. That sounds nihilistic; it is not. Real problems are not drawn uniformly from all possible problems - they have structure: smoothness, locality, compositionality, translation invariance. An algorithm wins when its **inductive bias** matches the structure of your data. Convolutions encode locality and translation invariance; recurrence encodes sequential order; attention encodes 'any position may matter'; trees encode axis-aligned thresholds; L1 encodes 'few features matter'. Choosing a model is choosing an assumption.

Exercises

1. Simulate the bias-variance decomposition: fit polynomials of degree 1, 3, 9 to 30 noisy samples of sin(x), repeat over 200 resampled datasets, and plot bias^2, variance and total error against degree.
2. Take any tabular dataset and deliberately introduce a target leak; measure the inflated CV score, then remove it and measure the honest one.
3. Explain in two sentences why 10-fold CV on a dataset with 20 patients and 2,000 sensor windows is likely to overstate accuracy dramatically.

4. Data: Collection, Cleaning, Features and Splits

Models are interchangeable; data is not. Two teams using the same architecture but different data pipelines routinely differ by more than the gap between architectures. This chapter is the least glamorous and the highest-leverage in Part I.

4.1 Types of data and what they demand

Type	Examples	Preferred models	Preprocessing
Tabular numeric	Sensor readings, prices	GBDT, linear, MLP	Scale for linear/NN; trees need nothing
Tabular categorical	Country, device type	GBDT (native), NN with embeddings	One-hot (low cardinality), target/embedding (high)
Text	Reviews, logs	Transformers, TF-IDF + linear	Tokenisation, subwords, truncation
Images	Photos, X-rays	CNN, ViT	Resize, normalise, augment
Audio	Speech, machine sound	CNN on spectrogram, Conformer	STFT / mel spectrogram, per-channel norm
Time series	IMU, ECG, demand	GBDT on windows, 1D-CNN, LSTM	Windowing, resampling, detrending
Graphs	Molecules, social	GNN	Adjacency, node features

4.2 Cleaning: missing values, outliers, duplicates

Missing data

- **MCAR** (missing completely at random): dropping rows is unbiased but wasteful.
- **MAR** (missing at random given observed features): impute using the other features - iterative/MICE imputation, or k-NN imputation.
- **MNAR** (missing not at random): the fact of being missing carries information - always add a binary `was_missing` indicator column, because 'income not stated' is itself predictive.

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

num = Pipeline([('imp', SimpleImputer(strategy='median', add_indicator=True)),
                ('sc',  StandardScaler())])
cat = Pipeline([('imp', SimpleImputer(strategy='most_frequent')),
                ('oh',  OneHotEncoder(handle_unknown='ignore',
                                      min_frequency=10))])

pre = ColumnTransformer([('num', num, numeric_cols),
                         ('cat', cat, categorical_cols)])
# pre is now a fit-on-train-only object; put it first in your model pipeline
```

Listing 4.1 - Leak-proof preprocessing with ColumnTransformer.

Outliers

Decide whether an extreme value is an **error** (a heart rate of 900 bpm) or a **rare truth** (a genuine 10x sale). Errors get removed or corrected; rare truths must stay, because they are often exactly what you are trying to detect. Detection tools: z-score for roughly Gaussian data, IQR fences for skewed data, isolation forest or local outlier factor for multivariate structure. Robust alternatives - median instead of mean, `RobustScaler`, Huber loss - are usually better than deletion.

4.3 Scaling and transformation

Transform	Formula	Use when
Standardisation	(x - mu) / sigma	Default for linear models, SVM, PCA, neural nets
Min-max	(x - min) / (max - min)	Bounded inputs, image pixels, when you need [0, 1]
Robust	(x - median) / IQR	Heavy outliers present
Log1p	log(1 + x)	Right-skewed positive quantities: counts, prices
Box-Cox / Yeo-Johnson	power transform	Make a variable approximately Gaussian
Quantile / rank	map to uniform or normal	Very non-linear distributions; robust to outliers

FIT ON TRAIN, APPLY TO TEST - ALWAYS

`scaler.fit_transform(X_train)` then `scaler.transform(X_test)`. Calling fit on the test set - or on the full dataset before splitting - is leakage. Inside cross-validation the scaler must be refit on each training fold, which is exactly what a Pipeline does for you and what hand-rolled code usually gets wrong.

4.4 Encoding categorical variables

- **One-hot:** safe, interpretable, explodes with cardinality. Cap rare levels with `min_frequency`.
- **Ordinal:** only when the categories genuinely have an order (small < medium < large). Using it for unordered categories injects a false ranking that linear models will happily believe.
- **Target / mean encoding:** replace a category with the mean target for that category. Powerful for high cardinality, and a leakage magnet - it must be computed out-of-fold, with smoothing towards the global mean.
- **Hashing:** fixed-width, streaming-friendly, collisions are usually tolerable.
- **Learned embeddings:** map each level to a trainable dense vector. The standard approach inside neural networks, and how recommender systems represent millions of item IDs.

4.5 Feature engineering that pays

- **Domain ratios and differences:** debt/income, price per square metre, acceleration magnitude sqrt(ax^2+ay^2+az^2). Almost always beat raw columns.
- **Time features:** hour, day of week, is-holiday, and crucially cyclic encodings sin(2 pi h/24), cos(2 pi h/24) so that 23:00 is close to 01:00.
- **Lags and rolling windows** for time series: value at t-1, t-7; rolling mean, std, min, max, slope. Compute strictly with past data.
- **Aggregations across entities:** per-user mean, count, recency. This is where most of the signal lives in behavioural data.
- **Interactions:** explicit products x_i * x_j for linear models; trees and networks find them on their own.
- **Text:** TF-IDF n-grams for a strong cheap baseline; sentence embeddings when semantics matter.

FEATURE WORK VERSUS MODEL WORK

On tabular problems, a day spent on features usually beats a week spent on architectures. On perceptual problems (images, audio, text), the reverse holds - the network learns better features than you can write, so spend the day on data quality, augmentation, and a better pretrained backbone instead.

4.6 Class imbalance

When 1 in 500 transactions is fraud, accuracy is meaningless and most losses are dominated by the majority class. Options, roughly in order of what to try first:

- Use the right metric: precision-recall AUC, F-beta, recall at fixed precision. Never accuracy.

- Class weights in the loss (`class_weight='balanced'`, `pos_weight` in PyTorch) - cheap and usually sufficient.
- Threshold tuning on the validation set: train normally, then choose the decision threshold that maximises your business metric. This is underused and often the single biggest win.
- Resampling: random undersampling of the majority, or SMOTE-style synthetic oversampling of the minority - only ever applied to the training fold, never to validation or test.
- Focal loss for extreme imbalance in detection tasks (Chapter 21).
- Reframe as anomaly detection if positives are both rare and diverse.

4.7 Data augmentation

Domain	Standard augmentations	Advanced
Images	Flip, crop, rotate, colour jitter, blur	RandAugment, Mixup, CutMix, CutOut, copy-paste
Audio	Time shift, noise, gain, speed	SpecAugment (mask time and frequency bands)
Text	Synonym swap, back-translation, dropout of tokens	LLM paraphrase, EDA, span corruption
Time series / IMU	Jitter, scaling, time warping, window slicing	Rotation of the sensor frame, magnitude warping, mixup
Tabular	Gaussian noise on numeric columns	SMOTE, CTGAN, feature dropout

AUGMENTATION ENCODES AN INVARIANCE

Every augmentation is a statement: 'the label does not change under this transformation'. Horizontal flip is right for cats and wrong for road signs with text; rotation is right for satellite images and wrong for handwritten digits (6 and 9). Choose augmentations that are true invariances of your task, and you are injecting free domain knowledge; choose false ones and you are injecting label noise.

4.8 A worked cleaning session

Below is a five-row extract from a realistic sensor dataset, with the problems a real file contains. Each column heading is followed by the decision it forces.

user_id	timestamp	hr_bpm	steps	device	label
A17	2024-03-01 08:00	72	1200	watch_v2	walking
A17	2024-03-01 08:01		1350	watch_v2	walking
B03	2024-03-01 08:01	910	0	Watch V2	sitting
B03	2024-03-01 08:02	68	-5	watch_v2	sitting
A17	2024-03-01 08:00	72	1200	watch_v2	walking

- **Missing heart rate (row 2):** not random - the sensor drops readings during motion, so missingness correlates with the label. Impute, and add an `hr_missing` indicator column; dropping the row would bias the dataset towards stationary activities.
- **910 bpm (row 3):** physiologically impossible, so it is an error, not a rare truth. Clip to a plausible range or mark as missing. Decide the rule from domain knowledge, never from the data alone.
- **-5 steps (row 4):** a counter reset or an integer underflow. Same treatment.
- **'Watch V2' versus 'watch_v2':** the same device in two spellings. Normalise case and whitespace, or you will train a model with two unrelated one-hot columns for one device.
- **Row 5 duplicates row 1:** exact duplicates inflate the effective weight of one moment and, if they land on opposite sides of a split, leak. Deduplicate on the natural key (user, timestamp).
- **user_id present:** never feed it as a feature, and always split by it. Otherwise the model learns 'A17 walks' rather than 'this signal means walking', and it will fail on every new user.

4.9 Windowing time series, correctly

Sensor and time-series data must be converted into fixed-length windows before most models can consume it. Three choices define the conversion, and each one has a trap.

Choice	Typical value	The trap
Window length	1-10 seconds for human activity; long enough to contain one cycle of the phenomenon	Too short and the pattern is not in the window at all
Overlap / stride	50% overlap is common for training	Overlapping windows across a train/test boundary share samples - a leak. Split by TIME or by SUBJECT first, then window each part separately
Label of a window	Majority label, or discard mixed windows	Windows straddling a transition carry two activities and add label noise

```
import numpy as np

def windows(sig, fs=50, sec=2.0, overlap=0.5):
    n = int(fs * sec); step = int(n * (1 - overlap))
    return np.stack([sig[i:i + n]
                     for i in range(0, len(sig) - n + 1, step)])

# Feature set per window that is hard to beat on IMU data:
def feats(w):                        # w: (n_samples, 3) for x, y, z
    mag = np.linalg.norm(w, axis=1)
    out = []
    for ch in [w[:, 0], w[:, 1], w[:, 2], mag]:
        out += [ch.mean(), ch.std(), ch.min(), ch.max(),
                np.percentile(ch, 25), np.percentile(ch, 75),
                np.abs(np.diff(ch)).mean(),          # mean abs change
                ((ch[:-1] * ch[1:]) < 0).sum()]      # zero crossings
    # frequency domain: dominant frequency and spectral energy
    P = np.abs(np.fft.rfft(mag)) ** 2
    out += [P[1:].argmax() + 1, P.sum(), (P / P.sum() *
            np.log(P / P.sum() + 1e-12)).sum() * -1]   # spectral entropy
    return np.array(out)
```

Listing 4.2 - Window extraction and a strong classical feature set. On many sensor tasks these features plus gradient boosting beat a deep network trained on raw signal, and they train in seconds.

4.10 Dataset documentation and ethics

- Record provenance, collection dates, licences, consent, and known population gaps - a datasheet for the dataset.
- Check subgroup balance and measure performance per subgroup, not only in aggregate. A model can be 95% accurate overall and 60% accurate for one group.
- Remove or protect personal data; prefer aggregation, hashing, or on-device processing (Chapter 31) when the data is sensitive.
- Version datasets with the same rigour as code: a result you cannot reproduce because 'the data changed' is not a result.

Exercises

1. Build a ColumnTransformer for a mixed dataset and confirm, by fitting on a subset, that no statistic of the test rows influences the transform.
2. Take an imbalanced dataset, plot the precision-recall curve, and find the threshold that maximises recall subject to precision above 0.8.
3. For a wearable-sensor dataset, list five window-level features you would compute and state, for each, why it is physically meaningful.

PART II

Classical Machine Learning

The algorithms that still win on tabular data, and the ideas that every deep model inherits: linear models, trees, kernels, ensembles, clustering and honest evaluation.

5. Linear Regression, Derived Completely

Linear regression is worth studying far beyond its own usefulness: it is the smallest model in which every concept - parameters, loss, gradients, closed-form solutions, regularisation, and the geometry of fitting - appears in a form you can fully verify by hand.

5.1 The model

```
y_hat = w_1 x_1 + w_2 x_2 + ... + w_d x_d + b   =   w . x + b

with a bias column of ones:   y_hat = X w,   X in R^(n x (d+1))
```

Folding the bias into the weight vector by appending a constant 1 feature keeps every formula below clean. The assumption embedded in this model is that the effect of each feature is additive and constant - one extra square metre adds the same amount of money whether the house is small or large. When that is false, you either transform features (log, splines, interactions) or move to a non-linear model.

5.2 The loss and where it comes from

```
J(w) = (1/n) ||X w - y||^2 = (1/n) SUM_i (x_i . w - y_i)^2
```

As shown in Chapter 2, squared error is the negative log-likelihood under Gaussian noise. That also tells you when it is the wrong choice: squared error punishes a single large error as much as many small ones, so outliers dominate the fit. Robust alternatives:

Loss	Formula	Behaviour
MSE	(1/n) SUM (y - y_hat)^2	Smooth; heavily influenced by outliers
MAE	(1/n) SUM |y - y_hat|	Robust; estimates the median; not differentiable at 0
Huber	quadratic within delta, linear beyond	Robust and smooth - usually the best default when outliers exist
Quantile / pinball	asymmetric absolute error	Predicts a chosen quantile; the basis of prediction intervals
Log-cosh	log cosh(y - y_hat)	Smooth approximation of Huber

5.3 Solution 1: the normal equations (closed form)

FULL DERIVATION

Write J(w) = (1/n)(Xw - y)^T (Xw - y) = (1/n)(w^T X^T X w - 2 w^T X^T y + y^T y). Differentiate with the matrix rules d(w^T A w)/dw = 2Aw for symmetric A, and d(w^T c)/dw = c. This gives grad J = (2/n)(X^T X w - X^T y). Setting the gradient to zero gives the NORMAL EQUATIONS X^T X w = X^T y, whose solution is w = (X^T X)^-1 X^T y whenever X^T X is invertible.

```
w* = (X^T X)^(-1) X^T y
```

Geometrically, `X w` ranges over the column space of X, and the best approximation to y in that subspace is its orthogonal projection - which is exactly what the normal equations state: the residual `Xw - y` is orthogonal to every column of X.

- Cost is O(n d^2 + d^3): fine for d in the hundreds, hopeless for d in the millions.
- `X^T X` is singular when features are perfectly collinear or when d > n. Never invert it explicitly - use `numpy.linalg.lstsq` or a QR/SVD-based solver, which handle rank deficiency gracefully.
- Ridge regression fixes singularity outright: `w = (X^T X + lam I)^-1 X^T y` is always invertible for lam > 0 (Chapter 7).

5.4 Solution 2: gradient descent (the scalable way)

```
grad J(w) = (2/n) X^T (X w - y)

w <- w - eta * grad J(w)
```

```python
import numpy as np

def fit_linear_gd(X, y, lr=0.05, epochs=500, batch=64, seed=0):
    rng = np.random.default_rng(seed)
    n, d = X.shape
    Xb = np.hstack([X, np.ones((n, 1))])          # bias trick
    w = np.zeros(d + 1)
    hist = []
    for ep in range(epochs):
        idx = rng.permutation(n)
        for s in range(0, n, batch):
            b = idx[s:s + batch]
            resid = Xb[b] @ w - y[b]
            grad = 2.0 / len(b) * Xb[b].T @ resid
            w -= lr * grad
        hist.append(np.mean((Xb @ w - y) ** 2))
    return w, hist

# Sanity check against the closed form on standardised data:
# w_closed = np.linalg.lstsq(Xb, y, rcond=None)[0]
```

Listing 5.1 - Mini-batch gradient descent for linear regression, 15 lines.

FEATURE SCALING IS NOT OPTIONAL FOR GRADIENT DESCENT

If one feature ranges over [0, 1] and another over [0, 100000], the loss surface is a long thin valley: the learning rate that is stable for the steep direction is far too small for the flat one, and training crawls. Standardise. The closed form does not care about scaling; gradient descent cares enormously.

5.5 Interpreting and validating a linear model

Coefficients

With standardised features, `w_j` is the expected change in y for a one-standard-deviation increase in feature j, **holding the other features fixed**. That last clause is where interpretations usually go wrong: with correlated features, the coefficients split the shared effect arbitrarily and can even flip sign. Check the **variance inflation factor**; a VIF above 5-10 signals multicollinearity, and ridge regularisation or dropping redundant features is the cure.

Goodness of fit

```
R^2      = 1 - SS_res / SS_tot

adj R^2  = 1 - (1 - R^2)(n - 1)/(n - d - 1)

RMSE     = sqrt( (1/n) SUM (y - y_hat)^2 )     (same units as y)

MAPE     = (100/n) SUM |y - y_hat| / |y|       (beware y near 0)
```

R^2 always increases when you add features, which is why adjusted R^2 exists. Report RMSE or MAE alongside it - stakeholders understand 'the prediction is off by 12,000 EUR on average' far better than 'R^2 is 0.83'.

Residual diagnostics

- Residuals versus fitted values should look like a formless cloud. A funnel shape means **heteroscedasticity** - consider modelling log(y) or using weighted least squares.
- A curve in the residuals means a missing non-linearity - add a polynomial term, a spline, or switch to a tree model.
- A Q-Q plot far from the diagonal means non-Gaussian errors; prediction intervals from ordinary least squares will be wrong.
- Autocorrelated residuals in time series mean the model missed temporal structure; add lags.

5.6 Polynomial and basis-function regression

Linear regression is linear **in the parameters**, not in the inputs. Replacing x by a basis expansion phi(x) keeps every formula above intact while fitting curves:

```
y_hat = w_0 + w_1 phi_1(x) + ... + w_m phi_m(x)
```

- Polynomials: simple, but high degrees oscillate wildly near the edges (Runge's phenomenon).
- Splines: piecewise polynomials with continuity constraints - far better behaved than a single high-degree polynomial and the standard choice in statistics.
- Radial basis functions: exp(-gamma ||x - c||^2) around chosen centres; this is the bridge to kernel methods in Chapter 10.

THE FIRST APPEARANCE OF THE CAPACITY DIAL

Degree 1 underfits a curve; degree 15 on 20 points passes through every point and is useless between them. The degree is a hyperparameter, chosen on validation data, and it is the simplest possible instance of the bias-variance trade-off from Chapter 3.

5.7 The five houses, solved completely by hand

Chapter 1 quoted a fitted line without deriving it. Here is the whole computation, with every number, so that the closed-form solution stops being a formula you trust and becomes one you can check.

i	size x_i	price y_i	x_i - x_bar	y_i - y_bar	(x-x_bar)(y-y_bar)	(x-x_bar)^2
1	50	150	-40	-102	4080	1600
2	70	195	-20	-57	1140	400
3	90	260	0	8	0	0
4	110	300	20	48	960	400
5	130	355	40	103	4120	1600
sum	450	1260	0	0	10300	4000
mean	90	252	-	-	-	-

```
w = SUM (x-x_bar)(y-y_bar) / SUM (x-x_bar)^2  =  10300 / 4000  =  2.575

b = y_bar - w * x_bar  =  252 - 2.575 * 90  =  20.25
```

So the fitted line is `price = 2.575 * size + 20.25`: each square metre is worth 2,575 EUR, and the intercept of 20.25k is the model's guess for a house of zero size - a reminder that the intercept is often meaningless in isolation and exists only to position the line.

Size	Actual	Predicted	Residual
50	150	149.00	+1.00
70	195	200.50	-5.50

Size	Actual	Predicted	Residual
90	260	252.00	+8.00
110	300	303.50	-3.50
130	355	355.00	0.00

```
MSE  = (1.00^2 + 5.50^2 + 8.00^2 + 3.50^2 + 0^2) / 5 = 21.5
RMSE = sqrt(21.5) = 4.64  (thousand EUR - the natural error unit)
R^2  = 1 - 107.5 / 26630 = 0.996
```

Two sanity checks worth internalising. The residuals sum to zero - that is a mathematical consequence of fitting an intercept by least squares, and if yours do not, you have a bug. And RMSE is in the units of the target, which is why it is the number to quote to a stakeholder; R^2 = 0.996 sounds impressive but says nothing about whether being off by 4,600 EUR is acceptable.

WHY THE FORMULA LOOKS LIKE THAT

Set the derivative of J to zero. dJ/db = 0 gives b = y_bar - w x_bar, i.e. the line passes through the centre of mass of the data. Substituting that back into dJ/dw = 0 gives w = Cov(x,y)/Var(x). So the slope is literally 'how much y moves with x, divided by how much x moves on its own'. Every regression coefficient in this book is a version of that ratio.

5.8 The same fit by gradient descent, step by step

With standardised features (x_std = (x - 90)/28.28) and a learning rate of 0.1, starting from w = b = 0:

Step	w	b	J (MSE)	dJ/dw	dJ/db
0	0.00	0.00	68830	-145.7	-504.0
1	14.57	50.40	44059	-116.5	-403.2
2	26.22	90.72	28205	-93.2	-322.6
3	35.54	122.98	18059	-74.6	-258.0
4	43.00	148.78	11566	-59.7	-206.4
...	...	...	...	...	...
converged	72.83	252.00	21.5	0.0	0.0

It arrives at the same solution the normal equations gave in one line - on standardised inputs the converged slope 72.83 corresponds to 72.83 / 28.28 = 2.575 in original units, and the converged intercept is the mean price. Gradient descent is slower here and would be absurd for five points; its advantage appears when there are ten million rows and ten thousand features, where the matrix inverse is impossible and each step costs only one pass over a mini-batch.

WHY THE GRADIENTS START SO LARGE

At w = b = 0 the model predicts 0 for every house, so residuals are around -250 and the squared loss is 68,830. Large loss means large gradient means large first steps - which is exactly why an untuned learning rate diverges most often in the first few iterations, and why warmup (Chapter 17) exists.

5.9 From least squares to probability

Ordinary least squares gives a point prediction. Two upgrades are worth knowing:

- **Bayesian linear regression** places a Gaussian prior on w and returns a posterior distribution rather than a point, giving calibrated predictive intervals that widen where data is sparse. With a Gaussian prior of variance tau^2 the posterior mean is exactly the ridge solution - regularisation is a prior in disguise.
- **Generalised linear models** keep the linear predictor `eta = w.x` and pass it through a link function: identity for regression, logit for binary classification (next chapter), log for Poisson counts. Same machinery, different noise assumption.

Exercises

1. Implement the normal equations and compare against `lstsq` on a matrix with two perfectly correlated columns. Explain the failure.
2. Fit polynomial degrees 1..15 on 25 noisy points from a cubic, plot train and validation RMSE, and identify the sweet spot.
3. Derive the ridge solution by adding lam ||w||^2 to J and repeating the matrix derivation above.

6. Logistic and Softmax Regression

Despite the name, logistic regression is a **classifier**. It is the workhorse baseline for binary problems, the last layer of almost every neural classifier, and the cleanest place to learn cross-entropy and the logit view of probability.

6.1 From linear score to probability

A linear model produces an unbounded score `z = w.x + b`, called the **logit**. The **sigmoid** squashes it into (0, 1):

```
sigma(z) = 1 / (1 + exp(-z))

sigma'(z) = sigma(z) (1 - sigma(z))

logit(p) = log( p / (1 - p) )      <- inverse of sigmoid
```

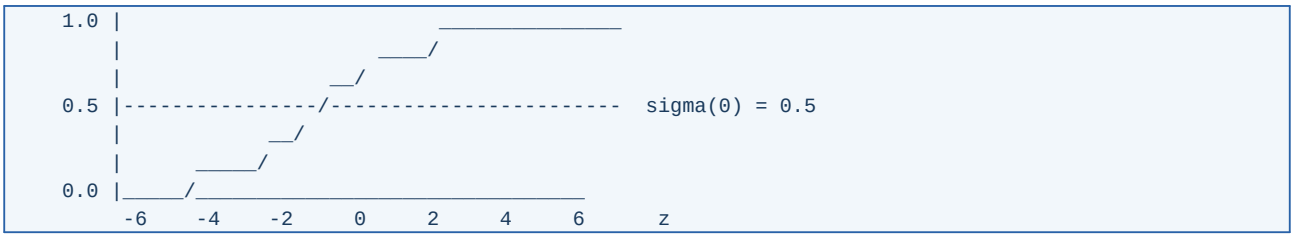

Figure 6.1 - The logistic function: linear in the middle, saturating at the ends.

The saturation is important twice over: it keeps probabilities in range, and it causes **vanishing gradients** when |z| is large, because sigma' approaches zero. That is why sigmoid is no longer used as a hidden activation in deep networks (Chapter 16), only as an output.

6.2 The loss: binary cross-entropy

DERIVATION AND THE CLEAN GRADIENT

Model P(y=1|x) = p = sigma(z). The Bernoulli likelihood of one sample is p^y (1-p)^(1-y). Taking the negative log gives L = -[y log p + (1-y) log(1-p)]. Now differentiate with respect to the LOGIT: dL/dp = -(y/p) + (1-y)/(1-p), and dp/dz = p(1-p). Multiplying, everything cancels: dL/dz = p - y. Therefore dL/dw = (p - y) x. The gradient is (prediction minus truth) times the input - identical in form to linear regression, and the reason cross-entropy pairs so well with sigmoid: the saturating derivative in the activation is cancelled by the logarithm in the loss.

```
J(w) = -(1/n) SUM_i [ y_i log p_i + (1 - y_i) log(1 - p_i) ]

grad J = (1/n) X^T (p - y)
```

NEVER SQUARE-ERROR A CLASSIFIER

Using MSE with a sigmoid output multiplies the loss gradient by sigma'(z), which is near zero exactly when the model is confidently wrong - so the worst mistakes produce the smallest updates and learning stalls. Cross-entropy removes that factor. Also: always feed raw logits to `BCEWithLogitsLoss` / `CrossEntropyLoss` rather than applying sigmoid or softmax yourself, so the library can use the numerically stable formulation.

6.3 Multiclass: softmax regression

```
z = W x + b,      z in R^K

softmax(z)_k = exp(z_k) / SUM_j exp(z_j)

L = -SUM_k y_k log softmax(z)_k = -log softmax(z)_[true class]

dL/dz = softmax(z) - y      (one-hot y)
```

Softmax is the natural generalisation of sigmoid to K classes, and the gradient keeps the same beautiful form. Two practical notes: softmax is shift-invariant, `softmax(z + c) = softmax(z)`, which is what makes the max-subtraction trick safe; and the outputs are coupled - raising one logit lowers every other probability. For **multi-label** problems, where several classes can be true at once, use K independent sigmoids instead.

```
import numpy as np

def softmax(Z):                        # Z: (n, K) logits
    Z = Z - Z.max(axis=1, keepdims=True)      # numerical stability
    E = np.exp(Z)
    return E / E.sum(axis=1, keepdims=True)

def fit_softmax(X, Y1h, lr=0.1, epochs=200, l2=1e-4):
    n, d = X.shape; K = Y1h.shape[1]
    W = np.zeros((d, K)); b = np.zeros(K)
    for _ in range(epochs):
        P = softmax(X @ W + b)
        G = (P - Y1h) / n                     # dL/dz, the whole trick
        W -= lr * (X.T @ G + l2 * W)
        b -= lr * G.sum(axis=0)
    return W, b
```

Listing 6.1 - Softmax regression in NumPy; this is also the output layer of every classification network in Part III.

6.4 Decision boundary, odds, and interpretation

The boundary is the set where `p = 0.5`, i.e. `w.x + b = 0` - a hyperplane. Logistic regression can therefore only separate classes linearly; curved boundaries require feature expansion or a non-linear model. Its interpretability comes from odds:

```
log( p / (1-p) ) = w.x + b

exp(w_j) = odds ratio: the multiplicative change in odds per unit of x_j
```

A coefficient of 0.7 on 'smoker' means the odds of the outcome are exp(0.7) = 2.0 times higher for smokers, all else equal. This direct reading is why logistic regression remains the standard model in medicine, credit scoring and any regulated setting.

6.5 Logistic regression on four points, by hand

Four samples with one feature: x = 1, 2 label 0; x = 3, 4 label 1. Start from w = 0, b = 0 with a learning rate of 0.5.

Step	w	b	Predictions p	Loss J	Gradient (w, b)
0	0.000	0.000	0.50, 0.50, 0.50, 0.50	0.693	(-0.500, 0.000)
1	0.250	0.000	0.56, 0.62, 0.68, 0.73	0.625	(-0.058, 0.149)
2	0.279	-0.074	0.55, 0.62, 0.68, 0.74	0.612	(-0.053, 0.148)
converged	5.80	-14.32	0.00, 0.00, 1.00, 1.00	~0	(0, 0)

Read the first row carefully, because it is the whole of Chapter 6 in one line. At w = b = 0 every prediction is 0.5 and the loss is -log(0.5) = 0.693 - which is the loss any binary classifier has before it learns anything, and therefore the number your training log should start near. The gradient with respect to w is the average of `(p -`

`y) * x` = (0.5*1 + 0.5*2 - 0.5*3 - 0.5*4)/4 = -0.5: negative, so w increases, so the score rises with x, which is exactly the right direction because the positive class sits at large x.

WHY THE CONVERGED WEIGHTS ARE SO LARGE

This data is perfectly separable, so the likelihood keeps improving as the boundary gets steeper: w grows without bound and the model becomes infinitely confident. With any regularisation at all (scikit-learn's default C = 1) w stops at a modest value. Unbounded weights on separable data is not a curiosity - it is why an unregularised logistic model can produce probabilities of 0.99999 that mean nothing.

```
Decision boundary:  w x + b = 0  =>  x = -b/w = 14.32/5.80 = 2.47

Odds ratio per unit of x:  exp(w) = exp(5.80) = 331
```

6.6 Sigmoid, logit and probability: three views of one number

Logit z	Probability sigma(z)	Odds p/(1-p)	Reading
-4	0.018	1 : 55	Almost certainly negative
-2	0.119	1 : 7.4	Probably negative
-1	0.269	1 : 2.7	Leaning negative
0	0.500	1 : 1	No information
+1	0.731	2.7 : 1	Leaning positive
+2	0.881	7.4 : 1	Probably positive
+4	0.982	55 : 1	Almost certainly positive

Two facts to carry forward. Adding 1 to the logit always multiplies the odds by e = 2.718, whatever the starting point - that constancy is what makes coefficients interpretable. And beyond about |z| = 4 the probability barely moves while the logit keeps growing, which is the saturation that kills gradients in deep sigmoid networks.

6.7 Practical matters

- **Regularisation is on by default** in scikit-learn (`C` is the inverse of the penalty strength). With separable data and no penalty, weights diverge to infinity - the likelihood keeps improving as the margin grows.
- **Solvers:** `lbfgs` for small dense problems, `saga` for large sparse ones or L1/elastic-net penalties, `liblinear` for small L1 problems.
- **Class imbalance:** `class_weight='balanced'` reweights the loss; then tune the decision threshold on validation data rather than accepting 0.5.
- **Calibration:** logistic regression is usually well calibrated out of the box, which is exactly why Platt scaling - fitting a logistic regression on another model's scores - is the standard calibration method (Chapter 13).
- **Multiclass strategies:** true multinomial softmax is preferred; one-vs-rest trains K binary models and is a reasonable fallback for models that cannot do multinomial natively.

Exercises

1. Derive dL/dz = p - y for softmax by differentiating log-sum-exp. Watch the two cases k = true class and k != true class.
2. Train logistic regression on a linearly separable 2-D dataset with the penalty switched off and plot the norm of w against iterations.
3. Fit a logistic model on an imbalanced dataset, then sweep the threshold from 0 to 1 and plot precision and recall against it.

7. Regularization and Model Selection

Regularisation is any modification to a learning algorithm intended to reduce test error but not training error. It is how you buy generalisation when you cannot buy more data.

7.1 Penalty-based regularisation

```
Ridge  (L2):    J = MSE + lam * SUM_j w_j^2

Lasso  (L1):    J = MSE + lam * SUM_j |w_j|

Elastic net:    J = MSE + lam * ( a * SUM |w_j| + (1-a) * SUM w_j^2 )
```

Property	Ridge (L2)	Lasso (L1)
Effect on coefficients	Shrinks all towards zero, none exactly zero	Drives many exactly to zero
Feature selection	No	Yes - built in
Correlated features	Splits weight evenly among them	Arbitrarily picks one, drops the rest
Solution	Closed form: (X^T X + lam I)^-1 X^T y	No closed form; coordinate descent or proximal methods
Bayesian reading	Gaussian prior on w	Laplace prior on w
When to prefer	Many small effects, multicollinearity	You believe few features matter and want a sparse model

WHY L1 PRODUCES EXACT ZEROS

Think of the constrained form: minimise MSE subject to ||w||_1 <= t. The L1 ball is a diamond with corners ON the axes; the elliptical contours of the MSE touch it, in general, at a corner - and a corner has one or more coordinates exactly zero. The L2 ball is round, has no corners, and the contact point almost never lies on an axis. Equivalently: the subgradient of |w| is a constant +/-1 all the way to zero, so shrinkage does not weaken as w gets small, whereas the L2 gradient 2w fades away.

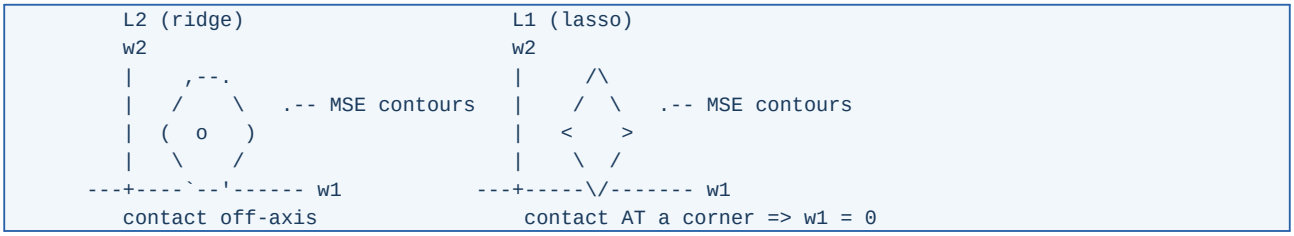

Figure 7.1 - The geometry that makes lasso a feature selector.

7.2 Other forms of regularisation

- **Early stopping:** stop when validation loss stops improving. For linear models trained by gradient descent this is provably similar to L2.
- **Data augmentation:** more effective than any penalty when you can define true invariances (Chapter 4).
- **Noise injection:** adding Gaussian noise to inputs is equivalent to L2 on the weights for linear models; adding it to weights or activations flattens the minima found.
- **Dropout, label smoothing, weight decay, stochastic depth:** the deep-learning-specific toolkit, Chapter 19.
- **Ensembling:** averaging independently trained models reduces variance directly, Chapter 11.
- **Parameter sharing:** convolution is a hard constraint that the same weights apply everywhere - one of the strongest regularisers in existence, and it is architectural rather than a penalty.

7.3 Choosing hyperparameters

Method	How it works	When to use
Grid search	Exhaustive over a discrete grid	Few hyperparameters (<= 3), cheap models
Random search	Sample from distributions for a fixed budget	The right default: better than grid when only a few dimensions matter
Bayesian optimisation	Fit a surrogate model of the objective, sample where improvement is expected (Optuna, scikit-optimize)	Expensive training runs, many hyperparameters
Hyperband / ASHA	Start many configs, kill the weak ones early	Deep learning, where a bad config is obvious after 2 epochs
Population-based training	Evolve a population, copy and perturb winners	Long RL and large-model runs with schedules

SEARCH IN LOG SPACE, AND SEARCH THE RIGHT THINGS

Learning rate, regularisation strength and layer widths should be sampled log-uniformly (1e-5 to 1e-1), not uniformly. Random search beats grid search because performance usually depends strongly on one or two hyperparameters and weakly on the rest - random search gives you many distinct values of the important one, grid search wastes the budget repeating them.

```
import numpy as np, optuna
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.ensemble import HistGradientBoostingClassifier

def objective(trial):
    params = dict(
        learning_rate = trial.suggest_float('lr', 1e-3, 3e-1, log=True),
        max_leaf_nodes= trial.suggest_int('leaves', 8, 256, log=True),
        min_samples_leaf = trial.suggest_int('min_leaf', 5, 200, log=True),
        l2_regularization= trial.suggest_float('l2', 1e-8, 10.0, log=True),
        max_iter = 500)
    model = HistGradientBoostingClassifier(early_stopping=True, **params)
    cv = StratifiedKFold(5, shuffle=True, random_state=0)
    return cross_val_score(model, X, y, cv=cv, scoring='roc_auc').mean()

study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=60, n_jobs=4)
print(study.best_params, round(study.best_value, 4))
```

Listing 7.1 - Bayesian hyperparameter search with correct log-scale ranges.

7.4 Model selection criteria without a validation set

$$AIC = 2k - 2 \log L \qquad \text{(prediction-oriented)}$$

$$BIC = k \log n - 2 \log L \qquad \text{(penalises complexity harder; consistent)}$$

$$MDL: \text{choose the model that compresses data + model description best}$$

These are useful when data is too scarce to hold out, and in classical statistics generally. In modern practice, cross-validation is preferred because it makes no distributional assumptions - but AIC/BIC remain the standard in time-series order selection and in fields where models are fitted by maximum likelihood.

7.5 A disciplined selection protocol

Before you report a number

- The test set was split off before any exploration and touched once.
- Every data-dependent step (imputation, scaling, selection, encoding) lives inside the cross-validation loop.

- Hyperparameters were chosen on validation folds, not on the test set.
- Results are averaged over at least 3-5 random seeds, and you report the spread, not only the mean.
- You compare against a trivial baseline (majority class, mean predictor, last-value-carried-forward) and a simple strong baseline (ridge or gradient boosting).
- The difference you are claiming is larger than the seed-to-seed noise.

Exercises

1. Plot the lasso regularisation path (coefficients versus lam) on a dataset with 20 features, 5 of which are informative.
2. Compare grid search and random search on the same budget of 50 trials for a model with 4 hyperparameters, one of which is irrelevant.
3. Show empirically that ridge with lam -> 0 recovers ordinary least squares, and that lam -> infinity drives predictions to the mean.

8. Instance-Based and Probabilistic Models

8.1 k-Nearest Neighbours

k-NN does no training at all. To predict, it finds the k closest training samples and takes a majority vote (classification) or an average (regression). It is the purest form of **non-parametric** learning: the training data *is* the model.

```
y_hat(x) = majority{ y_i : x_i in N_k(x) }        (classification)

y_hat(x) = (1/k) SUM_(i in N_k(x)) y_i             (regression)
```

Aspect	Detail
Hyperparameters	k, distance metric, weighting (uniform or 1/d)
k small	Low bias, high variance; k = 1 fits training data perfectly and is very sensitive to noise
k large	High bias, low variance; k = n predicts the global mean
Distance	Euclidean by default; Manhattan for high dimensions; cosine for embeddings and text; Hamming for binary
Cost	O(1) to train, O(n d) per query - the opposite profile of most models. KD-trees and ball-trees help in low dimensions; HNSW and IVF indexes are what production vector search actually uses
Must-do	Scale the features. An unscaled feature with a large range silently dominates the distance

THE CURSE OF DIMENSIONALITY, CONCRETELY

In high dimensions, all pairwise distances concentrate: the ratio (farthest - nearest)/nearest tends to zero as d grows. With d = 100 roughly uniform features, 'nearest neighbour' stops meaning anything. Also, to keep the same density you need exponentially more samples: covering [0,1]^d at resolution 0.1 requires 10^d points. k-NN is excellent for d up to about 10-20 with enough data, and unreliable far beyond that unless you first reduce dimensionality or use a learned embedding.

k-NN is still deeply relevant: retrieval-augmented generation, recommendation, deduplication, few-shot classification with foundation-model embeddings, and face recognition are all approximate nearest neighbour search over learned vectors. The lesson is that k-NN works wonderfully **once the representation is good** - which is exactly what deep learning provides.

8.2 Naive Bayes

A generative classifier built directly on Bayes' rule, with one strong simplifying assumption: features are conditionally independent given the class.

```
P(y | x) proportional to P(y) PROD_j P(x_j | y)

y_hat = argmax_y [ log P(y) + SUM_j log P(x_j | y) ]
```

- **Gaussian NB:** each P(x_j | y) is a normal distribution - continuous features.
- **Multinomial NB:** counts - the classic text classifier over bag-of-words.
- **Bernoulli NB:** binary presence/absence features.
- **Laplace (add-one) smoothing** is mandatory: a single unseen word-class pair would otherwise make the whole product zero.

The independence assumption is almost always false, yet Naive Bayes often classifies well, because it only needs the *argmax* to be right, not the probabilities. Its probability estimates, however, are badly calibrated - typically pushed towards 0 or 1. Use it as a fast baseline on text, as a component in streaming systems, or when you have very little data; do not trust its confidence values without calibration.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import make_pipeline

clf = make_pipeline(TfidfVectorizer(ngram_range=(1, 2), min_df=2,
                                    sublinear_tf=True),
                    MultinomialNB(alpha=0.3))       # alpha = smoothing
clf.fit(train_texts, train_labels)
# Trains on 100k documents in about a second and is a genuinely strong
# baseline that any transformer should be required to beat.
```

Listing 8.1 - The five-second text-classification baseline.

8.3 Linear and quadratic discriminant analysis

LDA models each class as a Gaussian with a **shared** covariance matrix; the resulting decision boundary is linear. QDA gives each class its own covariance and yields quadratic boundaries at the cost of many more parameters. LDA doubles as a supervised dimensionality reduction method: it projects onto at most K-1 directions that maximise between-class scatter relative to within-class scatter - a useful contrast with PCA, which ignores labels entirely (Chapter 12).

Exercises

1. Plot k-NN test accuracy against k from 1 to 50 and mark the bias-variance regimes on the curve.
2. Demonstrate distance concentration: sample 1,000 points uniformly in [0,1]^d for d = 2, 10, 100, 1000 and plot the ratio of maximum to minimum pairwise distance.
3. Compare Multinomial Naive Bayes and logistic regression on a text dataset of 1,000 documents and again on 100,000 - explain the crossover.

9. Decision Trees

A decision tree asks a sequence of yes/no questions about the features until it reaches a leaf that holds a prediction. It is the only major model whose reasoning a non-technical stakeholder can read directly - and it is the building block of the ensembles in Chapter 11, which are still the best general-purpose tabular models available.

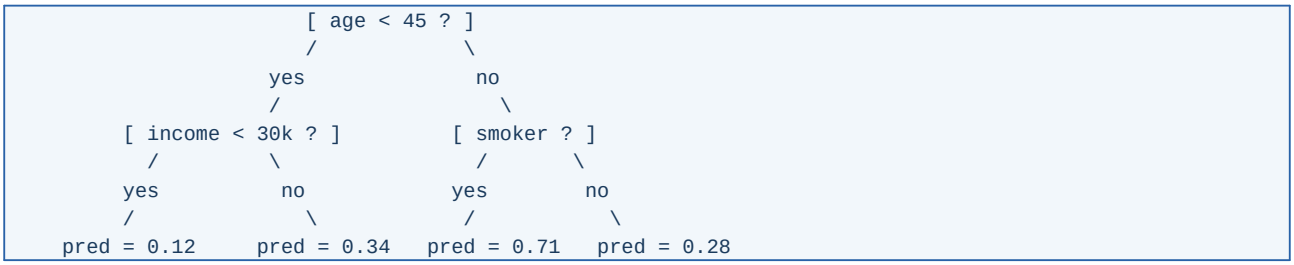

Figure 9.1 - A depth-2 tree partitions feature space into axis-aligned boxes.

9.1 How a split is chosen

At each node the algorithm considers every feature and every candidate threshold, and picks the split that most reduces an impurity measure. Cost is O(n d log n) per level with sorted features.

```
Gini      G = 1 - SUM_k p_k^2

Entropy   H = -SUM_k p_k log2 p_k

Gain      IG = Impurity(parent) - SUM_child (n_child/n_parent) * Impurity(child)

Regression: MSE reduction, i.e. variance reduction
```

A CONCRETE SPLIT CALCULATION

A node has 100 samples: 60 positive, 40 negative. Gini = 1 - (0.6^2 + 0.4^2) = 0.48. A candidate split gives child A with 50 samples (45 pos, 5 neg, Gini = 1 - 0.9^2 - 0.1^2 = 0.18) and child B with 50 samples (15 pos, 35 neg, Gini = 1 - 0.3^2 - 0.7^2 = 0.42). Weighted child impurity = 0.5*0.18 + 0.5*0.42 = 0.30, so the gain is 0.18. The split with the largest gain across all features and thresholds wins.

Gini and entropy almost always select the same splits; Gini is marginally cheaper (no logarithm) and is the default in most libraries. Neither is worth tuning.

9.2 Controlling growth

An unconstrained tree grows until every leaf is pure - it memorises the training set and has near-zero bias and enormous variance. Control it with:

Hyperparameter	Effect	Sensible start
max_depth	Hard cap on tree depth	3-10 for a single tree
min_samples_leaf	Minimum samples in a leaf; the most effective single knob	1-5% of n
min_samples_split	Minimum samples required to split a node	20+
max_features	Features considered per split - the source of diversity in random forests	sqrt(d) for classification
ccp_alpha	Cost-complexity (post-)pruning strength	Tune by CV
max_leaf_nodes	Best-first growth with a leaf budget	31-255 in boosting

Cost-complexity pruning grows a large tree, then removes the subtree whose removal costs the least error per removed leaf, using `R_alpha(T) = R(T) + alpha |leaves(T)|`. Sweeping alpha produces a nested

sequence of trees; cross-validation picks one. Post-pruning generally beats early stopping, because a weak split can enable a strong one beneath it.

9.3 Strengths, weaknesses, and the properties that matter downstream

Strengths	Weaknesses
No scaling or normalisation needed	High variance: a small data change can restructure the whole tree
Handles mixed numeric and categorical data	Axis-aligned splits only; a diagonal boundary needs a staircase of splits
Captures interactions and non-linearities automatically	Cannot extrapolate beyond the training range - predictions are constant outside it
Robust to outliers in the inputs	Biased towards features with many distinct values when using naive impurity gain
Missing values handled natively in modern implementations	A single tree rarely competitive alone - use ensembles

WHY TREES DOMINATE TABULAR DATA

Real tabular features are heterogeneous - different units, skewed distributions, irrelevant columns, non-smooth relationships with thresholds ('approve if credit score above 700'). Trees are invariant to monotone transformations of each feature, ignore irrelevant features cheaply, and model thresholds exactly. Neural networks have to learn all of that from data, which is why they need more of it to reach the same point on tabular problems.

9.4 A complete split search on a tiny dataset

Ten samples, one feature (age), one binary label (bought). The algorithm sorts by the feature, considers each midpoint between adjacent distinct values as a candidate threshold, and scores them all.

Age	22	25	28	33	37	41	45	52	58	63
Bought	0	0	0	1	0	1	1	1	1	1

Parent impurity: the ten samples split 4 zeros and 6 ones, so Gini = 1 - 0.4^2 - 0.6^2 = 0.48.

Threshold	Left (n, ones)	Gini L	Right (n, ones)	Gini R	Weighted	Gain
age < 26.5	2, 0	0.000	8, 6	0.375	0.300	0.180
age < 30.5	3, 0	0.000	7, 6	0.245	0.171	0.309
age < 35.0	4, 1	0.375	6, 5	0.278	0.317	0.163
age < 39.0	5, 1	0.320	5, 5	0.000	0.160	0.320
age < 43.0	6, 2	0.444	4, 4	0.000	0.267	0.213
age < 48.5	7, 3	0.490	3, 3	0.000	0.343	0.137

The winner is `age < 39.0` with a gain of 0.320. Look at why it beats `age < 30.5`, which also looks natural: both isolate a clean group, but the 39.0 split makes the right child perfectly pure (five ones, Gini 0) while leaving only one misplaced sample on the left, whereas the 30.5 split leaves the awkward 37-year-old buried in a seven-sample child that stays impure. Impurity gain is a weighted average, so it rewards making one LARGE child clean over making one small child clean.

Note also what the algorithm never considered: a rule such as `33 <= age <= 45`. A single split is always one threshold on one feature, so every band, diagonal or interaction has to be assembled from nested splits. That is why trees need depth to express what a linear model states in one coefficient, and why a diagonal boundary comes out of a tree as a staircase.

DOING THE ARITHMETIC FOR ONE ROW YOURSELF

Take `age < 39.0`. The left child holds ages 22, 25, 28, 33, 37 with labels 0, 0, 0, 1, 0 - one positive out of five, so Gini = 1 - 0.2^2 - 0.8^2 = 0.32. The right child holds 41, 45, 52, 58, 63, all positive, so Gini = 0. Weighted impurity = (5/10)(0.32) + (5/10)(0) = 0.16, and the gain is 0.48 - 0.16 = 0.32. Re-derive one more row from the table and the algorithm will never be mysterious again - a decision tree is this loop, repeated over every feature and every threshold, then recursed on each child.

9.5 Reading a tree out loud

A trained tree is a set of nested if-statements, and you should be able to convert one into English. The tree in Figure 9.1 says:

- If the customer is under 45 and earns under 30k, predicted probability 0.12 - the low-risk group.
- If under 45 and earning 30k or more, 0.34.
- If 45 or over and a smoker, 0.71 - the highest-risk leaf.
- If 45 or over and not a smoker, 0.28.

Each leaf value is simply the fraction of positive training samples that landed in that leaf. That is worth stating plainly because it explains two behaviours: leaves with few samples give extreme, unreliable probabilities (hence `min_samples_leaf`), and a tree can never predict a value it did not see in training - it cannot extrapolate above the highest leaf mean, which is why trees are poor at trending time series.

9.6 Feature importance - and how it misleads

- **Impurity-based (Gini) importance:** free, but biased towards high-cardinality and continuous features, and computed on training data. Treat it as a rough hint only.
- **Permutation importance:** shuffle one column on held-out data and measure the drop in score. Model-agnostic and much more trustworthy; but correlated features share credit and can both appear unimportant.
- **SHAP values:** a game-theoretic attribution with strong consistency guarantees, with an exact fast algorithm for trees (TreeSHAP). The current standard for explaining tabular models, both globally and per prediction (Chapter 33).

```
from sklearn.inspection import permutation_importance
r = permutation_importance(model, X_val, y_val, n_repeats=20,
                           scoring='roc_auc', random_state=0)
order = r.importances_mean.argsort()[::-1]
for i in order[:10]:
    print(f'{feature_names[i]:30s} {r.importances_mean[i]:.4f}'
          f' +/- {r.importances_std[i]:.4f}')
```

Listing 9.1 - Permutation importance measured on validation data.

Exercises

1. Implement CART for classification in NumPy: recursive best-split search with Gini, plus max_depth and min_samples_leaf. 80 lines is enough.
2. Fit an unpruned tree and a pruned one on the same data and compare training/test accuracy and the number of leaves.
3. Show that a decision tree cannot represent y = (x1 > x2) compactly, and explain what feature you would add to fix it.

10. Support Vector Machines and Kernels

SVMs were the state of the art before deep learning, and they remain the best explanation of two ideas that keep reappearing: **margin** as a principled notion of confidence, and the **kernel trick** as a way to work in a high-dimensional space without ever visiting it.

10.1 Maximum margin classification

Among the infinitely many hyperplanes that separate two classes, the SVM chooses the one whose distance to the nearest point of either class - the **margin** - is largest. A large margin is a robustness statement: the boundary can tolerate perturbation before misclassifying anything.

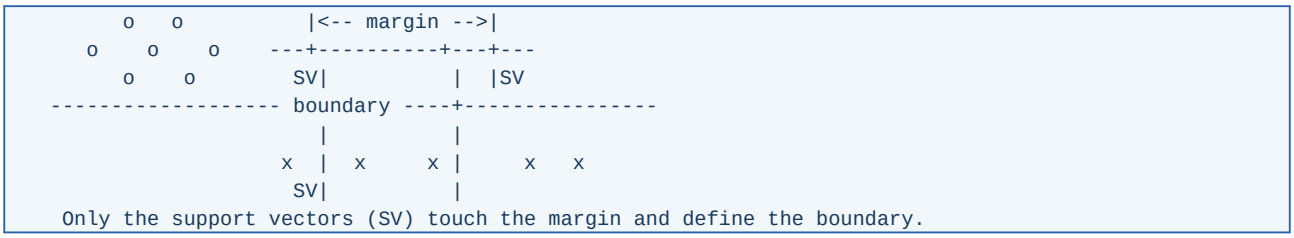

Figure 10.1 - The maximum-margin hyperplane and its support vectors.

```
                          minimise   (1/2)||w||^2

        subject to y_i (w . x_i + b) >= 1   for all i          (hard margin)

      geometric margin = 2 / ||w||       -> maximising margin = minimising ||w||
```

Real data is rarely separable, so slack variables xi_i allow violations, penalised by C:

```
    minimise (1/2)||w||^2 + C SUM_i xi_i,   y_i(w.x_i + b) >= 1 - xi_i,  xi_i >= 0

          equivalently   min  (1/2)||w||^2 + C SUM_i max(0, 1 - y_i f(x_i))
```

That second form shows the SVM is just a linear model with **hinge loss** and L2 regularisation. Large C means little regularisation (fit the training data hard); small C means a wide, soft margin. C is the hyperparameter that matters most.

10.2 The dual problem and the kernel trick

WHY THE DUAL MATTERS

Introducing Lagrange multipliers alpha_i for the margin constraints and eliminating w and b gives the dual: maximise SUM alpha_i - (1/2) SUM_ij alpha_i alpha_j y_i y_j (x_i . x_j), subject to 0 <= alpha_i <= C and SUM alpha_i y_i = 0. The data appears ONLY through inner products x_i . x_j. So if you replace that inner product with any function K(x_i, x_j) that equals an inner product in some (possibly infinite-dimensional) feature space, you get a non-linear classifier at no extra representational cost. Also, alpha_i is non-zero only for support vectors - the solution is sparse in the training set.

```
                    f(x) = SUM_i alpha_i y_i K(x_i, x) + b
```

Kernel	K(a, b)	Character
Linear	a . b	Text, very high-dimensional sparse data
Polynomial	(gamma a.b + r)^p	Explicit interactions up to degree p
RBF / Gaussian	exp(-gamma ||a - b||^2)	The default; infinite-dimensional feature space, local influence
Sigmoid	tanh(gamma a.b + r)	Rarely used; not always a valid kernel

Kernel	K(a, b)	Character
String / graph kernels	domain-specific	Structured inputs without vectorisation

A valid kernel must produce a positive semi-definite Gram matrix (Mercer's condition). For the RBF kernel, gamma sets the radius of influence of each support vector: large gamma means very local, wiggly boundaries and overfitting; small gamma approaches a linear model. Tune **C and gamma jointly on a log grid** - they interact strongly.

10.3 Practical guidance

- **Always scale features.** RBF distances are meaningless otherwise.
- Training cost is between O(n^2) and O(n^3): SVMs are excellent up to roughly 10^4-10^5 samples and impractical beyond. For large n use `LinearSVC`, SGD with hinge loss, or random-feature approximations (Nystroem, Random Fourier Features) which approximate the kernel map explicitly and restore linear cost.
- **SVR** (support vector regression) uses an epsilon-insensitive tube: errors smaller than epsilon cost nothing, which yields sparse, robust regression.
- **One-class SVM** learns the support of a distribution and is a classical anomaly detector.
- SVMs output distances, not probabilities. `probability=True` fits Platt scaling internally, which requires an extra internal cross-validation - it is slow and often better done explicitly (Chapter 13).

HINGE LOSS VERSUS CROSS-ENTROPY

Hinge loss is exactly zero once a point is correctly classified with margin at least 1, so well-classified points stop contributing - hence sparsity in the support vectors. Cross-entropy never reaches zero and keeps pushing confident points further, which is why logistic models produce calibrated probabilities while SVMs produce good boundaries. Choose by what you need: a decision or a probability.

Exercises

1. On a 2-D toy dataset, plot the RBF-SVM decision boundary for gamma in {0.01, 0.1, 1, 10} and C in {0.1, 1, 100}. Identify overfitting visually.
2. Verify the dual sparsity claim: count support vectors as C decreases.
3. Approximate an RBF kernel with `Nystroem` plus `LinearSVC` and compare accuracy and training time against the exact `SVC` on 20,000 samples.

11. Ensembles: Bagging, Random Forests and Boosting

An ensemble combines many models into one predictor that is better than any member. There are two fundamentally different recipes, and the bias-variance decomposition of Chapter 3 tells you exactly what each one is for.

Family	Members are	Trained	Attacks	Examples
Bagging	Strong, low-bias, high-variance	In parallel, independently	Variance	Random forest, extra trees
Boosting	Weak, high-bias	Sequentially, each fixing the last	Bias (and variance, via shrinkage)	AdaBoost, GBM, XGBoost, LightGBM
Stacking	Diverse model types	Level-0 in parallel, level-1 on out-of-fold predictions	Both	Any blend + a meta-learner

11.1 Why averaging works

THE VARIANCE REDUCTION FORMULA

Average M models each with variance s^2 and pairwise correlation rho. The variance of the average is rho*s^2 + (1 - rho) s^2 / M. As M grows the second term vanishes, but the first does not: the floor is set by how CORRELATED the members are. This single formula explains the entire design of random forests - every mechanism in them exists to lower rho.

11.2 Bagging and random forests

- **Bootstrap sample:** draw n samples with replacement; about 63.2% of the rows appear, the rest are **out-of-bag** and give a free validation estimate.
- **Feature subsampling at each split** (`max_features = sqrt(d)` for classification, `d/3` for regression) is what separates a random forest from plain bagged trees; it decorrelates the trees strongly.
- **Extra trees** go further: thresholds are drawn at random rather than optimised. Higher bias, much lower variance and much faster.
- Trees are grown deep and unpruned - variance is handled by the average, not by pruning each member.
- More trees never hurt accuracy, only time; 300-1,000 is typical, and the curve flattens.

Random forests are the best 'no-thought' baseline in existence: they need no scaling, tolerate irrelevant features, rarely overfit catastrophically, and their default hyperparameters are usually within a few percent of tuned performance.

11.3 Boosting, from AdaBoost to gradient boosting

AdaBoost, the original idea

Train a weak learner; increase the weights of the samples it got wrong; train the next learner on the reweighted data; repeat; combine with weights based on each learner's accuracy. AdaBoost was later shown to be gradient descent on the **exponential loss** in function space - which opened the door to the general framework.

Gradient boosting, stated properly

Build an additive model `F_M(x) = SUM_m nu * h_m(x)`. At each stage, fit the next weak learner to the **negative gradient of the loss with respect to the current predictions** - the pseudo-residuals:

```
r_im = - [ dL(y_i, F(x_i)) / dF(x_i) ]_(F = F_(m-1))

h_m  = argmin_h SUM_i ( r_im - h(x_i) )^2        (fit a tree to residuals)

F_m  = F_(m-1) + nu * gamma_m * h_m               (nu = learning rate)
```

With squared loss the pseudo-residuals are the ordinary residuals, which is the intuition most people learn first: each new tree predicts what the current ensemble is still getting wrong. Because the recipe only needs a gradient, the same algorithm works for logistic loss, Poisson loss, quantile loss, ranking losses and custom business losses.

SHRINKAGE AND THE NUMBER OF TREES TRADE OFF

The learning rate nu (shrinkage) and the number of trees M are coupled: halving nu roughly doubles the M you need. Small nu (0.01-0.1) with many trees and early stopping on a validation set generalises better than large nu with few trees. This pair, plus tree depth, is 80% of gradient boosting tuning.

11.4 Modern implementations

Library	Distinctive ideas	Best for
XGBoost	Second-order (Newton) boosting with an explicit regularisation term, sparsity-aware split finding, weighted quantile sketch	Strong all-rounder, huge ecosystem
LightGBM	Histogram binning, leaf-wise growth, GOSS sampling, EFB feature bundling	Very large datasets; usually the fastest
CatBoost	Ordered boosting to remove target leakage, native categorical handling with ordered target statistics, oblivious trees	Many categorical features; least tuning needed
scikit-learn HistGBDT	LightGBM-style histograms in scikit-learn	Zero extra dependencies

```
import lightgbm as lgb

params = dict(objective='binary', metric='auc',
              learning_rate=0.03,      # small + many rounds + early stop
              num_leaves=63,           # capacity; 2^depth is the cap
              min_data_in_leaf=50,     # main overfitting control
              feature_fraction=0.8,    # column subsampling per tree
              bagging_fraction=0.8, bagging_freq=1,   # row subsampling
              lambda_l2=1.0, verbose=-1)

dtr = lgb.Dataset(X_tr, y_tr)
dva = lgb.Dataset(X_va, y_va, reference=dtr)
model = lgb.train(params, dtr, num_boost_round=5000,
                  valid_sets=[dva],
                  callbacks=[lgb.early_stopping(200),
                             lgb.log_evaluation(200)])
print('best iteration:', model.best_iteration)
```

Listing 11.1 - A gradient-boosting configuration that is hard to beat on tabular data.

11.5 Gradient boosting on six points, three rounds

Target values 10, 12, 14, 20, 22, 30. The first model is the mean, 18. Each round fits a depth-1 tree (a single split) to the current residuals, and adds it with a learning rate of 0.5.

Round	Residuals fed to the tree	Tree's split and outputs	Ensemble prediction after the round	MSE
0	-	constant 18	18, 18, 18, 18, 18, 18	46.67

Round	Residuals fed to the tree	Tree's split and outputs	Ensemble prediction after the round	MSE
1	-8, -6, -4, +2, +4, +12	split after 3rd: -6 / +6	15, 15, 15, 21, 21, 21	19.67
2	-5, -3, -1, -1, +1, +9	split after 5th: -1.8 / +9	14.1, 14.1, 14.1, 20.1, 20.1, 25.5	7.52
3	-4.1, -2.1, -0.1, -0.1, +1.9, +4.5	split after 4th: -1.6 / +3.2	13.3, 13.3, 13.3, 19.3, 21.7, 27.1	3.68

The error falls 46.67 -> 19.67 -> 7.52 -> 3.68. Each tree is individually useless - it is a single threshold - but each one attacks precisely what the ensemble still gets wrong, and the errors shrink geometrically. That is the whole of boosting; XGBoost and LightGBM differ from this table only in how they find the split, how they regularise the leaf values, and how fast they do it.

WHY THE LEARNING RATE IS 0.5 AND NOT 1.0

With nu = 1.0 the first tree would remove the residuals entirely on the training data and the ensemble would immediately be fitting noise. Shrinkage forces each tree to take only part of the credit, so the correction is spread over many trees and no single one dominates. This is the same principle as a small learning rate in gradient descent, and it is the main reason boosting generalises rather than merely memorising.

11.6 Bagging versus boosting, side by side

Question	Random forest (bagging)	Gradient boosting
What is each tree trained on?	A bootstrap resample, with a random subset of features per split	The residuals of everything built so far
How deep are the trees?	Deep, often unlimited	Shallow - depth 3-8, or 31-255 leaves
Can it be parallelised?	Yes, trees are independent	No across trees (each needs the previous); yes within a tree
What happens with more trees?	Error flattens; more trees never hurt accuracy	Error keeps falling then RISES - you must early-stop
Main risk	Underfitting if trees are too shallow	Overfitting if the learning rate is high or you boost too long
Tuning effort	Very low - defaults are close to optimal	Moderate - learning rate, depth, and rounds interact
Typical winner on tabular data	Strong baseline	Usually the best, by a small but consistent margin

11.7 Stacking and blending

Train diverse level-0 models (a GBDT, a linear model, a k-NN, a small neural net), collect their **out-of-fold** predictions as new features, and train a simple level-1 model - usually regularised logistic or linear regression - on those. The out-of-fold requirement is absolute: using in-fold predictions leaks and produces a meta-model that trusts an overfitted base model.

PRACTICAL ENSEMBLING THAT IS WORTH THE COMPLEXITY

In order of value per unit of effort: (1) average several seeds of the same model - almost free, reliably worth a few tenths of a percent; (2) average a GBDT with a neural network - their errors are genuinely different; (3) full stacking with a meta-learner. In production, weigh the gain against the latency and maintenance cost of running five models.

Exercises

1. Implement gradient boosting with depth-2 trees and squared loss in 60 lines, and verify that its predictions approach the target as M grows.
2. Show the effect of `max_features` on random forest accuracy and on the average correlation between tree predictions.
3. Take a tuned LightGBM model and add a small MLP to a simple average. Measure whether the blend beats both members, and check the correlation of their errors to explain why.

12. Unsupervised Learning and Dimensionality Reduction

12.1 Clustering

k-means

Partition n points into k clusters minimising within-cluster squared distance. Lloyd's algorithm alternates two steps until assignments stop changing:

```
Assign:  c_i = argmin_k ||x_i - mu_k||^2

Update:  mu_k = mean of the points assigned to cluster k

Objective (inertia):  J = SUM_i ||x_i - mu_c_i||^2
```

- Each step never increases J, so the algorithm converges - to a **local** optimum that depends on initialisation. Use **k-means++** seeding and several restarts.
- Assumes clusters are roughly spherical, similar in size and density; it fails on elongated or nested shapes.
- Choosing k: the **elbow** of the inertia curve, the **silhouette score** (the more reliable of the two), gap statistic, or a downstream metric if the clusters feed another system.
- Scale features first; k-means is Euclidean and unit-sensitive. MiniBatchKMeans scales to millions of points.

Gaussian mixture models and EM

A GMM is the soft, probabilistic generalisation: data is assumed drawn from a mixture of K Gaussians, each with its own mean, covariance and weight. **Expectation-Maximisation** alternates computing the posterior responsibility of each component for each point (E-step) with re-fitting the components weighted by those responsibilities (M-step). Each iteration provably does not decrease the likelihood.

```
E-step:  gamma_ik = pi_k N(x_i | mu_k, S_k) / SUM_j pi_j N(x_i | mu_j, S_j)

M-step:  mu_k = SUM_i gamma_ik x_i / SUM_i gamma_ik   (and similarly S_k, pi_k)
```

GMMs give soft assignments, elliptical clusters, a proper likelihood (so BIC can select K), and a density model usable for anomaly detection. k-means is the limiting case with spherical, equal, vanishing-variance components.

Density and hierarchical methods

- **DBSCAN:** clusters are dense regions separated by sparse ones. Finds arbitrary shapes, labels outliers as noise, needs no k - but needs eps and min_samples, and struggles with varying density.
- **HDBSCAN:** builds a hierarchy over eps and extracts the most stable clusters. The best modern default for exploratory clustering.
- **Agglomerative:** merge the closest pair repeatedly; the dendrogram shows structure at every scale. Linkage choice (ward, average, complete) changes the results substantially.
- **Spectral clustering:** cluster the eigenvectors of a similarity graph Laplacian - excellent for manifold-shaped data, O(n^3) without approximation.

CLUSTERING ALWAYS RETURNS CLUSTERS

Every algorithm partitions whatever you give it, including pure noise. Before believing a clustering: check stability across seeds and subsamples, check that silhouette is meaningfully above zero, and, most importantly, validate against something external - a downstream metric or a domain expert's reading of the clusters.

12.2 Dimensionality reduction

PCA, derived

PCA finds the orthogonal directions of maximum variance. Centre X, then either eigendecompose the covariance `C = X^T X / (n-1)` or - better numerically - take the SVD of X directly.

```
X = U S V^T   =>   principal directions are the columns of V

explained variance ratio of component j = s_j^2 / SUM_k s_k^2

projection to k dims:  Z = X V_k        reconstruction: X_hat = Z V_k^T
```

TWO EQUIVALENT CHARACTERISATIONS

Maximising projected variance and minimising squared reconstruction error give the SAME subspace. That equivalence is why PCA is simultaneously a compression method and a de-noising method, and why a linear autoencoder with squared loss learns exactly the PCA subspace (Chapter 25).

- **Scale first** unless all features share units - otherwise the largest-variance unit dominates.
- Choose k by cumulative explained variance (90-99%) or by an elbow in the scree plot.
- PCA is linear and unsupervised: it may discard exactly the low-variance direction that carries your label. Check downstream, not just variance.
- **Whitening** rescales components to unit variance - useful before some downstream models, harmful when it amplifies noise directions.
- Variants: randomised/truncated SVD for large matrices, kernel PCA for non-linear structure, sparse PCA for interpretable loadings, incremental PCA for streaming.

Manifold learning: t-SNE and UMAP

Aspect	t-SNE	UMAP
Objective	Match pairwise neighbour probabilities with KL divergence	Match fuzzy topological structure with cross-entropy
Speed	Slow, O(n log n) with Barnes-Hut	Much faster; scales to millions
Global structure	Poorly preserved - inter-cluster distances are not meaningful	Better preserved, still not metric
Key knob	perplexity (5-50)	n_neighbors, min_dist
New points	No natural transform	Has a transform method

HOW TO MISREAD A T-SNE PLOT

Cluster sizes carry no meaning. Distances between clusters carry almost no meaning. Random seeds change the picture. Different perplexities can invent or dissolve clusters. Use these plots to generate hypotheses and to sanity-check embeddings - never as evidence for a claim about geometry, and never to choose k for clustering.

Other reductions worth knowing

- **LDA** (supervised, K-1 dimensions, maximises class separation).
- **NMF** for non-negative data: parts-based, interpretable topics.
- **Autoencoders** for non-linear compression, and **VAEs** for a probabilistic latent space (Chapter 25).
- **Random projection**: the Johnson-Lindenstrauss lemma guarantees that a random linear map into O(log n / eps^2) dimensions preserves pairwise distances to within eps. Astonishingly cheap and useful for very high-dimensional sparse data.

12.3 k-means, iteration by iteration

Six one-dimensional points - 1, 2, 4, 7, 8, 10 - with k = 2 and the unlucky initial centroids 1 and 10:

Iteration	Assignment	New centroids	Inertia at the start of the step
1	{1, 2, 4} -> c1; {7, 8, 10} -> c2	c1 = 2.33, c2 = 8.33	23.00
2	{1, 2, 4} -> c1; {7, 8, 10} -> c2	c1 = 2.33, c2 = 8.33 (unchanged)	9.33
3	no change - converged	-	9.33

Two iterations, and the inertia drops from 23.0 to 9.33 - the value the algorithm reports. Now try initialising at 1 and 2 instead: the first assignment puts {1} in one cluster and {2, 4, 7, 8, 10} in the other, and the algorithm converges to a visibly worse partition with higher inertia. Same data, same k, different answer. That is why you run `n_init` restarts and keep the lowest inertia, and why k-means++ seeding (choose the next centroid far from the existing ones, with probability proportional to squared distance) is the default.

12.4 PCA on ten points, computed in full

Ten two-dimensional points with a strong positive correlation. Centre them, form the covariance matrix, and take its eigenvectors:

```
                    mean = (1.81, 1.91)

              covariance = [ 0.6166  0.6154 ]
                           [ 0.6154  0.7166 ]

              eigenvalues  = 1.284  and  0.049
PC1 = ( 0.678,  0.735)      explains 1.284/1.333 = 96.3% of variance
PC2 = (-0.735,  0.678)      explains 3.7%
```

PC1 points along the diagonal, which is where the data actually varies; PC2 is perpendicular to it, as it must be, and captures almost nothing. Projecting onto PC1 alone turns each 2-D point into one number - a 50% compression - and reconstructing from that single number recovers the original points with an RMSE of 0.15, small compared with the spread of the data. That is the entire method: rotate so the axes line up with the variance, then drop the axes that barely move.

PCA AS CHOOSING A CAMERA ANGLE

Imagine a flat, disc-shaped galaxy of points floating in 3-D. Photographed face-on you see its full structure; photographed edge-on it collapses to a line and you lose everything. PCA finds the face-on angle automatically, by maximising the spread of the shadow. The eigenvalues tell you how much structure each angle preserves, and the explained-variance ratio is just those numbers normalised to sum to one.

THE DIRECTION WITH THE MOST VARIANCE IS NOT ALWAYS THE ONE YOU WANT

PCA is unsupervised - it never looks at the labels. If your classes differ along a low-variance direction (a small but consistent offset) PCA may discard exactly that direction while faithfully preserving an irrelevant high-variance one such as overall brightness. Always check downstream accuracy after reducing, and consider LDA when you have labels and separation is the goal.

12.5 Anomaly detection

Method	Idea	Notes
Z-score / IQR	Univariate thresholds	Trivial baseline; ignores correlations
Mahalanobis distance	Distance under the covariance	Assumes one Gaussian blob
Isolation Forest	Random splits isolate outliers in fewer steps	Fast, few assumptions - a strong default

Method	Idea	Notes
Local Outlier Factor	Compares local density to neighbours' density	Catches local anomalies a global method misses
One-class SVM	Learns a boundary around the data	Sensitive to nu and gamma
Autoencoder reconstruction error	Anomalies reconstruct badly	Good for images and signals; needs clean training data

Exercises

1. Implement k-means and k-means++ initialisation; compare final inertia over 50 random restarts of each.
2. Run PCA on a face or digit dataset and display the first 16 components as images; then reconstruct with k = 5, 20, 100 and compare.
3. Cluster the same dataset with k-means, GMM, DBSCAN and HDBSCAN and compare silhouette scores and the number of points labelled noise.

13. Evaluation: Metrics, Calibration and Imbalance

A model is only as good as the number you use to judge it. Choosing that number is a modelling decision at least as important as choosing the architecture, and it is where most projects quietly fail.

13.1 Classification: the confusion matrix and everything derived from it

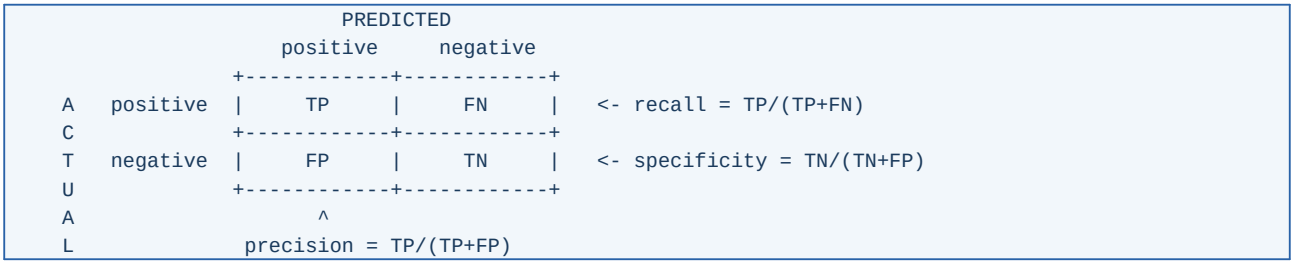

Figure 13.1 - Every classification metric is a ratio taken from this table.

Metric	Formula	Optimise it when
Accuracy	(TP+TN)/n	Classes are balanced and errors cost the same
Precision	TP/(TP+FP)	False alarms are expensive (spam filter, arrest, expensive follow-up)
Recall / sensitivity	TP/(TP+FN)	Misses are expensive (cancer screening, fraud, safety)
F1	2PR/(P+R)	You need a single number balancing the two
F-beta	(1+b^2)PR/(b^2 P + R)	You can state how much more recall matters than precision
Specificity	TN/(TN+FP)	The true-negative rate matters explicitly
Balanced accuracy	(recall + specificity)/2	Imbalanced data, single number
MCC	correlation of predictions and truth	Imbalanced data; the most informative single scalar
Cohen kappa	agreement above chance	Comparing against annotator agreement

13.2 Threshold-free metrics: ROC and PR curves

- **ROC curve:** true positive rate against false positive rate as the threshold sweeps. **AUC-ROC** is the probability that a random positive scores above a random negative. It is invariant to class balance - which is a strength when comparing across datasets and a serious weakness when positives are rare, because a huge number of false positives barely moves FPR.
- **Precision-recall curve:** precision against recall. **AUC-PR** (or average precision) is the right summary under heavy imbalance, because its baseline is the positive rate itself, not 0.5.
- **Rule:** with a positive rate below roughly 10%, report PR-AUC. Report ROC-AUC too if you like, but do not make decisions with it.

WHY AUC-ROC FLATTERS A RARE-EVENT MODEL

Take 1,000,000 negatives and 1,000 positives. A model that flags 20,000 negatives as positive has FPR = 2%, which looks excellent on an ROC curve. But if it also catches 800 positives, precision is 800/20,800 = 3.8% - 96% of the alerts are wrong, and the operations team will abandon the system in a week. The PR curve shows this immediately; the ROC curve hides it.

13.3 Choosing the operating threshold

Training produces scores; the threshold is a **separate decision** that should be made with the costs of the application in hand. If a false negative costs C_FN and a false positive costs C_FP, the expected-cost-minimising threshold on a calibrated probability is:

```
t* = C_FP / (C_FP + C_FN)
```

So if a miss is nine times more expensive than a false alarm, threshold at 0.1, not 0.5. Choose the threshold on validation data, never on test, and re-check it after any retraining, because score distributions drift.

13.4 Regression metrics

Metric	Formula	Character
MSE / RMSE	mean of squared errors (root)	Penalises large errors hard; RMSE is in the units of y
MAE	mean absolute error	Robust; the median-optimal predictor
MAPE	mean of |err|/|y| in percent	Interpretable, explodes near y = 0, asymmetric
sMAPE / WAPE	scaled variants	Fixes some MAPE pathologies
R^2	1 - SS_res/SS_tot	Relative to predicting the mean; can be negative
Pinball loss	asymmetric absolute	Evaluates quantile forecasts
MASE	error relative to a naive forecast	Time series: is the model beating last-value-carried-forward?

13.5 Probability calibration

A model is **calibrated** when among all samples it scores 0.7, about 70% are truly positive. Ranking quality (AUC) and calibration are independent: a model can rank perfectly and still be systematically overconfident. If the score feeds a threshold rule, an expected-cost calculation, or a human decision, calibration is not optional.

- **Diagnose** with a reliability diagram (predicted probability against observed frequency in bins) and summarise with **expected calibration error (ECE)** or Brier score.
- **Platt scaling:** fit a 1-D logistic regression on the scores. Few parameters, works well with little validation data, assumes a sigmoidal distortion.
- **Isotonic regression:** fit a monotone step function. More flexible, needs more data (about 1,000+ validation samples), can overfit.
- **Temperature scaling:** divide the logits by a single learned scalar T. The standard fix for modern neural networks, which are famously overconfident; it preserves accuracy exactly since it is monotone.
- Fit calibration on a **held-out** set, never on the training data.

```python
import torch, torch.nn.functional as F

def fit_temperature(logits, labels, iters=200):
    # logits: (n, K) from a frozen trained model on a VALIDATION set
    logT = torch.zeros(1, requires_grad=True)
    opt = torch.optim.LBFGS([logT], lr=0.1, max_iter=iters)
    def closure():
        opt.zero_grad()
        loss = F.cross_entropy(logits / logT.exp(), labels)
        loss.backward()
        return loss
    opt.step(closure)
    return logT.exp().item()      # T > 1 softens an overconfident model
```

Listing 13.1 - Temperature scaling: one parameter, large calibration gains.

13.6 One confusion matrix, every metric computed

A fraud model is evaluated on 1,000 transactions, of which 100 are genuinely fraudulent. It flags 140, of which 80 are correct.

```
                      PREDICTED
                   fraud      legit        total
               +----------+----------+
   A    fraud   |  TP = 80 |  FN = 20 |     100
   C           +----------+----------+
   T    legit   |  FP = 60 |  TN =840 |     900
   U           +----------+----------+
   A    total       140         860        1000
```

Figure 13.2 - The worked example used throughout this section.

Metric	Computation	Value	What it tells you
Accuracy	(80+840)/1000	0.920	Looks excellent - but always predicting 'legit' scores 0.900, so the model has bought you 2 points
Precision	80/140	0.571	43% of the alerts are wrong; this is the analyst's wasted time
Recall	80/100	0.800	One fraud in five still gets through
F1	2(0.571)(0.8)/(0.571+0.8)	0.667	The balance of the two
Specificity	840/900	0.933	Most legitimate customers are left alone
Balanced accuracy	(0.800+0.933)/2	0.867	Accuracy that is not fooled by the class ratio
MCC	correlation form	0.634	The single most informative scalar here
False positive rate	60/900	0.067	The x-axis of the ROC curve

NOW CONVERT THE METRICS INTO MONEY

Suppose an investigated alert costs 20 EUR of analyst time and a missed fraud costs 500 EUR. This model costs 140 x 20 + 20 x 500 = 12,800 EUR. Lower the threshold until recall reaches 0.95: perhaps 300 alerts and 5 misses, costing 300 x 20 + 5 x 500 = 8,500 EUR - materially better, with WORSE precision and WORSE accuracy. This is why Chapter 13 insists that the threshold is a business decision. Compute the cost curve, then pick the operating point; never accept 0.5 by default.

13.7 Reading a ROC curve and a PR curve of the same model

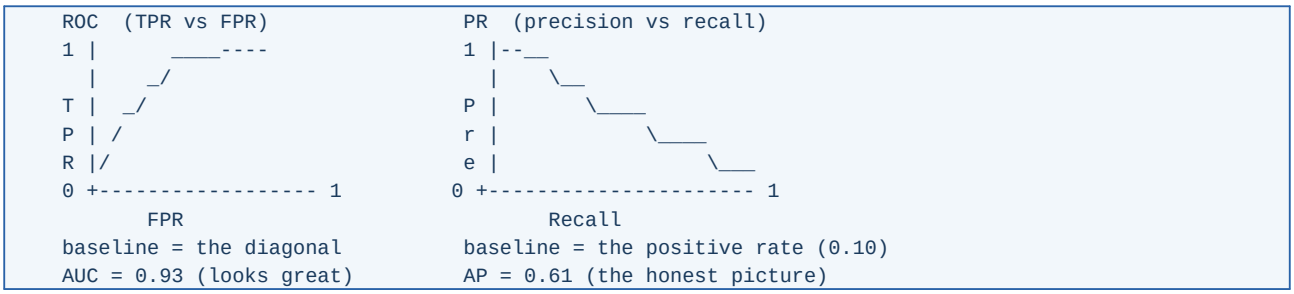

Figure 13.3 - The same predictions, two curves, two impressions.

The ROC curve's baseline is the diagonal regardless of class balance, so a rare-positive problem always looks flattering. The PR curve's baseline is the positive rate - here 0.10 - so an average precision of 0.61 is correctly read as 'six times better than guessing', not as 'nearly perfect'. When positives are rare, quote average precision and show the curve.

13.8 Statistical significance and reporting

- Report mean and standard deviation over at least 3-5 seeds. A single run is an anecdote.

- Use bootstrap confidence intervals on the test set: resample it with replacement 1,000 times and take the 2.5th and 97.5th percentiles of the metric.
- For paired model comparison on the same test set use McNemar's test (classification) or a paired bootstrap; for multiple datasets use the Wilcoxon signed-rank test.
- Correct for multiple comparisons when you test many models (Bonferroni is crude but honest).
- Always report the trivial baseline. 'We reached 94% accuracy' means nothing if the majority class is 93%.

GOODHART'S LAW IN MACHINE LEARNING

When a measure becomes a target, it ceases to be a good measure. A model tuned relentlessly against one offline metric will find the shortcuts that metric permits - background artefacts in X-rays, timestamp leakage in fraud data, annotation quirks in benchmarks. Defend with: multiple metrics, slice-based evaluation across subgroups, a hand-audited error sample every cycle, and an online test before you believe anything.

13.9 Slice-based evaluation

Aggregate metrics hide the failures that matter. Always evaluate per slice: by class, by subgroup (age, sex, device, region, language), by difficulty, by data source, and by time period. A useful discipline is to define the slices **before** training and to treat a large per-slice drop as a release blocker, exactly like a failing test.

Exercises

1. Construct a dataset with a 1% positive rate and a model with ROC-AUC 0.95; compute its precision at the 0.5 threshold and explain the gap.
2. Draw a reliability diagram for a random forest and for a neural network on the same data; apply isotonic regression and temperature scaling respectively and re-draw.
3. Compute a bootstrap 95% confidence interval for the F1 of your best model and decide whether it is genuinely better than the runner-up.

PART III

Deep Learning Core

Neurons, backpropagation derived by hand, activations, optimisers, normalisation, regularisation, and a practical playbook for making training actually work.

14. From a Single Neuron to a Multilayer Network

14.1 The artificial neuron

```
            z = w . x + b          (a linear score - identical to Chapter 5)

                        a = phi(z)              (a non-linearity)
```

That is the entire unit. Its power comes from two things: stacking many of them in a layer, and stacking layers. Without the non-linearity phi, stacking is pointless - the composition of linear maps is a linear map, so a 50-layer linear network has exactly the expressive power of one linear layer. **The non-linearity is what makes depth mean anything.**

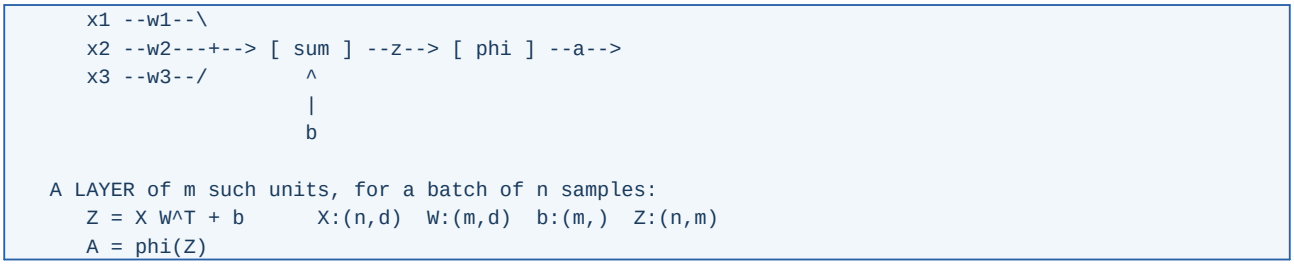

Figure 14.1 - A neuron, and the matrix form that a GPU actually executes.

14.2 The multilayer perceptron

```
                                a^(0) = x

                          z^(l) = W^(l) a^(l-1) + b^(l)

                a^(l) = phi( z^(l) )                for l = 1 .. L-1

                        y_hat = output_activation( z^(L) )
```

Task	Output units	Output activation	Loss
Regression	1	none (identity)	MSE / Huber
Binary classification	1	sigmoid (or none + BCEWithLogits)	Binary cross-entropy
Multiclass (one label)	K	softmax (or none + CrossEntropyLoss)	Categorical cross-entropy
Multi-label	K	K independent sigmoids	Sum of binary cross-entropies
Count	1	exp / softplus	Poisson NLL
Quantiles	Q	none	Pinball loss per quantile

14.3 The universal approximation theorem, honestly stated

WHAT IT DOES AND DOES NOT PROMISE

A feedforward network with ONE hidden layer and a non-polynomial activation can approximate any continuous function on a compact set to arbitrary accuracy, given ENOUGH hidden units. What the theorem does NOT say: how many units (it can be exponential in the input dimension), that gradient descent will find those weights, or that the result will generalise. It establishes that depth is not required for expressiveness - and practice establishes that depth is required for EFFICIENCY.

Depth buys exponential efficiency for compositional functions. A function built from repeated composition - edges to shapes to parts to objects, characters to words to phrases to meaning - is represented by a deep

network with a number of units linear in depth, and may need exponentially many units in a shallow one. Real perceptual data is compositional, which is why depth wins in practice.

14.4 Counting parameters and cost

```
Params of a dense layer: m * d + m       (weights + biases)

MLP 784 -> 256 -> 128 -> 10:

     784*256+256 = 200,960

     256*128+128 =  32,896

128*10 +10  =   1,290      TOTAL = 235,146 parameters

Memory in float32 = 235,146 * 4 B = 0.94 MB (weights only)
```

Training memory is far larger than the weights: you also store activations for the backward pass (batch size times all layer outputs), gradients (one per parameter), and optimiser state (two more per parameter for Adam). A useful rule for Adam in float32: **about 16 bytes per parameter** before activations. This arithmetic is the entry point to Part V, where reducing it is the whole game.

14.5 Your first network, twice

```
import torch, torch.nn as nn

model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(784, 256), nn.BatchNorm1d(256), nn.ReLU(), nn.Dropout(0.2),
    nn.Linear(256, 128), nn.BatchNorm1d(128), nn.ReLU(), nn.Dropout(0.2),
    nn.Linear(128, 10),                    # raw logits, no softmax here
)

opt = torch.optim.AdamW(model.parameters(), lr=3e-4, weight_decay=1e-2)
lossf = nn.CrossEntropyLoss(label_smoothing=0.05)
sched = torch.optim.lr_scheduler.OneCycleLR(opt, max_lr=3e-3,
                                            total_steps=epochs*len(train_dl))

for epoch in range(epochs):
    model.train()
    for xb, yb in train_dl:
        xb, yb = xb.to(dev), yb.to(dev)
        opt.zero_grad(set_to_none=True)
        loss = lossf(model(xb), yb)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        opt.step(); sched.step()
    model.eval()
    with torch.no_grad():
        acc = sum((model(x.to(dev)).argmax(1).cpu() == y).sum().item()
                  for x, y in val_dl) / len(val_dl.dataset)
    print(epoch, round(acc, 4))
```

Listing 14.1 - A complete, modern training loop. Every ingredient - BatchNorm, dropout, AdamW, warmup schedule, gradient clipping, label smoothing - is explained in Chapters 16-20.

TRAIN() AND EVAL() ARE NOT DECORATION

Dropout must be off and BatchNorm must use running statistics at evaluation time. Forgetting `model.eval()` is the single most common PyTorch bug and produces mysteriously poor, noisy validation numbers. Equally, forgetting `torch.no_grad()` at evaluation wastes memory building a graph you never use.

Exercises

1. Prove that a network with identity activations and L layers computes a single linear map, and give the resulting weight matrix.
2. Count the parameters and estimate the Adam training memory of an MLP 1024-1024-1024-10.
3. Train the network above on MNIST or Fashion-MNIST; then remove all non-linearities and compare - the gap is the value of depth.

15. Backpropagation, Derived and Implemented

Backpropagation is reverse-mode automatic differentiation applied to a neural network. It is not a learning algorithm - gradient descent is - but it is what makes gradient descent affordable: the cost of computing gradients for **all** parameters is about the same as one forward pass, regardless of how many parameters there are.

15.1 The four equations

Define the error signal at layer l as `delta^(l) = dJ / dz^(l)`. Then:

```
        (1)  delta^(L)   = grad_a J  *  phi'( z^(L) )          [output layer]

      (2)  delta^(l)   = ( W^(l+1)^T delta^(l+1) ) * phi'( z^(l) )   [recursion]

                    (3)  dJ/dW^(l)   = delta^(l) ( a^(l-1) )^T

                           (4)  dJ/db^(l)   = delta^(l)

                       ( * denotes elementwise multiplication )
```

DERIVING EQUATION (2), THE ONLY STEP THAT MATTERS

z^(l+1) = W^(l+1) a^(l) + b^(l+1) and a^(l) = phi(z^(l)). By the chain rule, dJ/dz^(l)_j = SUM_k (dJ/dz^(l+1)_k)(dz^(l+1)_k/da^(l)_j)(da^(l)_j/dz^(l)_j) = SUM_k delta^(l+1)_k W^(l+1)_kj phi'(z^(l)_j). Collecting over j gives exactly (W^(l+1)^T delta^(l+1)) * phi'(z^(l)). Notice what this says: the error is propagated backwards through the TRANSPOSE of the same weight matrix used in the forward pass, and it is gated by the local derivative of the activation.

```
  FORWARD    x --> [W1] --> z1 --> phi --> a1 --> [W2] --> z2 --> loss
                                                                  |
  BACKWARD       dW1 <-- d1 <-- *phi' <-- W2^T d2 <----------- d2 <-+

  Each layer needs, from the forward pass: its input a^(l-1) and z^(l).
  That is why activations are stored - and why training memory scales
  with batch size times depth.
```

Figure 15.1 - Forward stores activations; backward consumes them.

15.2 Why reverse mode, and what it costs

Mode	Cost	Efficient when
Forward-mode AD	One pass per INPUT variable	Few inputs, many outputs
Reverse-mode AD (backprop)	One pass per OUTPUT variable	Many inputs, one output - exactly the ML case: millions of parameters, one scalar loss
Numerical differences	Two evaluations per parameter	Never for training; only for checking a hand-written gradient

15.3 A complete NumPy implementation

```python
import numpy as np

def init(sizes, rng):
    # He initialisation, correct for ReLU (Chapter 16)
    return [ (rng.normal(0, np.sqrt(2.0/a), (b, a)), np.zeros(b))
             for a, b in zip(sizes[:-1], sizes[1:]) ]

def forward(params, X):
```

```
    cache = [X]
    A = X
    for i, (W, b) in enumerate(params):
        Z = A @ W.T + b
        A = Z if i == len(params) - 1 else np.maximum(0.0, Z)   # ReLU
        cache.append((Z, A))
    return A, cache                       # A = logits at the last layer

def softmax_xent(logits, Y1h):
    Z = logits - logits.max(1, keepdims=True)
    P = np.exp(Z); P /= P.sum(1, keepdims=True)
    n = len(Y1h)
    loss = -np.sum(Y1h * np.log(P + 1e-12)) / n
    return loss, (P - Y1h) / n            # dL/dlogits: the clean form

def backward(params, cache, dZ):
    grads = [None] * len(params)
    for l in reversed(range(len(params))):
        A_prev = cache[0] if l == 0 else cache[l][1]
        dW = dZ.T @ A_prev                # eq. (3)
        db = dZ.sum(axis=0)               # eq. (4)
        grads[l] = (dW, db)
        if l > 0:
            dA = dZ @ params[l][0]        # W^T delta
            dZ = dA * (cache[l][0] > 0)   # * phi'(z), ReLU derivative
    return grads

def sgd_step(params, grads, lr):
    return [ (W - lr*dW, b - lr*db)
             for (W, b), (dW, db) in zip(params, grads) ]
```

Listing 15.1 - Backpropagation for an MLP in 35 lines. Write this once from scratch and deep learning stops being magic.

15.4 Gradient checking

Before trusting a hand-written gradient, compare it against a central finite difference. Use double precision, a step of about 1e-5, and the relative error criterion below; anything under 1e-7 is right, above 1e-4 is a bug.

$$\text{numeric} = \frac{J(\theta + \varepsilon) - J(\theta - \varepsilon)}{2\varepsilon}$$

$$\text{rel\_err} = \frac{|\text{numeric} - \text{analytic}|}{\max(|\text{numeric}|, |\text{analytic}|, 10^{-8})}$$

CHECK GRADIENTS WITH DROPOUT AND BATCH NORM DISABLED

Any source of randomness or batch-dependence makes J(theta + eps) and J(theta - eps) evaluate different functions, and the check fails for reasons that have nothing to do with your derivation. Fix the seed, turn off dropout, use eval-mode normalisation, and check on a small batch.

15.5 Backpropagation on real numbers, end to end

Everything in this chapter becomes concrete once you push one sample through a network by hand. Here is a 2-2-1 network with ReLU hidden units and a sigmoid output, one training sample, and every number written out. Verify each line with a calculator; it takes ten minutes and permanently removes the mystery.

```
x1=1.0 ---.
           \   W1 = [ 0.5  0.3 ]   b1 = [ 0.1 ]
            >-------[ 0.2  0.8 ]        [-0.1 ]  -> h1, h2
x2=2.0 ---'

h1, h2 --> W2 = [1.0, -1.0], b2 = 0.5 --> z2 --> sigmoid --> p

true label y = 1        loss = binary cross-entropy
(note: W1 row 2 is [-0.2, 0.8] in the arithmetic below)
```

Figure 15.2 - The network used for the hand computation.

Forward pass

```
z1_1 = 0.5(1.0) + 0.3(2.0) + 0.1  =  1.2      a1_1 = ReLU(1.2) = 1.2

z1_2 = -0.2(1.0) + 0.8(2.0) - 0.1 =  1.3      a1_2 = ReLU(1.3) = 1.3

              z2   = 1.0(1.2) + (-1.0)(1.3) + 0.5 = 0.4

             p    = sigmoid(0.4) = 1/(1 + e^-0.4) = 0.5987

                L    = -log(0.5987) = 0.5130
```

Backward pass

Start at the output and walk backwards, applying the four equations from the previous section. Every quantity below is a number you can check.

```
delta2 = p - y = 0.5987 - 1 = -0.4013            <- the clean identity

dL/dW2 = delta2 * a1 = -0.4013 * [1.2, 1.3] = [-0.4816, -0.5217]

                    dL/db2 = delta2 = -0.4013

dL/da1 = W2^T * delta2 = [1.0, -1.0] * -0.4013 = [-0.4013, +0.4013]

delta1 = dL/da1 * ReLU'(z1) = [-0.4013, 0.4013] * [1, 1] = [-0.4013, 0.4013]

          dL/dW1 = delta1 (x)^T = [ -0.4013*1.0   -0.4013*2.0 ]

                                  [ +0.4013*1.0   +0.4013*2.0 ]

                                    = [ -0.4013  -0.8026 ]

                                      [ +0.4013  +0.8026 ]

                dL/db1 = delta1 = [-0.4013, +0.4013]
```

The update, and proof that it worked

```
                          With eta = 0.1:

          W1 <- [ 0.5401   0.3803 ]      W2 <- [1.0482, -0.9478]

                     [-0.2401   0.7197 ]      b2 <- 0.5401

            Re-running the forward pass with the new weights:

                 z2 = 1.0464     p = 0.7401     L = 0.3010

      The loss fell from 0.5130 to 0.3010 in one step, and the predicted

        probability moved from 0.60 towards the true label 1. That is

                          learning, in its entirety.
```

CONFIRMING THE GRADIENT NUMERICALLY

Perturb W1[0,0] by eps = 1e-6 in each direction and recompute the loss: (L(+eps) - L(-eps)) / (2 eps) = -0.401312, which matches the analytic -0.401312 to six decimals. Do this for all four entries of W1 and you have written your own gradient checker - the tool that distinguishes a derivation error from a training-hyperparameter problem.

READING THE NUMBERS AS A STORY

delta2 = -0.4013 says the output was too LOW by about 0.4 (in probability terms). The gradient on W2 is negative for both hidden units, so both connections strengthen - the network decides to listen more to h1 and less negatively to h2. The gradient on W1 row 2 is positive, so those weights DECREASE, which lowers h2, which raises z2 because W2's second entry is negative. Every sign in the computation has a plain-language reason, and tracing them is the best debugging skill you can develop.

15.6 What ReLU's gate does to the gradient

In the example above both hidden pre-activations were positive, so both gates were open and gradient flowed to every weight. Change x to (1, -2) and z1_1 becomes 0.5 - 0.6 + 0.1 = 0.0 while z1_2 becomes -0.2 - 1.6 - 0.1 = -1.9. The second unit's ReLU derivative is then 0, so:

```
delta1 = dL/da1 * [1, 0] = [-0.4013, 0]

dL/dW1 row 2 = [0, 0]     <- this unit learns NOTHING from this sample
```

That is the mechanism behind three things you will meet repeatedly: ReLU networks are sparse (typically half the units are inactive on any given input, which is a form of free regularisation); a unit whose pre-activation is negative for **every** sample receives zero gradient forever and is dead; and gradient can vanish not only by shrinking but by being gated off entirely. Leaky ReLU exists precisely to leave the gate slightly ajar.

15.7 Vanishing and exploding gradients

Equation (2) is a repeated matrix product. Over L layers the error signal is multiplied by L factors of the form `W^T` and `phi'`. If the typical singular value of that product is below 1, the gradient decays exponentially with depth; above 1 and it explodes.

Problem	Symptom	Remedies
Vanishing	Early layers barely change; loss plateaus early; deep net does no better than a shallow one	ReLU-family activations, He/Glorot init, residual connections, normalisation layers, LSTM/GRU gates for sequences
Exploding	Loss becomes NaN or oscillates violently; gradient norms in the thousands	Gradient clipping (norm 1.0 is a good default), lower learning rate, normalisation, careful init, gradient accumulation instead of huge steps

RESIDUAL CONNECTIONS IN ONE LINE OF INTUITION

If a block computes y = x + F(x), then dy/dx = I + dF/dx. The identity term guarantees that gradient flows to earlier layers even when dF/dx is tiny. That is why ResNets made 100+ layer networks trainable, and it is why essentially every modern architecture, Transformers included, is built out of residual blocks.

15.8 Automatic differentiation in practice

- Frameworks build a **computation graph** during the forward pass (dynamic in PyTorch, traced/compiled in JAX and TF), then walk it backwards applying each operation's vector-Jacobian product.
- **Gradient accumulation:** call backward on several small batches before stepping, to simulate a large batch on limited memory. Remember to scale the loss by 1/accumulation_steps.

- **Gradient checkpointing:** discard activations during the forward pass and recompute them during the backward pass - trades roughly 30% extra compute for a large memory saving, and is what makes very deep or very long-context models fit.
- **detach() / stop_gradient** cuts the graph: essential for target networks in RL, teacher outputs in distillation, and any quantity you want treated as a constant.
- The **straight-through estimator** replaces a non-differentiable step (rounding, sign, top-k) with the identity in the backward pass. It is the trick that makes quantization-aware training possible (Chapter 28).

Exercises

1. Implement Listing 15.1, train it on MNIST to above 97% test accuracy, and verify every gradient against finite differences first.
2. Replace ReLU with sigmoid in a 10-layer network and plot the norm of dJ/dW per layer. Then add residual connections and re-plot.
3. Derive the backward pass of a batch-normalisation layer. It is the most instructive non-trivial derivation in deep learning.

16. Activations, Initialization and Loss Functions

16.1 Activation functions

Name	Definition	Range	Notes
Sigmoid	1/(1+e^-z)	(0,1)	Output layer for binary tasks only. Saturates; not zero-centred
Tanh	(e^z-e^-z)/(e^z+e^-z)	(-1,1)	Zero-centred sigmoid; still saturates. Used inside LSTM gates
ReLU	max(0, z)	[0,inf)	The default for 20 years of CNNs: cheap, no saturation for z>0. Can die
Leaky ReLU	max(0.01z, z)	R	Fixes dying ReLU with a small negative slope
PReLU	max(az, z), a learned	R	Leaky with a learned slope
ELU	z if z>0 else a(e^z-1)	(-a,inf)	Smooth, mean activations near zero, costlier
GELU	z * Phi(z)	R	Smooth, probabilistic gating. Standard in Transformers
SiLU / Swish	z * sigmoid(z)	R	Smooth, often slightly better than ReLU in vision
Mish	z * tanh(softplus(z))	R	Smooth alternative, more compute
Softplus	log(1+e^z)	(0,inf)	Smooth ReLU; used to output positive parameters
GLU / SwiGLU	(Wx) * sigma(Vx)	R	Gated variants; SwiGLU is the standard feedforward in modern LLMs
Softmax	e^z_k / SUM e^z_j	simplex	Output layer for multiclass

THE DYING RELU PROBLEM

If a unit's pre-activation is negative for every sample, its gradient is exactly zero forever and the unit is dead. This usually follows a learning rate that was too high early in training, and can silently kill 30-50% of a layer. Diagnose by counting units with zero activation over a validation batch; fix with a lower learning rate, He initialisation, Leaky ReLU or GELU, and normalisation layers.

WHAT TO ACTUALLY USE

Hidden layers of a convnet or MLP: ReLU, or GELU/SiLU if you want the last half percent. Transformers: GELU or SwiGLU. LSTM gates: keep sigmoid and tanh - the gates need bounded outputs. Output layer: chosen by the task, per the table in Chapter 14. Do not spend a week tuning activations; spend it on data.

16.2 Weight initialisation

Initialisation controls the scale of activations and gradients at step zero. Get it wrong and signals vanish or explode before learning begins. The principle is to keep the variance of activations roughly constant across layers.

```
Xavier/Glorot (tanh, sigmoid, linear):

Var(W) = 2 / (fan_in + fan_out)

He/Kaiming (ReLU family) - accounts for half the outputs being zeroed:

Var(W) = 2 / fan_in

LeCun (SELU):   Var(W) = 1 / fan_in

Orthogonal:     W = an orthogonal matrix, good for RNNs and deep stacks
```

- **Never initialise all weights to zero** - every unit in a layer would compute the same thing and receive the same gradient forever. Symmetry must be broken randomly.
- **Biases** start at zero (a small positive value such as 0.01 was once recommended for ReLU; it makes little difference in practice).
- **Residual branches** are often initialised so the block starts as an identity (zero-init the last layer of each block, or use LayerScale). This lets very deep networks train stably from step one.
- Modern transformer stacks scale initialisation by 1/sqrt(2L) on residual projections to keep the variance of the residual stream bounded with depth.

16.3 Loss functions, and what each one assumes

Loss	Task	Assumption / behaviour
MSE	Regression	Gaussian noise; punishes outliers heavily
MAE	Regression	Laplace noise; estimates the conditional median
Huber / smooth L1	Regression, detection	Quadratic near zero, linear far away - robust and smooth
Cross-entropy	Classification	Correct probabilistic loss; pairs with softmax/sigmoid
Focal loss	Detection, heavy imbalance	Down-weights easy examples by (1-p)^gamma
Label-smoothed CE	Classification	Targets 1-eps instead of 1; reduces overconfidence and improves calibration
KL divergence	Distillation, VAE	Matches a full distribution rather than a label
Contrastive / InfoNCE	Self-supervision, retrieval	Pull positives together, push negatives apart
Triplet loss	Metric learning	Anchor closer to positive than to negative by a margin
Dice / IoU loss	Segmentation	Directly optimises overlap; robust to class imbalance in masks
CTC	Speech, handwriting	Aligns unsegmented sequences
Pinball	Quantile regression	Asymmetric; yields prediction intervals

```
Focal loss:   FL(p_t) = -alpha (1 - p_t)^gamma log(p_t),   gamma ~ 2

Label smoothing: y_smooth = (1 - eps) y_onehot + eps / K,  eps ~ 0.1
```

MULTI-TASK LOSS WEIGHTING

When you sum several losses, their scales decide the effective learning rate of each task. Options: normalise each loss by a running estimate of its magnitude; use uncertainty weighting, which learns a per-task log-variance s_i and optimises SUM (L_i / (2 exp(s_i)) + s_i / 2); or use gradient-based balancing such as GradNorm. Fixed hand-tuned weights work but become a maintenance burden as tasks are added.

Exercises

1. Plot ReLU, GELU, SiLU and their derivatives on [-5, 5] and explain the smoothness argument in terms of the gradient at zero.
2. Initialise a 20-layer ReLU MLP with Var(W)=1/fan_in, 2/fan_in and 4/fan_in, and plot the standard deviation of activations per layer at initialisation.
3. Train a classifier with and without label smoothing eps=0.1 and compare accuracy, expected calibration error, and the histogram of maximum softmax probabilities.

17. Optimization Algorithms and Learning-Rate Schedules

Everything in this chapter is a variation on one line: `theta <- theta - eta * (something derived from the gradient)`. The variations differ in how they estimate direction and how they set an effective per-parameter step size.

17.1 The algorithms, in the order they were invented

SGD

```
theta <- theta - eta * g,        g = gradient on the current mini-batch
```

Momentum and Nesterov

```
v <- beta v + g              theta <- theta - eta v          (momentum)

Nesterov: evaluate the gradient at the look-ahead point theta - eta beta v
```

Momentum accumulates a velocity in directions of consistent gradient and cancels oscillation across a narrow valley. With beta = 0.9 the effective step is about 1/(1-beta) = 10 times larger along a consistent direction. It is not a nicety: SGD with momentum, well tuned, still produces the best final accuracy on many vision benchmarks.

Adaptive methods

```
AdaGrad:  s <- s + g^2,          theta <- theta - eta g / (sqrt(s) + e)

RMSProp:  s <- rho s + (1-rho) g^2,   theta <- theta - eta g/(sqrt(s)+e)

Adam:  m <- b1 m + (1-b1) g            (first moment)

       v <- b2 v + (1-b2) g^2          (second moment)

m_hat = m/(1-b1^t),  v_hat = v/(1-b2^t)     (bias correction)

    theta <- theta - eta * m_hat / ( sqrt(v_hat) + eps )
```

AdaGrad's accumulated sum only grows, so the effective learning rate decays monotonically to zero - fine for convex sparse problems, fatal for long deep-learning runs. RMSProp replaces the sum with an exponential moving average, fixing that. Adam adds momentum and bias correction, and it is the default for a good reason: it works without tuning on an enormous range of problems.

ADAMW: DECOUPLED WEIGHT DECAY

Adding lambda*||w||^2 to the loss is NOT the same as decaying weights when the optimiser rescales the gradient per parameter - the penalty gets divided by sqrt(v) too, so parameters with large gradients are barely regularised. AdamW decouples it: theta <- theta - eta(m_hat/(sqrt(v_hat)+eps) + lambda*theta). This one change reliably improves generalisation, and AdamW is now the default for Transformers and essentially all large-model training.

Optimiser	Typical LR	When to choose it
SGD + momentum 0.9	0.1 with cosine decay (batch 256)	CNNs on vision benchmarks; best final accuracy with a good schedule
Adam / AdamW	1e-3 (small nets), 1e-4 to 3e-4 (Transformers)	Default for NLP, Transformers, GANs, RL, anything sparse or ill-conditioned
RMSProp	1e-3	RNNs, some RL algorithms
LAMB / LARS	layer-wise scaled	Very large batch training (32k+) where plain scaling diverges

Optimiser	Typical LR	When to choose it
Lion	3-10x smaller than Adam	Memory-lean alternative: sign-based update, one state tensor instead of two
Shampoo / K-FAC / Sophia	problem-specific	Second-order-ish methods; strong on large-scale pretraining, more complex
L-BFGS	line search	Small full-batch deterministic problems only

17.2 Learning-rate schedules

The learning rate should usually be large early (fast progress, escaping poor regions) and small late (fine convergence). The schedule matters at least as much as the optimiser.

Schedule	Shape	Use
Step decay	x0.1 at fixed epochs	Classic ResNet recipes
Cosine annealing	smooth decay to ~0 over the run	The modern default; often with restarts (SGDR)
Linear warmup + decay	rise for k steps, then decay	Mandatory for Transformers and large batches
OneCycle	up then down, momentum inversely	Fast convergence in few epochs; strong for fine-tuning
Exponential	eta * gamma^epoch	Simple, needs tuning of gamma
ReduceLROnPlateau	cut when validation stalls	Robust when you cannot predict run length
Constant	flat	Debugging, or very short fine-tunes

WHY WARMUP IS NEEDED

At initialisation Adam's second-moment estimate v is based on very few samples, so its variance is huge and early steps can be wildly mis-scaled; simultaneously, in a deep residual stack the output distribution is far from its trained state, so early large steps do lasting damage. A linear warmup over 1-10k steps (or 1-5% of training) lets both stabilise. The larger the batch and the deeper the model, the more warmup you need.

17.3 Batch size, learning rate, and their interaction

- **Linear scaling rule:** multiply the batch size by k and multiply the learning rate by k, with warmup. Holds well up to batch sizes of a few thousand, then breaks down.
- **Square-root scaling** is sometimes better for Adam, since Adam already normalises by gradient magnitude.
- Small batches inject gradient noise, which acts as a regulariser and often improves generalisation; very large batches converge to sharper minima unless compensated with warmup, LARS/LAMB and more epochs.
- Batch size is chosen mostly by hardware: the largest that fits, rounded to a multiple of 8 (or 64) so tensor cores are used efficiently.
- **Gradient accumulation** simulates a larger batch on small memory; **gradient checkpointing** frees memory to enlarge the real batch.

17.4 Finding the learning rate

```
# LR range test (Smith): sweep the LR exponentially over one epoch and
# plot loss against LR. Pick roughly one order of magnitude below the
# point of steepest descent - NOT the minimum, which is already unstable.
lrs, losses = [], []
lr = 1e-7
for xb, yb in train_dl:
    for g in opt.param_groups: g['lr'] = lr
    opt.zero_grad(set_to_none=True)
    loss = lossf(model(xb.to(dev)), yb.to(dev))
    loss.backward(); opt.step()
    lrs.append(lr); losses.append(loss.item())
    lr *= 1.1
    if loss.item() > 4 * min(losses): break      # diverged; stop
```

Listing 17.1 - The learning-rate range test: five minutes that saves days.

17.5 One Adam update, arithmetic included

Adam's formula has four moving parts, and seeing them evaluate on a constant gradient makes the design obvious. Take g = 0.5 at every step, with beta1 = 0.9, beta2 = 0.999, eta = 0.001, eps = 1e-8.

Step t	m (first moment)	v (second moment)	m_hat	v_hat	Update size
1	0.05000	0.000250	0.50000	0.250000	0.001000
2	0.09500	0.000500	0.50000	0.250000	0.001000
3	0.13550	0.000749	0.50000	0.250000	0.001000

THREE THINGS THIS TABLE PROVES

(1) BIAS CORRECTION MATTERS: the raw m at step 1 is 0.05, ten times smaller than the true gradient 0.5, because the moving average starts from zero. Dividing by (1 - beta1^t) restores it exactly. Without correction, Adam would take absurdly small steps for the first hundred iterations. (2) THE UPDATE IS SCALE-FREE: it comes out at 0.001 = the learning rate, and it would still be 0.001 if the gradient were 0.5, 50, or 5,000, because m_hat/sqrt(v_hat) cancels the magnitude. That is why Adam needs so little tuning across wildly different layers and losses. (3) THE UNITS ARE INTERPRETABLE: with Adam, the learning rate IS roughly the per-step change in each parameter - which is why 1e-3 and 3e-4 are such durable defaults.

The flip side of scale invariance is that Adam ignores information SGD uses: a parameter with a genuinely tiny gradient still gets a full-sized step. That is one reason well-tuned SGD with momentum still edges out Adam on some vision benchmarks, and why the gap closes when Adam is paired with decoupled weight decay.

17.6 Choosing a learning rate: what each failure looks like

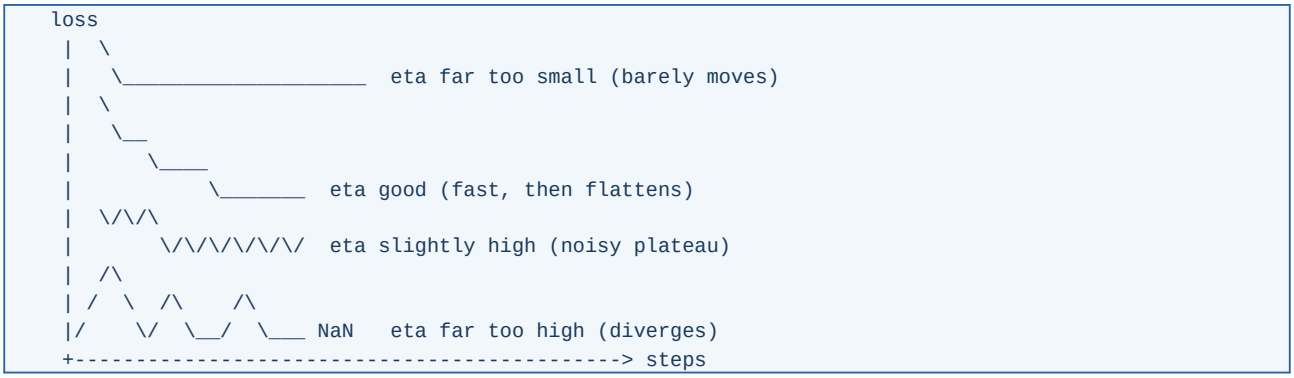

Figure 17.1 - Four learning rates, four distinctive loss-curve shapes.

What you see	Diagnosis	Action
Loss barely changes over an epoch	Learning rate 10-100x too small	Multiply by 10 and retry; run the LR range test
Smooth fall then a flat floor	Healthy	Add a decay schedule to squeeze the last few percent
Falls then plateaus with visible noise	Slightly too high for the final phase	Cosine or step decay; this is exactly what a schedule fixes
Spikes upward, then NaN	Far too high, or exploding gradients	Reduce by 10x, add gradient clipping, check for bad inputs
Falls, then rises steadily	Not an LR problem - this is overfitting	Regularise; early stop

17.7 Momentum, seen as a ball on a surface

WHY MOMENTUM IS NOT JUST 'BIGGER STEPS'

Picture a narrow valley whose floor slopes gently towards the minimum while its walls are steep. Plain gradient descent is dominated by the steep direction: it bounces from wall to wall and creeps along the floor. Momentum accumulates a velocity, and because the wall-bouncing components alternate in sign they CANCEL, while the floor component always points the same way and ACCUMULATES. With beta = 0.9 the consistent direction is amplified by roughly 1/(1 - 0.9) = 10x, and the oscillation is damped. That is why momentum both stabilises and accelerates, which sounds contradictory until you see the cancellation.

17.8 Gradient clipping and stability

```
if ||g|| > c :   g <- c * g / ||g||        (clip by global norm)
```

Clip by **global norm** (across all parameters), not per-parameter, so the update direction is preserved. c = 1.0 is a standard default for Transformers and RNNs. If you must clip constantly, your learning rate is too high or your data contains pathological samples - clipping is a seatbelt, not a steering wheel.

SHARPNESS, FLATNESS AND SAM

Solutions in flat regions of the loss surface tend to generalise better than those in sharp ones, because a flat minimum is robust to the shift between the training and test distributions. Sharpness-Aware Minimisation (SAM) makes this explicit: it takes an ascent step to the worst point within a small neighbourhood, computes the gradient THERE, and applies it at the original point. It roughly doubles the cost per step and reliably buys a point or two of accuracy on vision benchmarks.

Exercises

1. Implement SGD, momentum, RMSProp and Adam in 20 lines each and race them on the Rosenbrock function, plotting the trajectories.
2. Run the LR range test on a small CNN and compare the picked LR against a manual sweep of 5 values.
3. Train the same model with cosine decay, step decay and a constant LR to the same number of epochs and compare final accuracy.

18. Normalization Layers

Normalisation layers rescale intermediate activations so that each layer sees inputs with a stable distribution. They made deep networks trainable at practical learning rates, and every modern architecture contains one variety or another.

18.1 Batch normalisation

```
mu_B    = (1/m) SUM_i x_i                    (per channel, over the batch)

var_B   = (1/m) SUM_i (x_i - mu_B)^2

x_hat_i = (x_i - mu_B) / sqrt(var_B + eps)

y_i     = gamma * x_hat_i + beta             (learned scale and shift)
```

gamma and beta preserve expressiveness - the layer can undo the normalisation if that is what the loss prefers. At inference time the batch statistics are replaced by **running averages** collected during training, which is exactly what `model.eval()` switches on.

Benefit	Mechanism
Higher learning rates are stable	Rescaling makes the loss surface smoother and bounds the effect of a large step
Reduced sensitivity to initialisation	The layer re-centres and re-scales whatever arrives
A mild regularising effect	Each sample's normalisation depends on the random batch it landed in - noise that acts like dropout
Faster convergence	Typically 2-5x fewer epochs on deep convnets

BATCH NORM'S FAILURE MODES

(1) Small batches: with batch size 2-8 the statistics are noisy and performance collapses - a real problem for detection and segmentation, where images are large. (2) Train/test mismatch: running statistics differ from batch statistics, causing a gap that appears only at evaluation. (3) Sequence models: statistics vary with position and padding. (4) Distributed training needs SyncBatchNorm to pool statistics across GPUs. (5) It leaks information between samples in a batch, which breaks some privacy and contrastive setups.

18.2 The alternatives, and how they differ

```
Tensor (N, C, H, W). Shaded = the elements averaged together.

BatchNorm    : normalise over (N, H, W)  for each channel C
LayerNorm    : normalise over (C, H, W)  for each sample N
InstanceNorm : normalise over (H, W)     for each (N, C)
GroupNorm    : normalise over (C/g, H, W) for each sample and group
RMSNorm      : LayerNorm without mean subtraction; divide by RMS only
```

Figure 18.1 - The normalisation family differs only in which axes are pooled.

Layer	Depends on batch?	Standard use
BatchNorm	Yes	CNNs with batch >= 32
LayerNorm	No	Transformers, RNNs, any variable-length input
RMSNorm	No	Modern LLMs - cheaper than LayerNorm, equally effective
GroupNorm	No	Detection/segmentation with small batches (g = 32 is a good default)
InstanceNorm	No	Style transfer, image generation

Layer	Depends on batch?	Standard use
Weight norm / spectral norm	No	Reparameterise or constrain the weights themselves; spectral norm stabilises GAN discriminators

18.3 Placement: pre-norm versus post-norm

```
Post-norm (original Transformer):  x <- Norm( x + Sublayer(x) )

Pre-norm  (modern default):        x <- x + Sublayer( Norm(x) )
```

Pre-norm keeps a clean identity path through the whole stack, so gradients reach the first layer without passing through a normalisation, and deep models train without a delicate warmup. Post-norm can reach slightly better final quality but is much harder to train deep. Every large language model of the last few years uses pre-norm (usually pre-RMSNorm).

WHAT NORMALISATION IS ACTUALLY DOING

The original 'internal covariate shift' explanation has not held up well under scrutiny. The better-supported account is that normalisation reparameterises the loss surface so that it is smoother (bounded Lipschitz constant of the gradient), which permits larger stable steps. A second effect is scale invariance: with a normalisation layer downstream, scaling a weight matrix by c leaves the output unchanged, so weight decay acts on the effective learning rate rather than on the function - which is why weight decay and normalisation interact in ways that surprise people.

Exercises

1. Train a 20-layer MLP with and without BatchNorm at learning rates 0.001, 0.01, 0.1 and tabulate which combinations converge.
2. Replace BatchNorm with GroupNorm in a small CNN and compare accuracy at batch sizes 64, 8 and 2.
3. Derive the BatchNorm backward pass and verify it numerically.

19. Regularization for Deep Networks

Deep networks can memorise random labels, so they can certainly memorise your training set. Regularisation is how you push them towards solutions that generalise. The techniques below are cumulative - a strong recipe uses several at once.

19.1 Dropout

During training, zero each unit independently with probability p and scale the survivors by 1/(1-p) so the expected activation is unchanged (inverted dropout). At evaluation, nothing is dropped.

- **Why it works:** it prevents co-adaptation - no unit can rely on a specific partner being present - and it approximates an ensemble of exponentially many sub-networks that share weights.
- **Typical values:** 0.5 for wide fully connected layers, 0.1-0.3 in Transformers, and often 0 in convnets with BatchNorm, where the normalisation already supplies noise. **Spatial dropout** (drop whole channels) is the correct variant for convolutions.
- **Variants:** DropConnect drops weights instead of units; DropPath / stochastic depth drops entire residual blocks and is standard in modern vision transformers; DropBlock drops contiguous regions of a feature map.
- **Interaction warning:** dropout before BatchNorm changes the variance the normalisation sees, producing a train/test discrepancy. Put dropout after the normalisation, or omit one of the two.

19.2 Weight decay

Weight decay shrinks weights towards zero each step. As Chapter 17 explained, use the **decoupled** form (AdamW). Two practical details are usually ignored and matter:

- **Do not decay biases or normalisation parameters.** Decaying gamma and beta fights the normalisation layer and costs accuracy. Build two parameter groups.
- Typical values: 1e-4 for convnets with SGD, 0.01-0.1 for Transformers with AdamW. It interacts with the learning rate and the schedule; tune them together.

```
decay, no_decay = [], []
for name, param in model.named_parameters():
    if not param.requires_grad:
        continue
    if param.ndim <= 1 or name.endswith('.bias') or 'norm' in name.lower():
        no_decay.append(param)          # biases, LayerNorm/BatchNorm terms
    else:
        decay.append(param)             # weight matrices and conv kernels

opt = torch.optim.AdamW([
    {'params': decay,    'weight_decay': 0.05},
    {'params': no_decay, 'weight_decay': 0.0},
], lr=3e-4, betas=(0.9, 0.95))
```

Listing 19.1 - Correct parameter grouping for weight decay.

19.3 Early stopping

Monitor a validation metric, keep the best checkpoint, and stop when it has not improved for `patience` evaluations. Two rules: stop on the metric you actually care about, not on the training loss; and restore the best weights rather than keeping the last ones. Patience of 5-20 evaluations is typical; with a cosine schedule it is often better to train the full schedule and simply keep the best checkpoint.

19.4 Augmentation and label-level regularisers

Technique	What it does	Effect
Mixup	Train on convex combinations of two samples AND their labels	Smoother decision boundaries, better calibration
CutMix	Paste a patch of one image into another; mix labels by area	Strong for classification; forces use of the whole object
CutOut / random erasing	Blank a random rectangle	Robustness to occlusion
RandAugment / TrivialAugment	Sample augmentation ops randomly with one or two hyperparameters	Near-AutoAugment quality without the search cost
Label smoothing	Targets 1-eps instead of 1	Less overconfidence, better calibration
Noisy student / self-training	Pseudo-label unlabelled data, train a larger student with noise	Large gains when unlabelled data is plentiful

```
Mixup:   x_mix = lam x_i + (1-lam) x_j,   y_mix = lam y_i + (1-lam) y_j

              lam ~ Beta(alpha, alpha),  alpha in [0.1, 0.4] typically
```

19.5 Ensembling for deep networks

- **Independent seeds:** train the same architecture 3-5 times and average the softmax outputs. Reliably the largest easy gain, and it also gives a usable uncertainty estimate (deep ensembles, Chapter 33).
- **Snapshot ensembles:** use a cyclic learning rate and save a checkpoint at each minimum; you get an ensemble for the price of one run.
- **Stochastic Weight Averaging (SWA):** average the WEIGHTS of checkpoints from the late phase of training with a constant or cyclic learning rate. Costs one model at inference, finds flatter minima, and is essentially free. Remember to recompute BatchNorm statistics afterwards.
- **Exponential moving average (EMA)** of weights: keep a shadow copy updated as `ema <- d*ema + (1-d)*w` with d around 0.999 and evaluate with it. Standard practice in diffusion models and semi-supervised learning.

19.6 A regularisation recipe by data size

Situation	Recipe
Small data (<10k), pretrained backbone available	Freeze most layers, strong augmentation, dropout 0.3-0.5, weight decay, early stopping, 5-seed ensemble
Medium data (10k-1M)	Full fine-tune or train from scratch, RandAugment + Mixup, label smoothing 0.1, cosine schedule, EMA
Large data (>10M)	Weak augmentation, little or no dropout, weight decay, large batch with warmup - the data is the regulariser
Noisy labels	Label smoothing, symmetric or generalised cross-entropy, co-teaching, early stopping (networks fit clean data first)

Exercises

1. Train a CNN on 5,000 CIFAR images with no regularisation, then add augmentation, then Mixup, then an ensemble; report the accuracy after each addition.
2. Show that inverted dropout preserves the expected activation, and measure the variance it introduces.
3. Implement SWA over the last 25% of training and compare against the best single checkpoint.

20. Training in Practice: A Debugging Playbook

Most of the time lost in deep learning is lost to bugs that produce a plausible-looking but wrong result. This chapter is the ordered checklist that finds them.

20.1 Start correctly

The first hour of any new model

- Overfit a single batch of 8 samples to near-zero loss. If you cannot, the bug is in the model, loss or data pipeline - not in the hyperparameters. This is the single most valuable test in deep learning.
- Verify the loss at initialisation: for K balanced classes it should be about log(K) - 2.30 for 10 classes. A wildly different value means the output layer or the loss is wrong.
- Print shapes at every stage, once, and check them against what you intended.
- Visualise a batch after augmentation, with its labels. Half of all data bugs are visible immediately - wrong channel order, labels off by one, normalisation applied twice.
- Check the label mapping in both directions, and confirm that class indices in the loss match the ones in your evaluation code.
- Fix all seeds and record them; log the git commit, the config, and the data version.

20.2 Symptom-to-cause table

Symptom	Likely causes	What to try
Loss is NaN	Learning rate too high; log(0); division by zero; fp16 overflow; bad input values	Lower LR; use logits-based losses; add eps; enable loss scaling; assert finite inputs
Loss does not move	LR too low; zero_grad missing; frozen parameters; dead ReLUs; broken data pipeline	LR range test; check requires_grad; check gradient norms per layer
Train loss falls, validation does not	Overfitting, or a train/validation mismatch	More regularisation and data; verify that both use identical preprocessing
Validation better than training	Dropout/augmentation active only in training (often normal), or leakage	Compare at eval mode on the same data; audit the split
Metrics good offline, bad in production	Leakage, distribution shift, or skew between training and serving features	Audit features; re-split by time; log serving inputs and compare distributions
Results change wildly between runs	Seed sensitivity, too-small validation set, unstable LR	Average over seeds; enlarge validation; lower LR; add warmup
GPU out of memory	Batch too large; activations retained; memory leak from keeping loss tensors	Reduce batch; use AMP and checkpointing; store `loss.item()`, not the tensor
Training is slow	Data loading is the bottleneck; small batch; unfused ops; CPU-GPU sync per step	More workers + pin_memory; profile; AMP; torch.compile; avoid `.item()` inside the loop

20.3 Instrumentation worth having from day one

- Log the training loss, the validation metric, the learning rate, the gradient global norm, and the weight norm - all against step, not epoch.
- Log per-layer gradient norms occasionally; a layer with a norm 1000x different from its neighbours is a bug.
- Log a few predictions and their inputs every N steps; looking at the data is diagnostic in a way that scalars are not.
- Track throughput (samples/second) and GPU utilisation. Under 80% utilisation almost always means the data pipeline, not the model.

- Use an experiment tracker (MLflow, Weights & Biases, TensorBoard, or a CSV plus a config file) so that a result from three weeks ago can be reproduced.

20.4 Mixed precision and throughput

```
scaler = torch.amp.GradScaler('cuda')
for xb, yb in train_dl:
    opt.zero_grad(set_to_none=True)
    with torch.amp.autocast('cuda', dtype=torch.bfloat16):
        loss = lossf(model(xb), yb)          # matmuls run in low precision
    scaler.scale(loss).backward()            # loss scaling avoids fp16
    scaler.unscale_(opt)                     # underflow in the gradients
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    scaler.step(opt); scaler.update()
```

Listing 20.1 - Automatic mixed precision: roughly 2x faster and half the activation memory. bfloat16 has fp32's exponent range and usually needs no loss scaling; fp16 does.

Lever	Typical speedup	Cost
Mixed precision (bf16/fp16)	1.5-3x	Rare numerical issues
torch.compile / graph capture	1.1-2x	Compile time, occasional graph breaks
Larger batch + scaled LR	up to linear	Memory; needs warmup
Better data pipeline (workers, prefetch, webdataset)	often 2x+	Engineering time
Gradient checkpointing	0.7x speed	Enables much larger models
Multi-GPU DDP	near-linear	Communication; code complexity
FSDP / ZeRO sharding	enables huge models	Communication overhead

20.5 Reproducibility

- Seed Python, NumPy and the framework; set `torch.use_deterministic_algorithms(True)` when you need bit-exactness, and accept the slowdown.
- Pin library versions and record the CUDA/cuDNN version - kernel changes alter results.
- Version the data, not just the code. A hash of the dataset belongs in the run log.
- Save the full config with every checkpoint. A checkpoint whose hyperparameters are unknown is close to worthless.
- Accept that exact reproducibility across hardware is often impossible; aim instead for statistical reproducibility over seeds.

Exercises

1. Deliberately introduce four bugs (missing zero_grad, wrong label order, missing eval mode, unnormalised inputs) and practise finding each from the loss curve alone.
2. Profile a training step and determine whether you are data-bound or compute-bound.
3. Reproduce one of your earlier results from its config and checkpoint only. If you cannot, fix your logging before doing anything else.

PART IV

Architectures

Convolutional networks, recurrent networks, attention and Transformers, large language models, generative models, graph networks and self-supervised learning.

21. Convolutional Neural Networks

A dense layer connecting a 224x224x3 image to 1,000 hidden units needs 150 million weights, treats neighbouring pixels as unrelated, and has to relearn every pattern separately at every position. Convolution fixes all three problems with two structural assumptions: **locality** (nearby pixels are related) and **translation equivariance** (a cat is a cat wherever it appears).

21.1 The convolution operation

```
y[i, j] = SUM_c SUM_u SUM_v  w[c, u, v] * x[c, i + u, j + v]  +  b

Output size:  O = floor( (I + 2P - K) / S ) + 1

I input size, K kernel, P padding, S stride

Params of a conv layer: K*K*C_in*C_out + C_out

FLOPs (multiply-adds): K*K*C_in*C_out*H_out*W_out
```

```
input 5x5            kernel 3x3        output 3x3  (stride 1, no pad)
+--+--+--+--+--+      +--+--+--+        +--+--+--+
| a| b| c| .| .|      | 1| 0|-1|        | y| .| .|
+--+--+--+--+--+      +--+--+--+        +--+--+--+
| d| e| f| .| .|  *   | 1| 0|-1|   =    | .| .| .|
+--+--+--+--+--+      +--+--+--+        +--+--+--+
| g| h| i| .| .|      | 1| 0|-1|        | .| .| .|
+--+--+--+--+--+      +--+--+--+        +--+--+--+
y = a+d+g - (c+f+i)   <- this kernel detects vertical edges
```

Figure 21.1 - A 3x3 convolution slides one small weight matrix over the whole input: parameter sharing in action.

THREE PROPERTIES, ONE AT A TIME

SPARSE CONNECTIVITY: each output depends on a small patch, so cost is linear in image size instead of quadratic. PARAMETER SHARING: the same kernel is applied everywhere, so a feature learned in one corner transfers to all others - a massive reduction in parameters and a powerful regulariser. EQUIVARIANCE: shift the input and the feature map shifts identically; adding pooling or global average pooling converts equivariance into approximate INVARIANCE, which is what classification wants.

21.2 The standard building blocks

Layer	What it does	Typical use
Conv 3x3	The workhorse; two stacked 3x3 have the receptive field of one 5x5 with fewer parameters and more non-linearity	Everywhere
Conv 1x1	Mixes channels only; changes depth cheaply	Bottlenecks, projections, channel attention
Stride 2 conv	Downsamples while learning	Modern replacement for pooling
Max pool 2x2	Downsamples by taking the maximum	Classic; adds small translation invariance
Global average pool	Averages each channel to one number	Replaces the huge final dense layer
Transposed conv	Learned upsampling	Segmentation decoders, generators
Dilated conv	Inserts gaps in the kernel to enlarge the receptive field without cost	Segmentation, audio (WaveNet)
Depthwise separable	Depthwise spatial conv + pointwise 1x1; about 8-9x fewer FLOPs than a dense 3x3	MobileNet and every efficient on-device model

```
Standard 3x3 conv:      3*3*C_in*C_out    multiply-adds per position

Depthwise separable:    3*3*C_in + C_in*C_out

Ratio ~ 1/C_out + 1/9   ->  about 8-9x cheaper for C_out = 256
```

21.3 Receptive field - the quantity to reason about

```
RF_out = RF_in + (K - 1) * PROD(previous strides)
```

A stack of ten 3x3 convolutions with stride 1 has a receptive field of 21 pixels; if a decision needs context wider than that, the architecture cannot make it, no matter how much data you have. Downsampling, dilation and attention are the three ways to grow the receptive field quickly, and the choice among them defines much of modern architecture design.

21.4 Convolution arithmetic worked through

Three formulas do all the work. Here they are applied to the cases you will actually meet.

Input	Kernel	Padding	Stride	Output	Comment
32x32	3x3	1	1	32x32	'same' padding - the standard feature-extraction layer
32x32	3x3	0	1	30x30	'valid' - loses a ring of pixels each layer
32x32	3x3	1	2	16x16	Strided downsample, replaces pooling
224x224	7x7	3	2	112x112	The ResNet stem
32x32	1x1	0	1	32x32	Channel mixing only - spatial size untouched

```
O = floor( (I + 2P - K) / S ) + 1

Check the third row: (32 + 2 - 3)/2 + 1 = floor(15.5) + 1 = 16.  OK.
```

21.5 Cost of one real layer, counted exactly

Take a 3x3 convolution with 64 input channels and 128 output channels, operating on a 32x32 feature map:

```
Parameters = K*K*C_in*C_out + C_out

  = 3*3*64*128 + 128 = 73,728 + 128 = 73,856

Multiply-adds = K*K*C_in*C_out*H_out*W_out

  = 3*3*64*128*32*32 = 75,497,472  (~75 M per image)
```

Now replace it with a depthwise separable block - a 3x3 depthwise convolution followed by a 1x1 pointwise convolution:

```
Parameters = (3*3*64 + 64) + (64*128 + 128) = 640 + 8,320 = 8,960

Multiply-adds = (3*3*64 + 64*128) * 32*32 = 8,978,432   (~9 M)

Parameters:  8.2x fewer.    Compute:  8.4x fewer.
```

WHERE THE SAVING COMES FROM

A standard convolution does two jobs at once: it mixes across SPACE (the 3x3 neighbourhood) and across CHANNELS (all 64 inputs feeding all 128 outputs), and it pays the product of the two costs. Separating them turns a product into a sum: 3*3*64 for space plus 64*128 for channels. The saving is roughly 1/C_out + 1/K^2, which for typical values is 8-9x. Every efficient mobile architecture is built on this one observation.

21.6 Receptive field, computed layer by layer

A unit's receptive field is the patch of input that can influence it. Grow it with the recurrence `RF <- RF + (K - 1) * (product of all previous strides)`:

Layer	Kernel	Stride	Cumulative stride	Receptive field
input	-	-	1	1
conv 3x3	3	1	1	3
conv 3x3	3	1	1	5
maxpool 2x2	2	2	2	6
conv 3x3	3	1	2	10
conv 3x3	3	1	2	14
maxpool 2x2	2	2	4	16
conv 3x3	3	1	4	24

Two practical consequences. Downsampling grows the receptive field far faster than depth alone - after the second pool, each extra 3x3 convolution adds 8 pixels of context instead of 2. And if your task needs context wider than the final receptive field - a 200-pixel object seen by units with a 24-pixel view - no amount of training will fix it; the architecture is the constraint. Dilated convolutions, more downsampling, or attention are the three ways out.

21.7 A short history worth knowing

Model	Year	Contribution
LeNet-5	1998	Conv + pool + dense; digits; the template
AlexNet	2012	ReLU, dropout, GPUs, augmentation; won ImageNet by a huge margin and started the era
VGG	2014	Uniform 3x3 stacks; showed depth matters; very heavy
GoogLeNet / Inception	2014	Multi-scale branches; 1x1 bottlenecks
ResNet	2015	Residual connections; 152 layers trainable; the single most influential idea in the list
DenseNet	2016	Concatenate all previous feature maps
MobileNet / ShuffleNet	2017	Depthwise separable convolutions for phones
EfficientNet	2019	Compound scaling of depth, width and resolution together
ConvNeXt	2022	A convnet modernised with Transformer-era training recipes; matches ViT

```
Residual block:   y = x + F(x)      (identity path + learned residual)

Bottleneck block: 1x1 reduce -> 3x3 -> 1x1 expand, all with a skip
```

21.8 Beyond classification

- **Object detection:** two-stage (Faster R-CNN: propose regions, then classify) versus one-stage (YOLO, SSD, RetinaNet with focal loss). Metrics: mAP at IoU thresholds. DETR reframes detection as set prediction with a Transformer and removes anchors and NMS.
- **Semantic segmentation:** per-pixel classification. U-Net's encoder-decoder with skip connections is still the standard in medical imaging; DeepLab adds dilated convolutions and multi-scale pooling.
- **Instance and panoptic segmentation:** Mask R-CNN adds a mask head to detection; panoptic unifies things and stuff.
- **Video:** 3D convolutions, two-stream (RGB + optical flow), or factorised (2+1)D convolutions; increasingly video Transformers.
- **Non-image uses:** 1D convolutions over time series and audio, and over text characters; convolution is about locality, not about pixels.

21.9 Transfer learning - the default workflow for vision

```
import torch, torchvision as tv, torch.nn as nn

m = tv.models.resnet50(weights=tv.models.ResNet50_Weights.IMAGENET1K_V2)
for p in m.parameters():
    p.requires_grad = False              # stage 1: freeze the backbone
m.fc = nn.Linear(m.fc.in_features, NUM_CLASSES)   # new head, trainable

# stage 1: train the head only, LR ~1e-3, a few epochs
# stage 2: unfreeze the last block(s) and fine-tune with a small LR
for p in m.layer4.parameters():
    p.requires_grad = True
opt = torch.optim.AdamW([
    {'params': m.layer4.parameters(), 'lr': 1e-4},   # discriminative LRs:
    {'params': m.fc.parameters(),     'lr': 1e-3},   # deeper = smaller LR
], weight_decay=1e-4)
```

Listing 21.1 - Two-stage fine-tuning with discriminative learning rates.

HOW MUCH TO FINE-TUNE

Small dataset and similar domain: train the head only. Small dataset, different domain: fine-tune the last block or two with a small learning rate. Large dataset: fine-tune everything, still with a lower learning rate for early layers. Always keep the preprocessing (resize, mean/std normalisation) identical to what the backbone was pretrained with - this is a surprisingly common silent failure.

Exercises

1. Compute the receptive field, parameter count and FLOPs of a 5-layer CNN by hand, then verify with a profiler.
2. Implement a residual block and train a 20-layer plain CNN and a 20-layer ResNet on CIFAR-10; compare convergence.
3. Replace every 3x3 convolution in a small CNN with a depthwise separable block; report the accuracy, parameter and latency changes.

22. Sequence Models: RNN, LSTM and GRU

Sequences - text, speech, sensor streams, prices - have order and variable length. A recurrent network processes them one step at a time, carrying a hidden state that summarises everything seen so far.

```
h_t = tanh( W_hh h_(t-1) + W_xh x_t + b )

y_t = W_hy h_t + b_y
```

The same weights are used at every timestep - parameter sharing across time, exactly analogous to a convolution's sharing across space. Training uses **backpropagation through time**: unroll the network over the sequence and apply ordinary backpropagation, usually truncated to a window of a few hundred steps to bound memory.

22.1 Why plain RNNs fail on long sequences

The gradient over T steps contains a factor `PROD_t (W_hh^T diag(tanh'))`. A repeated matrix product either vanishes or explodes exponentially in T. Exploding is easy to fix with clipping; vanishing is the hard one, and it means the network cannot connect an event at step 5 to a consequence at step 500.

22.2 LSTM: a cell state with gates

```
f_t = sigma( W_f [h_(t-1), x_t] + b_f )        forget gate
i_t = sigma( W_i [h_(t-1), x_t] + b_i )        input gate
g_t = tanh ( W_g [h_(t-1), x_t] + b_g )        candidate
o_t = sigma( W_o [h_(t-1), x_t] + b_o )        output gate
c_t = f_t * c_(t-1) + i_t * g_t                CELL STATE (additive!)
h_t = o_t * tanh(c_t)
```

THE ONE LINE THAT MATTERS

c_t = f_t * c_(t-1) + i_t * g_t. The cell state is updated ADDITIVELY, not by a matrix multiplication, so the gradient path along c is a product of forget gates rather than of weight matrices. If the forget gate stays near 1, information and gradient flow for hundreds of steps. This is the same trick as a residual connection, invented for time.

GRU merges the forget and input gates into one update gate and drops the separate cell state: about 25% fewer parameters, usually indistinguishable in accuracy, and slightly faster. Try both; there is no reliable winner.

- Initialise the **forget-gate bias to 1** so the cell remembers by default - a small change that historically made LSTMs much easier to train.
- **Bidirectional** RNNs run one pass in each direction and concatenate; only usable when the whole sequence is available (not for streaming or generation).
- Stack 2-4 layers; deeper recurrent stacks rarely help without residual connections between layers.
- **Variational (locked) dropout** applies the same mask at every timestep; ordinary per-step dropout destroys the recurrent signal.
- Pack padded sequences (`pack_padded_sequence`) so the network never consumes padding tokens.

22.3 Sequence-to-sequence and the birth of attention

An encoder RNN compresses the input into a single fixed vector; a decoder RNN generates the output from it. The bottleneck is obvious: one vector cannot hold a 50-word sentence. Attention (Bahdanau, 2014) let the decoder look back at **all** encoder states, weighting them per output step:

```
score(s_t, h_i) -> alpha_ti = softmax_i(score)

context c_t = SUM_i alpha_ti h_i

decoder consumes [s_t ; c_t]
```

This removed the bottleneck, gave interpretable alignments, and led directly to the conclusion of the next chapter: if attention is doing the work, the recurrence can be dropped entirely.

22.4 When to still use an RNN in the 2020s

- Streaming and low-latency inference with strict memory limits - an LSTM has O(1) state per step, while a Transformer's KV cache grows with length.
- Small on-device models over sensor streams, where a two-layer GRU of 50k parameters is enough and a Transformer is not affordable.
- Very long sequences where quadratic attention is prohibitive - though **state-space models** (S4, Mamba) are now the stronger option: they keep the recurrent O(1) inference state while training in parallel like a convolution, and are competitive with Transformers on long-context tasks.
- Otherwise, for text and most sequence modelling, the default is a Transformer.

Exercises

1. Implement an LSTM cell from the equations above and check it against `nn.LSTMCell`.
2. Train a plain RNN, an LSTM and a GRU on the copy task (repeat a sequence after a delay of T steps) for T = 10, 50, 200 and plot accuracy against T.
3. Add Bahdanau attention to a small seq2seq translation model and visualise the alignment matrix.

23. Attention and the Transformer

The Transformer replaced recurrence with attention entirely. Its advantage is not only accuracy: every position is processed in parallel during training, so the architecture scales with hardware in a way RNNs never could. Understanding this chapter thoroughly is the highest-value investment in modern deep learning.

23.1 Scaled dot-product attention

```
Attention(Q, K, V) = softmax( Q K^T / sqrt(d_k) ) V

Q = X W_Q   (n, d_k)      queries: what am I looking for?
K = X W_K   (n, d_k)      keys:    what do I contain?
V = X W_V   (n, d_v)      values:  what do I pass on?
```

THE DATABASE ANALOGY

Each token emits a QUERY describing what it needs, and every token offers a KEY describing what it has. The dot product query-dot-key scores the match; softmax turns the scores into weights that sum to one; the output is the weighted average of the VALUES. Unlike a hash lookup, the match is soft - every token contributes something, in proportion to relevance.

WHY DIVIDE BY SQRT(D_K)

If the components of q and k are independent with zero mean and unit variance, then q.k has variance d_k. With d_k = 64 the logits have a standard deviation of 8, and softmax of such large values is nearly one-hot with vanishing gradients. Dividing by sqrt(d_k) restores unit variance and keeps the softmax in its responsive range.

23.2 Multi-head attention

```
head_i = Attention(X W_Q^i, X W_K^i, X W_V^i),   i = 1..h

MHA(X) = Concat(head_1, ..., head_h) W_O

typically d_k = d_v = d_model / h
```

Splitting the representation into h heads lets the layer attend to different relationships at once - one head tracking syntactic dependency, another coreference, another position. The total compute is the same as one head of full width, so it is free capacity in the useful sense.

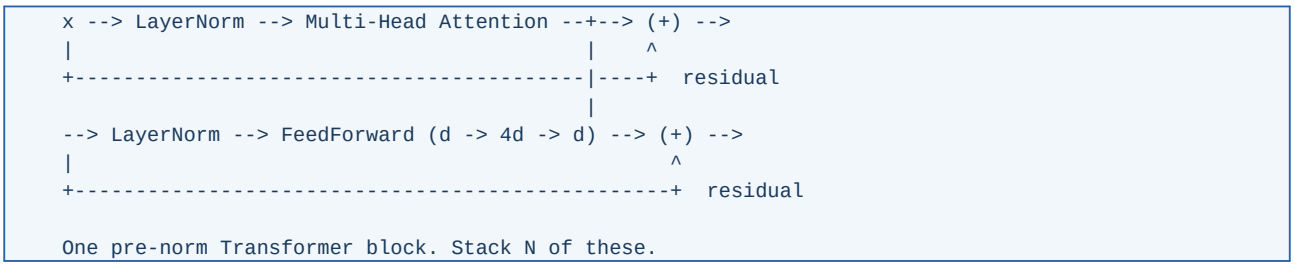

Figure 23.1 - The block: attention mixes across tokens, the feedforward mixes across channels, residuals carry the signal.

23.3 The other components

- **Position-wise feedforward:** two linear layers with a non-linearity, usually expanding by 4x. It holds most of the parameters and, on current evidence, most of the model's factual knowledge. Modern LLMs use SwiGLU here, which needs three matrices at roughly 2/3 the width.
- **Residual connections and normalisation** around every sublayer (pre-norm; see Chapter 18).

- **Causal masking** in decoders: set the scores for future positions to -inf before the softmax so a token can never see its own future. This single mask is what makes parallel training of an autoregressive model possible.
- **Cross-attention** in encoder-decoder models: queries come from the decoder, keys and values from the encoder.

23.4 Positional information

Attention is permutation-equivariant - shuffle the tokens and the outputs shuffle with them. Order must be injected explicitly.

Scheme	How	Properties
Sinusoidal (original)	Add fixed sin/cos of varying frequency	No parameters; some extrapolation to longer sequences
Learned absolute	A trainable embedding per position	Simple; cannot exceed the trained length
Relative (T5, Shaw)	Bias the attention logits by the distance i-j	Generalises across lengths; used in encoder models
RoPE (rotary)	Rotate q and k by an angle proportional to position	Relative by construction, extrapolates well, the de facto standard in modern LLMs; extendable by frequency scaling (YaRN, NTK)
ALiBi	Add a linear distance penalty to attention logits	Very simple, strong length extrapolation

23.5 Complexity and the efficiency ladder

```
Time:   O(n^2 d)      Memory (naive):  O(n^2)

n = sequence length, d = model dimension
```

Quadratic cost in sequence length is the Transformer's defining limitation, and a decade of work exists to soften it:

Technique	Idea	Status
FlashAttention	Tile the computation in SRAM; never materialise the n x n matrix. Exact, IO-aware	Universal in practice; the first thing to enable
Multi-query / grouped-query attention	Share K and V across heads	Shrinks the KV cache 8-64x; standard in modern LLMs
Sliding window / local attention	Attend within a window only	Mistral-style; combine with a few global tokens
Sparse patterns (Longformer, BigBird)	Local + global + random	Long documents
Linear attention (Performer, Linformer)	Kernel or low-rank approximation of softmax	O(n); some quality loss
State-space models (S4, Mamba)	Structured recurrence, parallel training, O(1) inference state	Strong for very long context
KV cache quantisation / paging	Store the cache in 8 or 4 bits, in pages	The main memory lever at serving time

23.6 Implementing attention

```
import torch, torch.nn as nn, torch.nn.functional as F

class MultiHeadSelfAttention(nn.Module):
```

```
    def __init__(self, d_model, n_heads, causal=True, p=0.0):
        super().__init__()
        assert d_model % n_heads == 0
        self.h, self.dk = n_heads, d_model // n_heads
        self.qkv  = nn.Linear(d_model, 3 * d_model, bias=False)
        self.proj = nn.Linear(d_model, d_model, bias=False)
        self.causal, self.p = causal, p

    def forward(self, x):                       # x: (B, T, D)
        B, T, D = x.shape
        q, k, v = self.qkv(x).split(D, dim=2)
        # (B, T, D) -> (B, heads, T, dk)
        q = q.view(B, T, self.h, self.dk).transpose(1, 2)
        k = k.view(B, T, self.h, self.dk).transpose(1, 2)
        v = v.view(B, T, self.h, self.dk).transpose(1, 2)
        # FlashAttention kernel: exact, memory-efficient, fused
        y = F.scaled_dot_product_attention(
                q, k, v, is_causal=self.causal,
                dropout_p=self.p if self.training else 0.0)
        y = y.transpose(1, 2).contiguous().view(B, T, D)
        return self.proj(y)

# The explicit form, for understanding only:
#   att = (q @ k.transpose(-2,-1)) / self.dk**0.5
#   att = att.masked_fill(mask == 0, float('-inf')).softmax(-1)
#   y   = att @ v
```

Listing 23.1 - Multi-head self-attention. The commented lines are the equations; the live code is what you should actually run.

23.7 Attention computed on three tokens

Nothing clarifies attention like doing it with small numbers. Three tokens, d_k = 2, with queries and keys chosen so the pattern is readable:

```
                        Q = K = [ 1  0 ]        V = [ 1  0 ]
                                [ 0  1 ]            [ 0  2 ]
                                [ 1  1 ]            [ 3  1 ]

            token 1 points along x, token 2 along y, token 3 along both.
```

Step 1 - scores

```
                    Q K^T / sqrt(2) = [ 0.707   0.000   0.707 ]
                                      [ 0.000   0.707   0.707 ]
                                      [ 0.707   0.707   1.414 ]
```

Row 3 is the interesting one: token 3 scores 1.414 against itself and 0.707 against each of the others, because its direction overlaps both. Rows 1 and 2 each score 0 against the orthogonal token - orthogonal means 'unrelated', and the dot product says so numerically.

Step 2 - softmax over each row

```
          weights = [ 0.401   0.198   0.401 ]     <- token 1 attends to 1 and 3
                    [ 0.198   0.401   0.401 ]     <- token 2 attends to 2 and 3
                    [ 0.248   0.248   0.504 ]     <- token 3 attends mostly to 3
```

Step 3 - weighted sum of values

```
output row 1 = 0.401*[1,0] + 0.198*[0,2] + 0.401*[3,1] = [1.604, 0.797]

output row 2 = 0.198*[1,0] + 0.401*[0,2] + 0.401*[3,1] = [1.401, 1.203]

output row 3 = 0.248*[1,0] + 0.248*[0,2] + 0.504*[3,1] = [1.759, 1.000]
```

Each token has been replaced by a blend of all tokens' values, weighted by relevance. That is the entire operation. Everything else in a Transformer - multiple heads, the projections W_Q/W_K/W_V, residuals, the feedforward block - is packaging around these three steps.

WATCHING THE SQRT(D_K) DO ITS JOB

Without the scaling, row 1's scores are 1, 0, 1 and the softmax gives 0.422, 0.155, 0.422 - a bit sharper. With d_k = 2 the difference is small, but scores grow like sqrt(d_k): at d_k = 64 an unscaled logit gap of 8 instead of 1 turns the softmax nearly one-hot, its gradient to nearly zero, and training stalls at initialisation. The division is not cosmetic; it is what keeps the softmax differentiable at realistic widths.

23.8 Why causal masking makes parallel training possible

```
Mask for a 4-token causal decoder (1 = allowed, . = -inf before softmax)

          attends to:  t1  t2  t3  t4
   query t1            1    .   .   .      token 1 sees only itself
   query t2            1    1   .   .      token 2 sees 1 and 2
   query t3            1    1   1   .
   query t4            1    1   1   1

One forward pass computes the prediction for EVERY position at once,
each conditioned only on its own past. An RNN needs 4 sequential steps
to do the same thing. This is why Transformers train so much faster.
```

Figure 23.2 - The causal mask: n training examples from one sequence, in one parallel pass.

At **inference** the advantage disappears - tokens must still be generated one at a time, each attending to all previous ones. That asymmetry (parallel training, sequential generation) is why the KV cache exists, why generation is memory-bandwidth bound, and why speculative decoding is worth the complexity. Chapter 24 follows the consequences.

23.9 Sizing a Transformer block

For d_model = 512 with 8 heads and a 4x feedforward expansion:

Component	Shape	Parameters
W_Q, W_K, W_V	3 x (512 x 512)	786,432
W_O (output projection)	512 x 512	262,144
Feedforward up	512 x 2048	1,048,576
Feedforward down	2048 x 512	1,048,576
2 x LayerNorm	2 x 2 x 512	2,048
Total per block	-	~3.15 M
12 blocks	-	~37.7 M, plus embeddings

Note the split: attention holds about a third of the parameters and the feedforward block about two thirds. That ratio holds across most model sizes, and it is why pruning and quantization work on the feedforward matrices give the largest returns (Chapters 28-29), and why mixture-of-experts replaces exactly that block.

23.10 The three architectural families

Family	Masking	Trained by	Examples	Best at
Encoder-only	Bidirectional	Masked token prediction	BERT, RoBERTa, DeBERTa, ViT	Classification, retrieval, embeddings
Decoder-only	Causal	Next-token prediction	GPT family, Llama, Mistral	Generation, few-shot, chat - the dominant design
Encoder-decoder	Both	Span corruption / seq2seq	T5, BART, Whisper	Translation, summarisation, speech

23.11 Transformers outside text

- **Vision Transformer (ViT):** split the image into 16x16 patches, embed each as a token, add positional embeddings, run a standard encoder. Needs large data or strong augmentation and distillation (DeiT) because it lacks the convolutional inductive bias - which is also why it eventually surpasses convnets given enough data.
- **Swin Transformer:** hierarchical windows with shifting - reintroduces locality and multi-scale structure for detection and segmentation.
- **Audio:** Whisper (encoder-decoder over log-mel spectrograms), Conformer (convolution + attention).
- **Multimodal:** CLIP aligns image and text encoders with a contrastive loss; modern vision-language models feed image patch embeddings into an LLM's token stream.
- **Time series and sensor data:** patch-based Transformers (PatchTST) are now competitive with classical methods, though strong linear baselines remain surprisingly hard to beat.

Exercises

1. Implement attention with explicit einsum and verify it matches `scaled_dot_product_attention` to 1e-5.
2. Remove the sqrt(d_k) scaling and plot the entropy of the attention weights during the first 200 steps.
3. Train a 4-layer character-level decoder-only Transformer on a small text corpus and sample from it. This is the single best exercise in this book.
4. Visualise attention maps for a trained model and check whether any head has a recognisable role.

24. Large Language Models

An LLM is a decoder-only Transformer trained to predict the next token on a very large text corpus, then adapted to follow instructions. Everything surprising about them - in-context learning, reasoning, code generation - emerges from that one objective at scale.

24.1 Tokenization

- **Byte-pair encoding (BPE)** and its variants (WordPiece, Unigram, SentencePiece) build a vocabulary by repeatedly merging the most frequent adjacent pair, producing subword units. Common words become one token; rare words split into pieces; nothing is ever out-of-vocabulary.
- Typical vocabularies are 32k-256k tokens. English averages roughly 0.75 words per token; code and non-Latin scripts are considerably less efficient, which is a real cost and fairness issue.
- Tokenisation explains several famous failure modes: character counting, arithmetic on long numbers, and reversal tasks are hard because the model never sees characters.
- Byte-level fallbacks guarantee coverage of any input; some recent work removes tokenisation entirely in favour of byte or patch-level models.

24.2 Pretraining

```
Objective:  maximise SUM_t log P(x_t | x_<t ; theta)

Loss:       cross-entropy;   Perplexity = exp(cross-entropy)
```

Ingredient	Current practice
Data	Trillions of tokens: filtered web crawl, code, books, scientific text. Quality filtering, deduplication and decontamination matter more than raw volume
Architecture	Decoder-only, pre-RMSNorm, RoPE, SwiGLU feedforward, grouped-query attention, no biases
Optimiser	AdamW, beta2 about 0.95, cosine decay with warmup, gradient clipping at 1.0, weight decay 0.1
Parallelism	Data + tensor + pipeline + sequence parallelism; ZeRO/FSDP sharding of optimiser state
Precision	bf16 compute with fp32 master weights; selective activation checkpointing
Context	4k-1M tokens, usually extended after the main run by adjusting RoPE frequencies and training on long documents

SCALING LAWS

Loss falls as a smooth power law in parameters N, data D and compute C - L(N) = (Nc/N)^a + irreducible. The Chinchilla result showed that for a fixed compute budget the optimum is roughly 20 tokens per parameter, and that most earlier models were badly undertrained on data rather than too small. In practice, inference cost now dominates for deployed models, so the industry deliberately overtrains smaller models far past Chinchilla optimality - a 7B model trained on 10-15T tokens is cheaper to serve forever than a 30B model trained to the same loss.

24.3 Post-training: from predictor to assistant

- **Supervised fine-tuning (SFT):** train on curated instruction-response pairs. A few thousand high-quality examples change behaviour dramatically; quality dominates quantity.
- **RLHF:** collect human preference comparisons, fit a reward model, then optimise the policy with PPO against that reward plus a KL penalty to stay near the SFT model. Powerful and operationally complex.
- **DPO and friends:** skip the explicit reward model - a closed-form loss on preference pairs directly optimises the same objective. Much simpler, now the common choice; variants include IPO, KTO, ORPO and SimPO.
- **RLAIF / Constitutional AI:** use model-generated critiques and preferences against a written set of principles, reducing the human labelling burden.

- **Reasoning training:** reinforcement learning on verifiable outcomes (maths, code tests) trains models to produce long chains of thought before answering, trading inference compute for accuracy.

24.4 Parameter-efficient fine-tuning

```
LoRA:   W' = W + (alpha/r) B A,   A in R^(r x d),  B in R^(d x r),  r << d

Train A and B only; W stays frozen. Merge B A into W at deployment for

zero added latency.
```

Method	Trainable share	Notes
Full fine-tune	100%	Best quality, needs ~16 bytes/param of optimiser + gradient memory
LoRA	0.1-1%	The default; r = 8-64, applied to attention and often MLP projections
QLoRA	0.1-1%	Base model quantised to 4-bit NF4, LoRA adapters in bf16 - fine-tunes a 70B model on a single 48 GB GPU
DoRA / rsLoRA	0.1-1%	Refinements on the LoRA parameterisation
Prefix / prompt tuning	<0.1%	Learn virtual tokens; weaker but extremely light
Adapters	1-5%	Small bottleneck modules inserted per layer; add inference latency unless merged

24.5 Inference: what actually costs money

- Generation is **memory-bandwidth bound**, not compute bound: each token requires reading all weights. This is why quantization (Chapter 28) speeds up generation almost proportionally to the bit width.
- **KV cache** size = 2 * layers * heads_kv * head_dim * seq_len * batch * bytes. For long contexts it exceeds the weights themselves; grouped-query attention, paged attention (vLLM) and cache quantisation are the standard mitigations.
- **Speculative decoding:** a small draft model proposes k tokens, the large model verifies them in one forward pass - a 2-3x speedup with identical output distribution.
- **Batching** (continuous/in-flight batching) is what makes serving economical: it converts a bandwidth-bound workload into a compute-bound one.
- **Sampling controls:** temperature, top-k, top-p (nucleus), repetition and presence penalties. Temperature 0 is deterministic-greedy; higher temperature increases diversity and error rate together.

24.6 What 'next token prediction' actually looks like

The training objective is unglamorous: given a prefix, predict the distribution over the next token. Every capability people find surprising is a side effect of doing this extremely well on a very large corpus.

```
text:      'The capital of France is Paris'
tokens:    [The] [ capital] [ of] [ France] [ is] [ Paris]

training example 1:  The                      -> ' capital'
training example 2:  The capital              -> ' of'
training example 3:  The capital of           -> ' France'
training example 4:  The capital of France    -> ' is'
training example 5:  The capital of France is -> ' Paris'

All five are computed in ONE forward pass, thanks to causal masking.
```

Figure 24.1 - One six-token sentence yields five supervised examples.

To predict ' Paris' reliably, a model must store a fact. To predict the closing bracket of a nested expression it must track state. To predict the next line of a proof it must do something that looks like reasoning. None of these were trained for directly; they are what minimising prediction error on a large enough corpus requires.

Loss value	Perplexity	Interpretation
11.0	~60,000	Untrained - uniform over the vocabulary
6.9	~1,000	Learned token frequencies only
4.0	~55	Basic grammar and local coherence
3.0	~20	Fluent text, weak factuality
2.0	~7.4	Strong modern model on general text
1.5	~4.5	Approaching the noise floor of natural text

Perplexity is just `exp(cross-entropy)` and reads as 'the model is as uncertain as if it were choosing uniformly among this many tokens'. Watch it during any language-model training run: it should start near the vocabulary size and fall fast.

24.7 The KV cache, sized with real numbers

During generation the model re-reads every previous token's keys and values. Caching them avoids recomputation, but the cache is large:

```
bytes = 2 (K and V) * layers * kv_heads * head_dim * seq_len
                    * batch * bytes_per_value

7B-class model, 32 layers, 32 heads, head_dim 128, 8k context, fp16:
2 * 32 * 32 * 128 * 8192 * 1 * 2 = 4.3 GB

Same model with grouped-query attention, 8 KV heads:
2 * 32 *  8 * 128 * 8192 * 1 * 2 = 1.07 GB    (4x smaller)
```

Compare that with the weights themselves: 7B parameters in fp16 is 14 GB. So a single 8k-context conversation adds nearly a third again on top of the model - and it scales linearly with both context length and batch size, which is why serving many users at long context is a memory problem before it is a compute problem. Grouped-query attention, paged attention and 8-bit KV quantization each attack this directly.

24.8 Prompting, from worst to best

Version	Prompt	Why it behaves better
Bad	'Summarise this.'	No audience, no length, no format - the model guesses all three
Better	'Summarise the text below in 3 bullet points for a non-technical manager.'	Audience, length and format are now constraints rather than guesses
Good	Add: 'Use only information from the text. If a fact is not stated, omit it.'	Reduces fabrication by making abstention an explicit option
Best	Add: 'Return JSON: {summary: string[], omitted_topics: string[]}' plus 2 examples	Machine-parseable, and the examples pin down the style far more precisely than adjectives can

THE TWO HIGHEST-VALUE PROMPTING HABITS

First, give the model somewhere to put its uncertainty - an 'unknown' field, permission to say the text does not answer the question, a confidence score. A model with no legitimate way to express doubt will invent an answer. Second, show rather than describe: two examples of the exact output you want beat two paragraphs describing it, because the examples constrain format, tone and depth simultaneously.

24.9 Using LLMs well

- **Prompting:** state the role, the task, the constraints, and the output format explicitly; give 2-5 examples for a format-sensitive task; ask for reasoning before the answer when the task needs it; request structured output (JSON schema) when a program will parse it.
- **Retrieval-augmented generation (RAG):** chunk documents, embed them, retrieve the top-k by vector similarity (plus keyword search - hybrid retrieval beats either alone), rerank, and put the evidence in the prompt with citations. This is the standard way to give a model private, current, verifiable knowledge without training.
- **Tool use / agents:** let the model call functions - search, a calculator, a database, code execution - and loop on the results. Reliability comes from constraining the action space and validating every result, not from longer prompts.
- **Evaluation:** build a fixed test set of real inputs with expected properties; use exact checks where possible, an LLM judge with a rubric where not, and always keep a human-reviewed sample. Track regressions like any other software test.

THE FAILURE MODES TO DESIGN AROUND

Hallucination (confident fabrication) - mitigate with retrieval, citations and abstention instructions. Prompt injection - never let untrusted text carry authority; separate data from instructions and constrain tools. Context-length degradation - relevant facts placed in the middle of a long context are recalled least well. Non-determinism - even at temperature 0, batching and kernel choice can change outputs. Data contamination - benchmark scores may reflect memorised test sets.

Exercises

1. Compute the KV-cache size for a 7B model (32 layers, 32 heads, head_dim 128) at 8k context in fp16, and again with 8 KV heads (GQA).
2. Fine-tune a small open model with LoRA on 500 instruction pairs and measure the change on a held-out set you wrote yourself.
3. Build a minimal RAG pipeline over 200 of your own documents; measure answer accuracy with and without retrieval.

25. Generative Models: Autoencoders, VAEs, GANs and Diffusion

A discriminative model learns P(y|x). A generative model learns P(x), or P(x|condition), and can therefore produce new samples. The families below differ in how they make the intractable business of modelling a high-dimensional distribution tractable.

25.1 Autoencoders

```
z = Encoder(x)      x_hat = Decoder(z)      L = ||x - x_hat||^2
```

A bottleneck forces a compressed representation. A linear autoencoder with squared loss recovers the PCA subspace exactly; non-linear ones learn curved manifolds. Variants: **denoising** (reconstruct a clean input from a corrupted one - the ancestor of both BERT and diffusion), **sparse** (penalise activations), **contractive** (penalise the Jacobian). Plain autoencoders are useful for compression, denoising and anomaly detection, but their latent space has holes - sampling a random z usually decodes to nonsense.

25.2 Variational autoencoders

A VAE makes the latent space a proper probability distribution. The encoder outputs a mean and log-variance; a sample is drawn; the decoder reconstructs. The loss is the **evidence lower bound**:

```
ELBO = E_q(z|x)[ log p(x|z) ]  -  KL( q(z|x) || p(z) )
          \_____ reconstruction _____/   \____ regulariser ____/

Reparameterisation: z = mu + sigma * eps,  eps ~ N(0, I)
```

WHY THE REPARAMETERISATION TRICK IS NECESSARY

You cannot backpropagate through 'sample from N(mu, sigma)' because the sampling node has no derivative with respect to mu and sigma. Writing z = mu + sigma * eps moves the randomness into eps, which carries no parameters, leaving a deterministic differentiable path to mu and sigma. This trick is the reason VAEs train with ordinary gradient descent, and it reappears throughout probabilistic deep learning.

- **Posterior collapse:** with a powerful decoder the KL term can drive q(z|x) to the prior and the latent is ignored. Fix with KL annealing, free bits, or a weaker decoder.
- **beta-VAE** scales the KL term to trade reconstruction quality against disentangled, interpretable latents.
- **VQ-VAE** replaces the continuous latent with a discrete codebook; it underpins modern image and audio tokenisers, which is how images enter and leave a Transformer.
- VAE samples tend to be blurry because the Gaussian likelihood averages over plausible reconstructions.

25.3 Generative adversarial networks

```
min_G max_D  E_x[ log D(x) ] + E_z[ log(1 - D(G(z))) ]
```

A generator turns noise into samples; a discriminator tries to tell real from fake; they train against each other. At the theoretical optimum the generator matches the data distribution and the discriminator is at chance everywhere. GANs produce the sharpest images of any family per unit of inference compute, and they are notoriously unstable to train.

Problem	Cause	Standard fixes
Mode collapse	The generator finds a few outputs that fool D and stops exploring	Minibatch discrimination, unrolled GAN, WGAN-GP, diverse conditioning

Problem	Cause	Standard fixes
Vanishing generator gradient	D wins too easily	Non-saturating loss, WGAN with Lipschitz constraint, weaker D
Training oscillation	It is a two-player game, not a minimisation	Two time-scale update rule (different LRs), spectral normalisation, EMA of generator weights
No usable likelihood	Implicit model	Evaluate with FID/KID and human study; do not expect a density

Landmarks: DCGAN (convolutional recipe), WGAN-GP (Wasserstein distance with a gradient penalty), Pix2Pix and CycleGAN (paired and unpaired translation), StyleGAN (style-based generator, still the reference for face synthesis), and SRGAN for super-resolution.

25.4 Diffusion models

Diffusion is now the dominant family for images, audio and video. The idea is disarmingly simple: destroy data with noise in small steps, then learn to reverse each step.

```
Forward:  q(x_t | x_(t-1)) = N( sqrt(1-b_t) x_(t-1), b_t I )

Closed form:  x_t = sqrt(a_bar_t) x_0 + sqrt(1 - a_bar_t) * eps

Training loss (DDPM):   L = E || eps - eps_theta(x_t, t) ||^2

i.e. a network that predicts the noise that was added.
```

```
x0 --noise--> x1 --noise--> ... --noise--> xT ~ N(0, I)      FORWARD
x0 <--denoise-- x1 <--denoise-- ... <--denoise-- xT          REVERSE
     ^ the network eps_theta(x_t, t) predicts the noise at each step
```

Figure 25.1 - Diffusion: a fixed corruption process and a learned reversal.

- **Architecture:** a U-Net with residual blocks, self-attention at low resolutions, and a timestep embedding; increasingly a Transformer (DiT) instead.
- **Latent diffusion** runs the process in the latent space of a VAE rather than in pixels - roughly a 48x reduction in compute, and the reason Stable Diffusion runs on consumer hardware.
- **Conditioning:** cross-attention on text embeddings; **classifier-free guidance** trains with and without the condition and extrapolates at sampling time, `eps = eps_uncond + w(eps_cond - eps_uncond)`, trading diversity for prompt fidelity.
- **Samplers:** DDPM needs hundreds of steps; DDIM, DPM-Solver++ and flow matching reduce this to 10-50. Distillation (progressive, consistency, adversarial) reaches 1-4 steps.
- **Flow matching / rectified flow** reframes the same idea as learning a velocity field along straight paths between noise and data - simpler objective, fewer steps, and the current direction of the field.

WHY DIFFUSION BEAT GANS

Diffusion replaces one impossibly hard problem - map noise to a complex distribution in a single shot, judged by an adversary - with a thousand easy, stable regression problems. The training signal is a plain mean-squared error, so there is no minimax game, no mode collapse, and scaling behaves predictably. The cost is inference: many network evaluations per sample, which is exactly what step-distillation research attacks.

25.5 The other families, briefly

Family	Mechanism	Trade-off
Autoregressive (PixelCNN, LLMs)	Factorise P(x) into a product of conditionals	Exact likelihood, best quality on text; slow sequential sampling
Normalising flows	Invertible transforms with tractable Jacobians	Exact likelihood and fast sampling; architecturally constrained

Family	Mechanism	Trade-off
Energy-based models	Learn an unnormalised energy	Very flexible; sampling requires MCMC
Consistency models	Learn a direct map from any noise level to data	One-to-few step sampling; distilled from diffusion

25.6 Evaluating generative models

- **FID** compares the mean and covariance of Inception features between real and generated sets - lower is better, but it is sensitive to sample count and preprocessing, and it rewards matching the training distribution rather than quality per se.
- **Precision/recall for generative models** separates fidelity from coverage - useful when FID hides mode dropping.
- **CLIP score** measures prompt adherence for text-to-image.
- **Human evaluation** remains the ground truth; report it with proper sample sizes and blinding.
- For likelihood-based models, report bits-per-dimension; never compare it against a GAN, which has none.

Exercises

1. Train an autoencoder and a VAE on MNIST; interpolate between two latent codes in each and compare the decoded paths.
2. Implement DDPM training on 32x32 images in under 150 lines and sample with both DDPM and DDIM; compare step counts for equal quality.
3. Train a small DCGAN and deliberately induce mode collapse by making the discriminator too strong; then fix it.

26. Graph Neural Networks

Molecules, road networks, social graphs, program dependency graphs and meshes are not grids or sequences. A GNN generalises convolution to arbitrary graphs by passing messages along edges.

26.1 Message passing

```
m_v^(k)  = AGGREGATE( { M(h_u^(k-1), h_v^(k-1), e_uv) : u in N(v) } )

h_v^(k)  = UPDATE( h_v^(k-1), m_v^(k) )

AGGREGATE must be permutation-invariant: sum, mean, max, or attention.
```

Model	Aggregation	Character
GCN	Normalised mean: D^-1/2 A D^-1/2 H W	Simple, strong baseline, spectral motivation
GraphSAGE	Mean/LSTM/max over a sampled neighbourhood	Scales to large graphs by sampling; inductive
GAT	Attention-weighted neighbours	Learns which neighbours matter; multi-head
GIN	Sum with an MLP	Provably as expressive as the Weisfeiler-Lehman test - the most expressive of the simple message-passing schemes
MPNN / SchNet / DimeNet	Physics-aware messages with distances and angles	Molecular property prediction, force fields
Graph Transformer	Full attention plus structural encodings	Avoids over-squashing; needs positional encodings for graphs

26.2 Tasks and readouts

- **Node classification:** predict a label per node (fraud in a transaction graph, role in a social network). Often transductive.
- **Link prediction:** score whether an edge exists - recommendation, knowledge-graph completion.
- **Graph classification/regression:** pool node embeddings into one vector (sum, mean, max, or a learned pooling) and predict - molecular property prediction is the canonical case.
- **Generation:** produce new graphs, e.g. candidate molecules, often with diffusion over adjacency and node features.

26.3 The characteristic problems

Problem	Description	Mitigation
Over-smoothing	After many layers all node embeddings converge to the same vector	2-4 layers; residual/jumping-knowledge connections; PairNorm; initial-residual (GCNII)
Over-squashing	Information from an exponentially growing neighbourhood is compressed into a fixed vector	Graph rewiring, virtual global nodes, graph Transformers
Scalability	Neighbourhood explosion when sampling k hops	Neighbour sampling (GraphSAGE), cluster-based batching (Cluster-GCN), historical embeddings
Expressiveness limit	Message passing cannot distinguish some non-isomorphic graphs (1-WL bound)	Add structural or positional features, subgraph counts, or higher-order schemes
Heterophily	Connected nodes often have DIFFERENT labels, breaking the smoothing assumption	Separate self and neighbour transformations; signed or higher-order aggregation

BEFORE REACHING FOR A GNN

Test a strong tabular baseline with hand-made graph features - degree, neighbour label counts, PageRank, triangle counts, embeddings from node2vec - fed to gradient boosting. On many industrial graph problems this matches or beats a GNN at a fraction of the engineering cost. Use a GNN when the relational structure is genuinely the signal and the features alone are weak.

Exercises

1. Implement a 2-layer GCN in raw PyTorch using a sparse adjacency matrix and train it on Cora.
2. Demonstrate over-smoothing: plot the average pairwise cosine similarity of node embeddings against the number of layers, 1 to 12.
3. Compare a GNN against gradient boosting on hand-crafted graph features for the same node-classification task.

27. Self-Supervised and Transfer Learning

Labels are expensive; raw data is not. Self-supervised learning invents a supervised task out of unlabelled data, learns a representation from it, and transfers that representation to the task you actually care about with a fraction of the labels.

27.1 Pretext tasks by modality

Modality	Objective	Representative methods
Text	Predict the next token; predict masked tokens; corrupt and reconstruct spans	GPT, BERT, T5, ELECTRA
Images - contrastive	Two augmented views of the same image should match; different images should not	SimCLR, MoCo, CLIP (image-text)
Images - non-contrastive	Match views without negatives, avoiding collapse by architecture	BYOL, SimSiam, DINO, DINOv2
Images - generative	Mask 75% of patches and reconstruct	MAE, BEiT, SimMIM
Audio	Contrastive prediction of masked latent speech units	wav2vec 2.0, HuBERT, BEATs
Video	Predict future frames, ordering, or masked spatio-temporal patches	VideoMAE, V-JEPA
Time series / sensors	Masked reconstruction, contrastive over augmented windows	TS2Vec, TF-C, PatchTST pretraining
Graphs	Mask node or edge attributes; contrast subgraphs	GraphMAE, GRACE

27.2 Contrastive learning in detail

```
InfoNCE:  L = -log [ exp(sim(z_i, z_j)/tau) /
                     SUM_k exp(sim(z_i, z_k)/tau) ]
sim = cosine similarity, tau = temperature (0.05-0.2)
```

- **Augmentation is the whole design.** The representation becomes invariant to exactly the transformations you apply. For SimCLR, random crop plus colour jitter is essential - without colour jitter the network solves the task by matching colour histograms.
- **Negatives matter:** large batches (4k+) or a momentum-updated queue (MoCo) provide enough negatives; otherwise the loss is too easy.
- **Collapse** - all embeddings identical - is the failure mode. Contrastive methods avoid it with negatives; BYOL/SimSiam with a predictor head plus a stop-gradient; DINO with centring and sharpening; Barlow Twins and VICReg with explicit decorrelation terms.
- **Projection head:** contrast in the space AFTER a small MLP, but transfer the representation from BEFORE it - the head discards information useful downstream.

27.3 Transfer learning strategies

Scenario	Strategy
Target task similar, few labels	Freeze the backbone, train a linear probe. Fast, and a strong evaluation of representation quality
Target similar, moderate labels	Fine-tune the last blocks with discriminative learning rates
Target different, many labels	Fine-tune everything, low LR, longer warmup; consider re-initialising the last block

Scenario	Strategy
Very few labels (<100/class)	Few-shot with a frozen foundation model plus k-NN or a prototype classifier; or LoRA on a large pretrained model
Domain shift only	Continue self-supervised pretraining on unlabelled target-domain data first (domain-adaptive pretraining), then fine-tune

CATASTROPHIC FORGETTING

Fine-tuning on a narrow task overwrites general capability. Mitigations: lower learning rates, freezing lower layers, replaying a sample of the original data, elastic weight consolidation (penalise movement of parameters important to the old task), or keeping the base model frozen and training adapters/LoRA so the original weights are never touched at all - which is also why adapter-based deployment is operationally attractive.

27.4 Semi-supervised learning

- **Pseudo-labelling:** predict on unlabelled data, keep confident predictions as labels, retrain. Simple and effective; guard against confirmation bias with a high threshold and strong augmentation.
- **Consistency regularisation:** the prediction should not change under augmentation - the core of FixMatch, which combines a weak-augmentation pseudo-label with a strong-augmentation consistency loss and remains a very strong baseline.
- **Mean teacher:** an EMA of the student's weights produces the targets, which stabilises training.
- **Noisy student:** iteratively train a larger student on pseudo-labels with noise (dropout, augmentation), then make it the teacher.

Exercises

1. Train SimCLR on an unlabelled subset of CIFAR-10, then compare a linear probe on 1%, 10% and 100% of the labels against training from scratch.
2. Ablate the augmentations in a contrastive setup one at a time and report linear-probe accuracy - this reproduces the key finding of the SimCLR paper.
3. Take a pretrained sensor or audio encoder and evaluate a k-NN classifier on its frozen embeddings with 5 labelled examples per class.

PART V

Expert Topics

Making models small and fast, training them on the device itself, learning from interaction, trusting the output, and shipping the whole thing to production.

28. Efficient Deep Learning I: Quantization

Quantization stores and computes with fewer bits. It is the single highest-leverage efficiency technique available: 8-bit integers cut memory by 4x against float32 and, on hardware with integer units, raise throughput by 2-4x, usually for well under one point of accuracy.

28.1 Why it works and why it pays

- Trained networks are heavily over-parameterised and their weights cluster in a narrow range; the mapping from weights to function is much less precise than float32 suggests.
- Inference for large models is **memory-bandwidth bound**. Fewer bits per weight means proportionally fewer bytes moved, so latency falls close to linearly with bit width even when the arithmetic is unchanged.
- Integer arithmetic units are smaller and far more energy-efficient than floating-point ones: an int8 multiply-accumulate costs roughly an order of magnitude less energy than an fp32 one, which is decisive on battery power.
- Microcontrollers frequently have no floating-point unit at all - int8 is not an optimisation there, it is the only option.

Format	Bits	Memory for 7B params	Typical use
FP32	32	28 GB	Training master weights, reference accuracy
TF32 / FP16 / BF16	19 / 16 / 16	14 GB	Training compute; BF16 has FP32's exponent range
FP8 (E4M3 / E5M2)	8	7 GB	Training and inference on recent accelerators
INT8	8	7 GB	The standard deployment format; excellent hardware support
INT4 / NF4	4	3.5 GB	LLM weight-only quantization; NF4 is information-theoretically matched to a normal distribution
INT2 / ternary / binary	2 / 1.58 / 1	<= 1.75 GB	Research and extreme edge; needs quantization-aware training from scratch

28.2 The mathematics of affine quantization

```
Quantize:    q = clamp( round( r / s ) + z ,  q_min , q_max )

Dequantize:  r_hat = s * ( q - z )

s = (r_max - r_min) / (q_max - q_min)          scale (a float)

z = q_min - round( r_min / s )                 zero-point (an integer)
```

Symmetric quantization forces z = 0 and uses a range centred on zero: `s = max|r| / (2^(b-1) - 1)`. It makes the arithmetic cheaper (no cross-terms with the zero-point) and is the right choice for weights, which are roughly zero-centred. **Asymmetric** quantization keeps z and uses the full range; it is the right choice for activations after ReLU, which are non-negative and would waste half the codes under a symmetric scheme.

WHERE THE ZERO-POINT COMES FROM, AND WHY IT MUST BE EXACT

Real zero must map to an exact integer, because padding, masking and ReLU all produce exact zeros and any error there shows up as a systematic bias across the whole tensor. Solving r = 0 in r = s(q - z) gives q = z, so the zero-point is precisely the integer code that represents 0.0. This is why the rounding in the definition of z is not optional.

```
Integer matmul with symmetric weights and asymmetric activations:

                    r_y = SUM_i r_w[i] r_x[i]

                = s_w s_x * SUM_i q_w[i] ( q_x[i] - z_x )

            = s_w s_x * ( SUM_i q_w[i] q_x[i]  -  z_x SUM_i q_w[i] )

                        \___ int32 accumulation ___/   \_ precomputed _/
```

The whole inner loop is integer multiply-accumulate into an int32 accumulator; the float scale is applied once at the end, usually as a fixed-point multiply-and-shift so that no floating-point unit is needed anywhere.

28.3 Granularity and what it costs

Granularity	One scale per	Accuracy	Overhead
Per-tensor	Whole weight tensor	Lowest	Negligible; fastest
Per-channel (per-output)	Output channel / row	Much better - the standard for weights	One scale per channel
Per-group (e.g. 128)	Block of weights within a row	Best for INT4 LLM weights	Scales stored per group; still cheap
Per-token (activations)	Row of the activation matrix	Handles varying dynamic range across tokens	Computed on the fly

PER-CHANNEL WEIGHT QUANTIZATION IS NEARLY FREE ACCURACY

Different output channels of a convolution or linear layer often have weight ranges that differ by 10-100x. A single per-tensor scale is then set by the widest channel and crushes all the others into a handful of codes. Per-channel scales fix this at the cost of one float per channel, and they are supported by essentially all inference runtimes. Use them by default.

28.4 Post-training quantization (PTQ)

PTQ quantizes a trained model without retraining. It needs only a small **calibration set** - typically 100-1,000 unlabelled samples - to estimate activation ranges.

- **Dynamic quantization:** weights are quantized offline, activation ranges computed at runtime per batch. Trivial to apply, no calibration data, good for LSTMs and Transformer linear layers where the memory of the weights dominates.
- **Static quantization:** activation ranges are calibrated offline and baked in, so the entire graph runs in integer arithmetic. Faster; needs representative calibration data.
- **Calibration methods:** min-max (simple, outlier-sensitive), percentile (clip at 99.9%), entropy/KL (TensorRT's default, minimises information loss), and MSE-optimal search over clipping thresholds.
- **Cross-layer equalisation:** exploit the positive homogeneity of ReLU to rescale consecutive layers so their per-channel ranges match, improving per-tensor quantization for free.
- **Bias correction:** quantization introduces a systematic shift in the mean activation; measure it on the calibration set and fold the correction into the bias.
- **AdaRound:** learn, per weight, whether to round up or down by minimising the layer output error rather than the weight error. Reliably recovers most of the INT4 gap without labels.
- **GPTQ / AWQ / SmoothQuant** for LLMs: GPTQ solves a layerwise second-order reconstruction problem column by column; AWQ scales salient channels identified by activation magnitude before quantizing; SmoothQuant migrates activation outliers into the weights so both become quantizable.

28.5 Quantization-aware training (QAT)

QAT simulates quantization during training so the network learns weights that are robust to it. Fake-quantize nodes round and clamp in the forward pass while the backward pass uses the straight-through estimator.

```
Forward:   x_q = s * ( clamp( round(x/s) + z, q_min, q_max ) - z )

Backward:  dL/dx = dL/dx_q * 1[ q_min <= round(x/s)+z <= q_max ]

           (identity inside the range, zero outside - the STE)
```

```
import torch, torch.nn as nn
from torch.ao.quantization import QConfig, prepare_qat, convert
from torch.ao.quantization.observer import (MovingAverageMinMaxObserver,
                                            MovingAveragePerChannelMinMaxObserver)

qconfig = QConfig(
    activation=MovingAverageMinMaxObserver.with_args(
        dtype=torch.quint8, qscheme=torch.per_tensor_affine),
    weight=MovingAveragePerChannelMinMaxObserver.with_args(
        dtype=torch.qint8, qscheme=torch.per_channel_symmetric))

model.train()
model.qconfig = qconfig
model_prepared = prepare_qat(model.eval(), inplace=False).train()

# Fine-tune for a few epochs at ~1/10 of the original learning rate.
# Freeze the observers near the end so the ranges stop moving:
#   model_prepared.apply(torch.ao.quantization.disable_observer)
#   model_prepared.apply(nn.intrinsic.qat.freeze_bn_stats)

model_int8 = convert(model_prepared.eval(), inplace=False)
```

Listing 28.1 - Quantization-aware training in PyTorch. QAT typically recovers most of the INT8 gap and is often the only way to reach INT4.

	PTQ	QAT
Data needed	100-1,000 unlabelled samples	The training set and a training loop
Time	Minutes	Hours to days (a fraction of full training)
INT8 accuracy loss	Typically <1% on CNNs; larger on compact models	Usually within noise of the float model
INT4 accuracy	Needs GPTQ/AWQ-class methods; noticeable loss	The reliable route
When to use	Always try first	When PTQ is not good enough, or below 8 bits

28.6 Quantizing eight real weights, by hand

Take these eight trained weights and quantize them to symmetric INT8 per-tensor. Every number below is computed, not illustrative.

```
w = [-0.82, -0.31, 0.05, 0.11, 0.47, 1.23, -1.05, 0.63]

absmax = 1.23

scale s = absmax / 127 = 1.23 / 127 = 0.009685

q_i = round(w_i / s), clipped to [-127, 127]
```

Original w	w / s	Quantized q	Dequantized q*s	Error
-0.82	-84.67	-85	-0.8232	-0.0032

Original w	w / s	Quantized q	Dequantized q*s	Error
-0.31	-32.01	-32	-0.3099	+0.0001
0.05	5.16	5	0.0484	-0.0016
0.11	11.36	11	0.1065	-0.0035
0.47	48.53	49	0.4746	+0.0046
1.23	127.00	127	1.2300	0.0000
-1.05	-108.42	-108	-1.0460	+0.0040
0.63	65.05	65	0.6295	-0.0005

```
Max absolute error = 0.0046      RMSE = 0.0028

Storage: 8 floats (32 B) -> 8 int8 (8 B) + 1 float scale (4 B) = 12 B
```

Two observations that generalise. The largest-magnitude weight is represented exactly, because it defines the scale - so the widest weight in a tensor gets perfect treatment while everything else is rounded, which is precisely why a single outlier ruins per-tensor quantization for the other 4,095 weights in its row. And the error is bounded by half a step, s/2 = 0.0048, uniformly across the range: quantization error is roughly uniform noise, not proportional error.

28.7 What happens as the bits come off

Bit width	Levels	Step size s	RMSE on the weights above	Rule of thumb
8-bit	255	0.0097	0.0028	Essentially free; PTQ suffices
4-bit	15	0.176	0.0507	18x worse; needs per-group scales or GPTQ/AWQ
2-bit	3	1.23	0.334	120x worse; needs QAT and usually still hurts

Error grows roughly as 2^(-b), so each bit removed doubles the noise. That is the whole trade-off curve, and it explains why INT8 is nearly free while INT4 needs help and INT2 needs a rethink of the training itself.

28.8 Asymmetric quantization for activations, worked

Post-ReLU activations are non-negative, so a symmetric scheme wastes half its codes on values that never occur. With a calibrated range of [0, 4.0] into uint8:

```
s = (4.0 - 0.0) / 255 = 0.015686        z = 0  (since r_min = 0)

a = 0.0  -> q = 0    -> 0.0000
a = 0.4  -> q = 26   -> 0.4078
a = 1.2  -> q = 76   -> 1.1922
a = 2.8  -> q = 178  -> 2.7922
a = 3.9  -> q = 249  -> 3.9059
```

WHY EXACT ZERO MUST MAP TO AN EXACT CODE

Here z = 0, so the real value 0.0 maps to the integer 0 with no error at all. That is essential: padding, masking and ReLU all produce exact zeros in bulk, and if 0.0 quantized to, say, 0.008 instead, every padded position in every sequence would contribute a small systematic bias that accumulates across layers. The zero-point exists to make this impossible.

28.9 The whole INT8 layer, arithmetic included

```
                Real:      y = SUM_i w_i x_i

        Quantized: w_i = s_w q_w_i,   x_i = s_x (q_x_i - z_x)

        y = s_w s_x [ SUM_i q_w_i q_x_i  -  z_x SUM_i q_w_i ]
                  \___ int8 x int8 -> int32 ___/  \__ constant __/

    The bracketed sum is pure integer arithmetic. The second term depends
     only on the weights, so it is precomputed once and folded into the
   bias. The float scales multiply the int32 accumulator once at the end -
    and even that is usually done as a fixed-point multiply-and-shift, so
     a device with no floating-point unit can run the layer end to end.
```

This is the reason INT8 inference is fast rather than merely small: the inner loop is integer multiply-accumulate, which is what DSPs, NPUs and microcontroller SIMD units are built for, and which costs roughly an order of magnitude less energy per operation than the float equivalent.

28.10 What breaks, and how to fix it

Failure	Cause	Remedy
Large drop on a depthwise-separable model	Depthwise layers have very different per-channel ranges	Per-channel weights; QAT; cross-layer equalisation
LLM collapses below 8 bits	A few activation channels have outliers 100x larger than the rest	SmoothQuant, AWQ, keeping outlier channels in fp16 (LLM.int8), group-wise scales
Accuracy fine offline, poor on device	The runtime fuses or reorders ops differently, or uses different rounding	Evaluate with the actual runtime, not the simulation
BatchNorm statistics wrong after quantization	BN folded into the conv with float statistics that no longer match	Fold BN before calibration; freeze BN statistics during QAT
Softmax/LayerNorm degrade	Narrow dynamic range in normalisation	Keep sensitive layers in higher precision - mixed precision by sensitivity

SENSITIVITY ANALYSIS: THE WORKFLOW THAT ACTUALLY FINDS THE RIGHT CONFIGURATION

Quantize one layer at a time, keeping everything else in float, and record the accuracy drop. This produces a sensitivity ranking, usually showing that the first layer, the last layer and a handful of normalisation-adjacent layers account for most of the damage. Keep those in 8 or 16 bits and push the rest to 4. Automating this - solving for a bit-width assignment under a size or latency budget - is the core of mixed-precision quantization methods such as HAWQ, which use Hessian traces as the sensitivity proxy.

28.11 Reporting quantization results honestly

A complete quantization report

- Baseline float accuracy on the same evaluation code and data.
- Bit widths per tensor type (weights, activations, accumulator), and the granularity used for each.
- Which layers, if any, were kept in higher precision.
- Calibration set size and selection method.
- Measured model size on disk and peak RAM at inference, not just the theoretical figure.

- Latency and energy measured ON THE TARGET DEVICE with the target runtime, over many runs, reporting median and tail.
- Accuracy per class or per slice - quantization damage is often concentrated in rare classes.

Exercises

1. Implement affine quantize/dequantize in NumPy and measure the reconstruction error of a trained weight tensor at 8, 6, 4 and 2 bits, per-tensor versus per-channel.
2. Apply dynamic and static PTQ to the same CNN and compare accuracy, size and latency; then run QAT and compare again.
3. Take a small Transformer, plot the per-channel maximum activation magnitude for each layer, and identify the outlier channels that make naive INT8 fail.

29. Efficient Deep Learning II: Pruning and Sparsity

Pruning removes parameters, channels or whole blocks from a network. Trained networks are massively redundant - 80-95% of the weights in a typical over-parameterised model can be removed with careful procedure and little or no accuracy loss. The difficulty is not deciding what to remove; it is turning removal into actual speed.

29.1 Structured versus unstructured

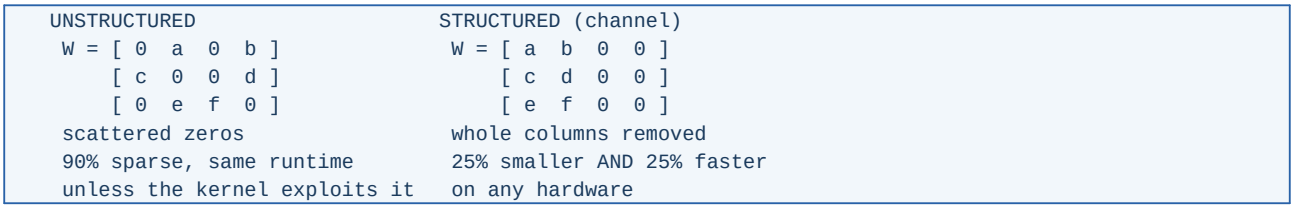

Figure 29.1 - Sparsity that a matrix multiply can exploit versus sparsity that it cannot.

Type	Granularity	Compression	Real speedup
Unstructured	Individual weights	Very high (90-99%)	Only with sparse kernels or a sparse accelerator; often none on a dense GPU
Semi-structured (2:4)	2 non-zeros in every group of 4	2x on weights	Yes - up to about 1.5-2x on NVIDIA Ampere and later sparse tensor cores
Channel / filter	Whole output channels	Moderate (30-70%)	Yes, on all hardware - the network is simply smaller
Block / head	Attention heads, FFN blocks, layers	Coarse	Yes; the cleanest option for Transformers
Layer dropping	Entire residual blocks	Coarse	Yes; large latency wins, larger accuracy risk

CHOOSE THE GRANULARITY YOUR HARDWARE CAN CASH IN

The most common mistake in pruning is reporting '95% sparsity' with no latency change. If your deployment target is a dense CPU or GPU kernel, prune channels or heads. If it is an Ampere-class GPU, use 2:4 semi-structured sparsity. Use unstructured pruning only when your runtime has genuine sparse kernels, or when you care about model SIZE (compressed storage, over-the-air update) rather than speed.

29.2 What to prune: saliency criteria

Criterion	Score	Comment
Magnitude	|w|	The strongest simple baseline; still competitive with everything else
L1/L2 norm of a filter	||W_c||	The channel analogue of magnitude
Gradient-based (SNIP)	|w * dL/dw|	Estimates the loss change from removal; works at initialisation
Taylor / first-order	|w * g| summed over a batch	Standard for structured channel pruning
Second-order (OBD/OBS, SparseGPT)	H-weighted importance	Theoretically better; SparseGPT makes it tractable for LLMs
Activation-based (APoZ)	Fraction of zero activations	Cheap channel criterion for ReLU networks
Wanda (LLMs)	|w| * ||activation||	Extremely cheap, no retraining, strong for one-shot LLM pruning
Learned gates	L0 / Gumbel gates trained jointly	Optimises structure and weights together; more complex

29.3 When to prune: the four schedules

- **One-shot after training:** train, prune, fine-tune. Simple and often sufficient up to moderate sparsity.
- **Iterative magnitude pruning:** repeat (prune a little, fine-tune) many times. Consistently the best accuracy at high sparsity, and the most expensive.
- **Gradual sparsity during training** (Zhu-Gupta): raise the sparsity from 0 to the target with a cubic schedule between step t0 and tn. Trains once, reaches high sparsity, and is the standard production recipe.
- **Sparse from the start / dynamic sparse training:** RigL, SET and similar methods keep a fixed sparsity budget throughout, periodically dropping the smallest weights and regrowing new connections where gradients are largest. They never require a dense model in memory, which is the property that makes them relevant to on-device training (Chapter 31).

Zhu-Gupta cubic schedule:

s_t = s_f + (s_0 - s_f) * (1 - (t - t_0)/(n dt))^3

RigL step: drop the |w|-smallest fraction f of active weights,

grow the same number where |dL/dw| is largest,

f decayed cosine-wise over training.

WHY REGROWTH BY GRADIENT MAGNITUDE IS THE RIGHT RULE

A weight that is currently zero has no effect on the loss, but its GRADIENT still says how much the loss would change if it became non-zero. RigL uses exactly that signal to decide where to spend its sparsity budget next, which is why it beats static random or magnitude-only sparsity at the same parameter count. The cost is one dense gradient computation at each update step - cheap if done every few hundred steps, and the reason the update interval is a key hyperparameter.

29.4 The lottery ticket hypothesis

A randomly initialised dense network contains a sparse subnetwork - a 'winning ticket' - that, when trained **from the original initialisation**, matches the full network's accuracy. The practical recipe that demonstrates it is iterative magnitude pruning with weight rewinding (reset the surviving weights to their values at initialisation, or at an early step, rather than keeping the trained values).

- It reframes pruning as **finding a good architecture and initialisation**, not merely as removing fat.
- For large models, rewinding to an early step (a few percent into training) works where rewinding to step 0 does not.
- It does not yet give a cheap way to find the ticket without training the dense model first, which limits its direct practical use - but it motivates sparse-training methods that try to.

29.5 Pruning in practice

```
import torch, torch.nn.utils.prune as prune

# --- global unstructured magnitude pruning across all conv/linear layers
params = [(m, 'weight') for m in model.modules()
          if isinstance(m, (torch.nn.Conv2d, torch.nn.Linear))]
prune.global_unstructured(params, pruning_method=prune.L1Unstructured,
                          amount=0.8)          # 80% of weights zeroed

# --- structured: remove whole output channels by L2 norm
for m in model.modules():
    if isinstance(m, torch.nn.Conv2d):
        prune.ln_structured(m, name='weight', amount=0.3, n=2, dim=0)
```

```
# --- fine-tune here with a small learning rate (masks stay applied) ---

for m, n in params:
    prune.remove(m, n)      # bake the mask into the weights, drop the mask

# NOTE: prune.remove leaves a DENSE tensor containing zeros. To gain
# speed you must either export to a sparse format your runtime supports,
# or physically rebuild the network with smaller layers (structured).
```

Listing 29.1 - Pruning in PyTorch, including the caveat that most tutorials omit.

Pruning workflow that produces a deployable model

- Establish the dense baseline accuracy and latency on the target device.
- Choose the granularity that your runtime can exploit.
- Decide global versus per-layer sparsity. Global allocates the budget automatically but can wipe out a small sensitive layer - always exclude the first and last layers.
- Use a gradual schedule with fine-tuning rather than one-shot, if you can afford the training time.
- After pruning, physically rebuild or export the compact model, then re-measure latency and memory on the device.
- Re-check per-class and per-slice accuracy: pruning damage concentrates in rare classes.
- Consider combining with quantization - prune first, then quantize, then fine-tune once more.

29.6 Pruning a small layer, weight by weight

Take a 4x4 weight matrix and apply 50% magnitude pruning three ways. The difference between the three is the entire practical content of this chapter.

```
W = [  0.90  -0.05   0.60   0.02 ]      row norms (L2):
    [  0.03   0.01  -0.02   0.04 ]         row 1: 1.08
    [ -0.70   0.80   0.10  -0.50 ]         row 2: 0.06
    [  0.20  -0.15   0.05   0.30 ]         row 3: 1.19
                                           row 4: 0.39
```

Scheme	What is removed	Result	Speedup on a dense kernel
Unstructured, 50%	The 8 smallest |w| anywhere	Scattered zeros; W keeps its 4x4 shape	None - the kernel still multiplies 16 numbers
2:4 semi-structured	The 2 smallest of every 4 consecutive weights	Exactly 2 non-zeros per group of 4	Up to ~2x on sparse tensor cores
Structured (rows), 50%	The 2 rows with the smallest norm - rows 2 and 4	A genuine 2x4 matrix	2x everywhere, on any hardware

Look at what structured pruning chose. Row 2 has every weight near zero: removing it costs almost nothing, and it removes a whole output channel - the next layer loses an input, the tensor genuinely shrinks, and every runtime on earth is faster as a result. Row 4 is a real sacrifice: its norm is 0.39, so some signal is lost. That is the structured/unstructured trade in miniature - unstructured pruning removes exactly the least useful weights but changes nothing about the computation, while structured pruning removes some useful weights and actually makes the model smaller.

THE REPORTED-SPARSITY TRAP

A paper or a blog post that says '95% of weights removed, 0.3% accuracy lost' and shows no latency measurement has almost certainly measured nothing that a deployment would feel. Ask three questions: which granularity, which runtime, and what was the measured milliseconds-per-inference before and after on the target device. If the answer to the third is missing, the compression is theoretical.

29.7 How far can you actually prune?

Sparsity	Typical accuracy effect (with fine-tuning)	Notes
0-50%	None measurable on an over-parameterised model	Essentially free; the network was carrying redundancy
50-80%	0-1 point	Needs gradual pruning plus fine-tuning
80-95%	1-4 points	Needs iterative pruning; small layers must be protected
>95%	Large and erratic	Only for heavily over-parameterised models; consider a smaller architecture instead

PRUNE THE RIGHT LAYERS

Never prune the first and last layers at the same rate as the middle. The first layer has few parameters but sees the raw input, and the last maps to the classes - damage there is disproportionate, while the savings are trivial because both are small. Global magnitude pruning with no exclusions will happily gut them, which is the most common reason a pruning run collapses.

29.8 Compression stacked, with the numbers

Stage	Size	Latency	Accuracy	What changed
Dense FP32 baseline	14.0 MB	100 ms	94.2%	-
+ 50% channel pruning	7.2 MB	58 ms	93.6%	Genuinely smaller tensors
+ INT8 quantization	1.9 MB	24 ms	92.7%	4x memory, integer kernels
+ distillation from the dense teacher	1.9 MB	24 ms	93.9%	Same model, better weights

Read the last row carefully: distillation changed nothing about the model's size or speed, and recovered most of the accuracy lost to the first two stages. That is the standard modern recipe - compress aggressively, then use the uncompressed model as a teacher to repair the damage. It is almost always better than compressing less.

29.9 Sparsity in large language models

- **SparseGPT** and **Wanda** prune LLMs in one shot to 50-60% unstructured or 2:4 sparsity using a small calibration set and no retraining.
- **Structured LLM pruning** (LLM-Pruner, Sheared-Llama) removes attention heads, FFN channels or layers and then continues pretraining briefly to recover - this is what produces genuinely smaller, faster models.
- **Mixture-of-experts** is conditional sparsity by design: only k of N expert FFNs run per token, so parameter count and compute decouple. It is the dominant way large models buy capacity without buying latency, at the cost of memory and routing complexity.
- **Activation sparsity:** ReLU-family models have naturally sparse activations that can be exploited to skip computation at inference (deja-vu-style predictors), and it is a reason some recent models deliberately reintroduce ReLU.

29.10 Combining compression techniques

Order	Rationale
Distill -> prune -> quantize -> fine-tune	Distillation defines a smaller architecture first; pruning trims what remains; quantization is applied last because it is the least forgiving; a final short fine-tune (or QAT) recovers the residual loss
Prune and quantize jointly	Better in principle - the two interact - but harder to tune and to debug
Quantize before pruning	Generally worse: pruning decisions made on quantized weights are noisier

```
Compound compression example (typical, MobileNet-class model):

     dense fp32        14.0 MB   100.0%   baseline accuracy

          + 50% channels     7.2 MB    51.4%   -0.6 pt

          + INT8             1.9 MB    13.6%   -0.9 pt

   + distillation     1.9 MB    13.6%   -0.3 pt  (recovered by teacher)
```

Exercises

1. Prune a trained CNN to 50%, 80%, 90%, 95% and 99% with global magnitude pruning and plot accuracy against sparsity, with and without fine-tuning.
2. Implement RigL's drop-and-grow step and compare against static sparse training at the same parameter budget.
3. Structurally prune 30% of channels, rebuild the network with the smaller layers, and measure the actual latency change on your target device. Compare with the latency change from unstructured pruning at the same parameter count.

30. Efficient Deep Learning III: Distillation, NAS and Efficient Design

30.1 Knowledge distillation

A small **student** is trained to imitate a large **teacher**. The teacher's full output distribution carries much more information than a one-hot label - the relative probabilities of the wrong classes encode which categories resemble each other, the 'dark knowledge'.

```
L = (1 - a) * CE( y_true, student )
+ a * T^2 * KL( softmax(z_T / T) || softmax(z_S / T) )

T = temperature (2-10);  the T^2 factor keeps gradient magnitudes
comparable between the two terms.
```

Variant	What is matched	Notes
Response / logit KD	Output distribution	The classic; simple and strong
Feature / hint KD	Intermediate activations (FitNets)	Needs a projection when widths differ
Attention transfer	Attention maps	Effective for CNNs and Transformers
Relational KD	Pairwise distances between samples in feature space	Transfers structure rather than points
Self-distillation	The model teaches itself (deeper layers teach shallower, or an EMA teacher)	Improves accuracy with no larger teacher
Sequence-level KD	Teacher-generated outputs used as training data	The standard way to make small LLMs; effectively synthetic data
Born-again networks	Student identical in size to the teacher	Often beats the teacher - evidence that the soft targets themselves are the benefit

DISTILLATION IS THE MOST RELIABLE COMPRESSION METHOD

Unlike pruning and quantization, distillation lets you choose the student architecture freely - so you can pick one that is fast on your actual hardware rather than one that is merely smaller. It also composes with everything else: distil into a pruned, quantized student and use the teacher's soft targets during quantization-aware fine-tuning.

30.2 Efficient architecture design

- **Depthwise separable convolutions** (MobileNet) - 8-9x fewer FLOPs per 3x3 layer.
- **Inverted residuals with linear bottlenecks** (MobileNetV2) - expand, depthwise, project, with the skip on the narrow tensors to keep memory traffic low.
- **Squeeze-and-excitation** - cheap channel attention, consistently worth its small cost.
- **Group convolution and channel shuffle** (ShuffleNet) - reduce cost while preserving cross-channel mixing.
- **Compound scaling** (EfficientNet) - scale depth, width and resolution together rather than one at a time.
- **Hardware-aware design:** FLOPs are a poor proxy for latency. Memory access, kernel support, and degree of parallelism often dominate; a model with fewer FLOPs can easily be slower. Measure on the device.

30.3 Neural architecture search

Approach	Mechanism	Cost
Reinforcement learning (NASNet)	A controller proposes architectures, reward = validation accuracy	Thousands of GPU-days; historical
Evolutionary	Mutate and select architectures	High but parallelisable
Differentiable (DARTS)	Relax the discrete choice into a weighted mixture and optimise with gradients	A few GPU-days; can collapse to trivial operations
One-shot / supernet (Once-for-All)	Train one weight-sharing supernet, then extract sub-networks per device	Train once, deploy many - the practical modern approach
Zero-cost proxies	Score architectures at initialisation from gradient or Jacobian statistics	Minutes; noisy but useful for filtering

NAS IS RARELY THE RIGHT FIRST MOVE

A well-tuned existing architecture with good training recipes beats a poorly executed NAS almost every time, and NAS results frequently fail to transfer across datasets and hardware. Reach for it when you have a hard, fixed hardware constraint, a stable task, and enough compute to do it properly - typically hardware-aware supernet search against a measured latency table.

30.4 Choosing a compression strategy

Constraint	First move	Then
Model too large for flash/RAM	INT8 quantization (4x)	Structured pruning, then INT4 with QAT
Latency too high	Structured pruning + a hardware-friendly architecture	Quantization; operator fusion; a better runtime
Energy budget	Quantization (integer arithmetic)	Reduce input resolution/sampling rate; early-exit networks
Accuracy must not drop	Distillation into a compact student	QAT rather than PTQ; keep sensitive layers at higher precision
Many target devices	Once-for-All supernet	Extract a sub-network per device from measured latency

Exercises

1. Distil a ResNet-50 teacher into a ResNet-18 student and compare against training the student from scratch, sweeping T and alpha.
2. Measure FLOPs and on-device latency for MobileNetV2, EfficientNet-B0 and ResNet-18 and show that the FLOPs ranking does not match the latency ranking.
3. Build a small supernet with three width options per layer and extract two sub-networks meeting different latency budgets.

31. On-Device and Federated Training

Inference on the edge is now routine. **Training** on the edge is the harder and more interesting problem: it lets a model personalise to its user, adapt to sensor drift, and improve without any data ever leaving the device.

31.1 Why train on the device at all

- **Privacy:** raw sensor data, keystrokes, audio and health signals never leave the hardware. This is often a legal requirement, not a preference.
- **Personalisation:** a gesture, gait or keyboard model tuned to one person beats a global model by a wide margin.
- **Adaptation:** sensors age, mounting positions change, environments shift. A model that can fine-tune locally survives drift that would otherwise require a recall or an update campaign.
- **Connectivity and cost:** no round trip, no bandwidth bill, no cloud inference cost, and it works offline.

31.2 The resource wall

```
Inference memory  ~  weights + one layer's activations

Training memory   ~  weights + gradients + optimiser state
                     + ALL activations kept for the backward pass

Adam, fp32:  weights 4B + grad 4B + m 4B + v 4B = 16 B / parameter

A 1M-parameter model therefore needs ~16 MB before activations -
more than the total SRAM of most microcontrollers.
```

Class of device	RAM	Compute	What is feasible
Cloud GPU	40-80 GB	100+ TFLOPs	Anything
Phone / SoC NPU	4-16 GB	1-10 TOPS	Full fine-tuning of small models; LoRA on medium ones
Embedded Linux (Pi-class)	0.5-8 GB	10-100 GFLOPs	Fine-tuning small CNNs; sparse or partial updates
MCU (Cortex-M)	256 KB - 1 MB SRAM	~100 MFLOPs	Last-layer or sparse-subset updates only; int8 arithmetic

31.3 Techniques that make on-device training possible

Technique	What it saves	Cost / caveat
Freeze most layers, train the head	Activations and gradients for frozen layers	Limited adaptation capacity
Bias-only / BitFit updates	Gradients and optimiser state (biases are <1% of parameters)	Surprisingly effective for domain shift
LoRA / adapters	Optimiser state and gradient memory	Still needs backward through the frozen layers
Sparse updates (a subset of channels/layers)	Both memory and compute, tunably	Needs a rule for choosing the subset - this is where RigL-style criteria are used
Quantized training (int8/fp8 forward and backward)	Memory bandwidth and energy	Needs stochastic rounding and careful scaling to avoid gradient underflow
Gradient checkpointing	Activation memory (the dominant term)	About 30% extra compute

Technique	What it saves	Cost / caveat
Small batch + gradient accumulation	Peak activation memory	More steps; noisier BatchNorm - prefer GroupNorm
SGD or Lion instead of Adam	8 bytes/parameter of optimiser state	May need more careful learning-rate tuning
Forward-only / zeroth-order methods	The entire backward pass	Much slower convergence; viable for tiny adaptation

ACTIVATIONS, NOT WEIGHTS, ARE THE BINDING CONSTRAINT

For a small model, the weights may be a few hundred kilobytes while the activations retained for backpropagation are several megabytes, because they scale with batch size, spatial resolution and depth. The first three things to try are therefore: batch size 1 with accumulation, gradient checkpointing, and freezing the early layers - which removes their stored activations entirely.

```
# Update only the last block and all bias terms - a strong, cheap
# on-device personalisation recipe.
for name, p in model.named_parameters():
    p.requires_grad = name.endswith('.bias') or name.startswith('layer4')

trainable = [p for p in model.parameters() if p.requires_grad]
print('trainable:', sum(p.numel() for p in trainable),
      'of', sum(p.numel() for p in model.parameters()))

opt = torch.optim.SGD(trainable, lr=1e-3, momentum=0.9)   # 4 B/param state

# Halve activation memory again at ~30% compute cost:
model.layer4 = torch.utils.checkpoint.checkpoint_wrapper(model.layer4)
```

Listing 31.1 - Partial fine-tuning: the practical basis of on-device personalisation.

31.4 Federated learning

Federated learning trains a shared model across many devices without collecting their data. The canonical algorithm, **FedAvg**, is simple:

```
   server                                devices
   ------                                -------
   broadcast global weights  ---------->  1..K selected clients
                                          each trains E local epochs
   aggregate:  w <- SUM_k (n_k/n) w_k  <-- each returns its weights
   repeat for R rounds
```

Figure 31.1 - One round of federated averaging.

Challenge	Why it is hard	Approaches
Non-IID data	Each device sees a biased slice; local models diverge and averaging degrades	FedProx (proximal term), SCAFFOLD (control variates), server momentum, personalised heads
System heterogeneity	Devices differ in speed and availability; stragglers stall a round	Asynchronous or semi-synchronous aggregation, client sampling, deadline-based partial updates
Communication cost	Model updates are large and uplinks are slow	Update quantization and sparsification, top-k updates, low-rank updates, fewer rounds with more local work
Privacy	Weights leak information; gradients can be inverted to reconstruct inputs	Secure aggregation, differential privacy (noise + clipping), trusted execution environments
Evaluation	No central test set	Federated evaluation, held-out clients, careful per-client metrics

```
Differentially private SGD, per client:

clip:   g <- g * min(1, C / ||g||)

noise:  g <- g + N(0, sigma^2 C^2 I)

privacy accounting composes over rounds -> (eps, delta) guarantee
```

WHAT THE PRIVACY GUARANTEE ACTUALLY SAYS

An (eps, delta)-differentially private mechanism guarantees that the output distribution barely changes if any single user's data is removed - so an adversary cannot confidently infer whether you participated. It does NOT guarantee that the model is safe to publish for other reasons, and eps values used in industry (often 5-10) are far weaker than the theoretical ideal of eps < 1. Report eps, delta, the clipping norm, the noise multiplier and the unit of privacy (per example or per user) - without all five, a privacy claim is not checkable.

31.5 The deployment stack for edge machine learning

Layer	Options
Training frameworks	PyTorch, JAX, TensorFlow
Export / IR	ONNX, ExecuTorch, TFLite FlatBuffer, TorchScript
Optimisation	Quantization, pruning, operator fusion, constant folding, layout transformation
Runtime	ONNX Runtime, TFLite / LiteRT, ExecuTorch, TVM, CoreML, NNAPI, TensorRT
Bare-metal / MCU	TFLite Micro, CMSIS-NN, microTVM, vendor SDKs
Accelerators	NPU, DSP, GPU, or a fixed-function CNN engine on the SoC

Before shipping a model to a device

- Measured latency, peak RAM and energy on the real hardware, not simulated.
- Accuracy verified with the target runtime's own arithmetic, not the training framework's.
- Thermal behaviour under sustained load checked - sustained throughput can be far below burst throughput.
- A fallback path if the accelerator is unavailable or busy.
- An update mechanism, with model versioning and rollback.
- On-device metrics collected (with consent) so drift is detectable.
- For on-device training: bounds on how far the local model may drift, and a way to reset to the global model.

Exercises

1. Measure the peak memory of full fine-tuning, head-only training and bias-only training for the same model, and separate the weight, gradient, optimiser and activation components.
2. Implement FedAvg over 20 simulated clients with IID and then pathologically non-IID splits; plot global accuracy against rounds.
3. Add gradient clipping and Gaussian noise to the federated setup and plot the accuracy/privacy trade-off for several noise multipliers.

32. Reinforcement Learning

In reinforcement learning an agent learns by acting. There is no dataset of correct answers - only a reward signal that may be sparse, delayed, and affected by everything the agent did earlier.

32.1 The formalism: Markov decision processes

```
MDP = (S, A, P, R, gamma)

Policy      pi(a | s)

Return      G_t = SUM_(k>=0) gamma^k r_(t+k+1)

Value       V^pi(s) = E[ G_t | s_t = s ]

Q-value     Q^pi(s,a) = E[ G_t | s_t = s, a_t = a ]

Advantage   A(s,a) = Q(s,a) - V(s)
```

```
Bellman optimality:  Q*(s,a) = E[ r + gamma * max_a' Q*(s', a') ]
```

The discount factor gamma in [0, 1) makes infinite-horizon returns finite and encodes how much the agent cares about the future; 0.99 is a common default, and small changes to it can change behaviour drastically.

32.2 The two families

Family	Learns	Representative algorithms	Character
Value-based	Q(s,a), acts greedily	Q-learning, DQN, Double DQN, Rainbow	Off-policy, sample-efficient, discrete actions
Policy-based	pi(a|s) directly	REINFORCE, TRPO, PPO	Handles continuous actions and stochastic policies; higher variance
Actor-critic	Both	A2C/A3C, DDPG, TD3, SAC	The practical middle ground; SAC and PPO are today's defaults
Model-based	A model of P and R	Dyna, MuZero, Dreamer	Much more sample-efficient; harder to build and to trust

32.3 Q-learning and DQN

```
Tabular update:

Q(s,a) <- Q(s,a) + alpha [ r + gamma max_a' Q(s',a') - Q(s,a) ]

DQN loss (neural approximation):

L = ( r + gamma max_a' Q_target(s',a') - Q(s,a) )^2
```

- **Replay buffer:** store transitions and sample uniformly (or by TD error - prioritised replay). Breaks the correlation between consecutive samples that would otherwise destabilise training.
- **Target network:** a periodically-copied frozen copy of Q supplies the bootstrap target, preventing the network from chasing its own moving predictions.
- **Double DQN:** select the action with the online network and evaluate it with the target network, removing the systematic overestimation of max.
- **Exploration:** epsilon-greedy decayed over training is the baseline; noisy networks and count-based or curiosity bonuses do better on hard-exploration tasks.

32.4 Policy gradients and PPO

```
Policy gradient theorem:

grad J = E[ grad log pi(a|s) * A(s,a) ]

PPO clipped objective:

L = E[ min( r_t A_t , clip(r_t, 1-e, 1+e) A_t ) ],  r_t = pi/pi_old
```

The clipping keeps each update inside a trust region: if the new policy moves too far from the old one on a sample, the objective stops rewarding the move. PPO is the default on-policy algorithm because it is robust, simple to implement, and parallelises well - which is also why it became the workhorse of RLHF for language models.

- **GAE** (generalised advantage estimation) interpolates between high-bias/low-variance TD and low-bias/high-variance Monte Carlo advantage estimates with a parameter lambda around 0.95.
- **SAC** adds an entropy bonus to the objective, producing a stochastic policy that explores by construction; it is the standard for continuous control and is markedly more sample-efficient than PPO.
- **Normalise observations and rewards**, clip the value loss, and anneal the learning rate - RL implementations are notoriously sensitive to these details, and published gains have repeatedly turned out to come from them rather than from the algorithm.

32.5 A gridworld you can solve on paper

Reinforcement learning becomes concrete on a four-state corridor. States S1..S4, actions left and right, reward +1 for reaching S4 and 0 elsewhere, gamma = 0.9.

```
     [S1] <---> [S2] <---> [S3] <---> [S4 goal, +1]

  Optimal values, working backwards from the goal:
     V(S4) = 1.0        (terminal)
     V(S3) = 0 + 0.9 * 1.0   = 0.90
     V(S2) = 0 + 0.9 * 0.90  = 0.81
     V(S1) = 0 + 0.9 * 0.81  = 0.729
```

Figure 32.1 - Value propagates backwards from reward, decaying by gamma per step.

This tiny example shows the three ideas that carry all the way to AlphaZero. Value is **discounted distance to reward**: gamma = 0.9 means a reward three steps away is worth 0.729 now, so the agent prefers shorter paths without being told to. Value **propagates backwards** one step per update, which is why sparse-reward problems need so many episodes - the signal has to crawl back from the goal. And a **policy falls out of the values for free**: in each state, move to the neighbouring state with the higher value.

gamma	Value of a reward 10 steps away	Behaviour
0.5	0.001	Extremely myopic - ignores anything beyond a few steps
0.9	0.35	Balanced; the common default
0.99	0.90	Far-sighted; needed for long tasks, but slows learning and increases variance
1.0	1.00	No discounting - only valid for episodes guaranteed to terminate

32.6 Why reinforcement learning is harder than supervised learning

Difficulty	Supervised learning	Reinforcement learning
Where the data comes from	A fixed dataset, independent of the model	The agent's own behaviour - a bad policy collects bad data
Feedback	The correct answer, for every sample	A scalar reward, often delayed by hundreds of steps
Credit assignment	Immediate: this prediction, this loss	Which of the last 200 actions caused the reward?

Difficulty	Supervised learning	Reinforcement learning
Stationarity	The target never moves	The target moves - the value function bootstraps from itself
Evaluation	Held-out test set	You must run the policy to know how good it is, and running it may be expensive or unsafe
Failure mode	Overfitting	Collapse, reward hacking, or silently converging to a mediocre policy

THE ORDER TO TRY THINGS IN

If you can log what a good decision-maker did, start with behaviour cloning - it is plain supervised learning and it works. If decisions do not affect future state, use a contextual bandit. Only when actions genuinely change the world you observe next is full reinforcement learning the right tool, and even then start with PPO or SAC and a carefully shaped reward rather than an exotic algorithm.

32.7 Where RL is worth it

- Sequential decisions where actions change the future state: robotics, control, inventory, ad auctions, game playing.
- Alignment of language models from preferences (Chapter 24), and training for verifiable outcomes such as passing unit tests.
- Not worth it when a supervised model on logged decisions would do - start with behaviour cloning or contextual bandits, which are far easier to make work and to evaluate.

OFFLINE EVALUATION IS THE HARD PART

You cannot evaluate a new policy from logged data by simply replaying it - the logged actions came from a different policy. Use off-policy evaluation (importance sampling, doubly robust estimators), a simulator you have validated, or a carefully bounded online experiment. Reward misspecification is equally dangerous: an agent optimises exactly what you wrote, not what you meant, and reward hacking is the norm rather than the exception.

Exercises

1. Implement tabular Q-learning on FrozenLake and plot the effect of epsilon decay and gamma.
2. Implement DQN on CartPole in 150 lines with a replay buffer and target network; then remove each of the two and observe the failure.
3. Implement PPO with GAE on a continuous-control task and compare against SAC in sample efficiency.

33. Uncertainty, Robustness and Interpretability

33.1 Two kinds of uncertainty

Type	Source	Reducible by more data?	Example
Aleatoric	Noise inherent in the data	No	Two identical X-rays with different outcomes; sensor noise
Epistemic	Ignorance of the right model or parameters	Yes	An input unlike anything in the training set

The distinction is operational: high epistemic uncertainty says 'collect data here or abstain'; high aleatoric uncertainty says 'this is as good as it gets - widen the interval'.

33.2 Estimating uncertainty in deep networks

Method	How	Cost	Quality
Softmax probability	Take the max output	Free	Poor - modern networks are overconfident and confident off-distribution
Temperature scaling	One scalar fitted on validation	Free	Fixes calibration in-distribution only
MC dropout	Keep dropout on at inference, average N passes	N forward passes	Cheap approximation to a Bayesian posterior
Deep ensembles	Train 5 models with different seeds	N trainings	The strongest practical baseline, including under shift
Bayesian NN (VI, Laplace, SWAG)	Posterior over weights	Moderate to high	Principled; Laplace approximations are now cheap and practical
Conformal prediction	Calibrate a nonconformity score on held-out data	Negligible	Distribution-free FINITE-SAMPLE coverage guarantee - underused and excellent
Evidential / quantile heads	Predict distribution parameters or quantiles directly	Free	Single-pass; needs the right loss

CONFORMAL PREDICTION IN THREE LINES

Fit any model. On a held-out calibration set compute a nonconformity score for each sample (for classification, s = 1 - p(true class)). Take the ceil((n+1)(1-alpha))/n empirical quantile q of those scores. At test time, output the SET of classes with 1 - p(class) <= q. That set contains the true label with probability at least 1 - alpha, with no assumption about the model or the data distribution beyond exchangeability. It is the cheapest rigorous guarantee available in machine learning.

33.3 Out-of-distribution detection and drift

- **Score-based OOD:** maximum softmax probability, energy score, Mahalanobis distance in feature space, or k-NN distance to the training set. Feature-space methods beat output-space ones.
- **Input drift:** monitor feature distributions with population stability index, KS tests or MMD. Cheap and catches pipeline breakages.
- **Prediction drift:** monitor the distribution of outputs - it moves before labels arrive.

- **Concept drift:** the relationship X -> y changes; only labels reveal it, so invest in a delayed-label pipeline and a small continuously labelled sample.

33.4 Adversarial robustness

```
FGSM:  x' = x + eps * sign( grad_x L(f(x), y) )

PGD :  iterate x' <- clip_(x, eps)( x' + a * sign(grad_x L) )

Adversarial training:  min_theta E[ max_(||d||<=eps) L(f(x+d), y) ]
```

- Imperceptible perturbations flip predictions on ordinary networks; this is a property of high-dimensional linear-ish decision boundaries, not a bug in a particular model.
- **Adversarial training** (train on PGD examples) is the only defence that has held up broadly. It costs 3-10x training time and typically several points of clean accuracy.
- **Certified defences** (randomised smoothing, interval bound propagation) give provable radii, at further cost.
- Beware evaluation pitfalls: gradient masking makes a defence look strong against weak attacks. Always evaluate with strong adaptive attacks such as AutoAttack.
- For most products, natural robustness (augmentation, distribution coverage, sanity checks on inputs) matters far more than L-infinity adversarial robustness - unless you face a genuine adversary.

33.5 Interpretability

Method	Scope	Notes
Linear/tree coefficients	Global, intrinsic	Use an interpretable model when the stakes require it - often the right answer
Permutation importance	Global, post-hoc	Model-agnostic; correlated features share credit
Partial dependence / ALE	Global	Shows the average shape of a feature's effect; ALE handles correlation better than PDP
SHAP	Local and global	Additive attributions with consistency guarantees; TreeSHAP is exact and fast
LIME	Local	Fits a local surrogate; unstable across runs
Integrated gradients	Local, differentiable models	Attribution along a path from a baseline; needs a meaningful baseline
Grad-CAM	Local, CNNs	Class-discriminative heat maps from the last convolutional layer
Attention maps	Local, Transformers	Attention is NOT explanation on its own; use with attribution methods
Concept-based (TCAV), probing	Global	Tests whether a human-defined concept is encoded
Mechanistic interpretability	Circuit level	Reverse-engineering internal algorithms; sparse autoencoders on activations are the current frontier

EXPLANATIONS CAN BE CONFIDENTLY WRONG

Saliency maps can look identical for a trained and a randomly initialised network. LIME and SHAP make independence assumptions that correlated features violate. A plausible explanation is not evidence that the model reasons that way. Use explanations to generate hypotheses, then TEST them - by intervening on the input, by ablating the feature and retraining, or by constructing counterfactual cases.

33.6 Fairness

- Define the harm before the metric. **Demographic parity** (equal positive rates), **equalised odds** (equal TPR and FPR across groups) and **calibration within groups** are mutually incompatible except in degenerate cases - you must choose, and the choice is a value judgement, not a technical one.
- Mitigations act at three stages: pre-processing (reweighting, resampling), in-processing (constrained optimisation, adversarial debiasing), post-processing (group-specific thresholds - often the most practical, sometimes legally constrained).
- Removing the protected attribute does not remove the bias; proxies (postcode, device, name) carry it. Measuring by group requires having the attribute, which creates its own governance problem.
- Report per-group performance as standard practice, exactly as you report per-class performance.

Exercises

1. Implement conformal prediction for a classifier and empirically verify 90% coverage on a held-out set.
2. Compare MC dropout, a 5-model deep ensemble and temperature scaling on both in-distribution and corrupted test data.
3. Attack a small CNN with FGSM and PGD, then adversarially train it and measure the clean/robust accuracy trade-off.

34. MLOps: Shipping and Keeping Models Alive

A model in a notebook has produced no value. This chapter is the engineering that stands between a good validation score and a system that keeps working for years.

34.1 The lifecycle

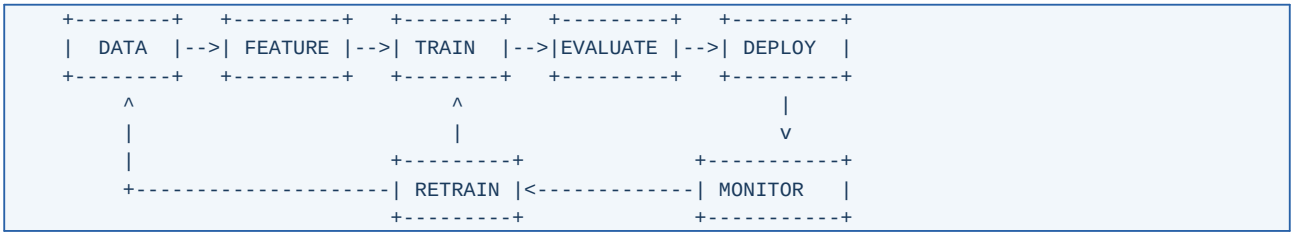

Figure 34.1 - The loop. Most teams build the top row and neglect the bottom one, which is where models die.

34.2 Versioning everything

Artefact	Tool / practice
Code	Git, with the training entry point and config in the repo
Data	DVC, LakeFS, Delta Lake, or immutable dated snapshots plus a content hash
Features	A feature store, or at minimum shared feature code used by both training and serving
Experiments	MLflow, Weights & Biases, or structured logs - config, metrics, environment, seeds
Models	A model registry with stages (staging, production, archived), lineage back to data and code, and signed artefacts
Environment	Pinned dependencies and a container image digest

34.3 Serving patterns

Pattern	Latency	Use when
Batch / offline scoring	Hours	Predictions can be precomputed - churn scores, recommendations refreshed nightly
Online / real-time API	10-500 ms	Predictions depend on live input
Streaming	Seconds	Event-driven scoring over Kafka/Flink
Edge / on-device	1-50 ms	Privacy, offline, or cost constraints (Chapter 31)
Embedded in the client	-	Small models shipped inside the app binary

- **Training/serving skew** is the number one production bug: the feature computed in the training pipeline differs from the one computed at serving time. The structural fix is to share one implementation, and to log serving features so you can compare distributions directly.
- **Shadow deployment:** run the new model alongside the old one on live traffic without acting on its output. Then **canary** to a small percentage, then ramp.
- **A/B test** against the business metric, not the offline metric. The two disagree more often than people expect.
- Always keep a **rollback** path, and rehearse it.

34.4 Monitoring

What a production model dashboard must show

- Operational: request rate, latency percentiles (p50/p95/p99), error rate, resource use.

- Input health: missing-value rates, out-of-range values, schema violations, cardinality changes.
- Drift: PSI or KS per important feature; embedding drift for unstructured inputs; prediction distribution over time.
- Performance: metrics on delayed labels, sliced by segment.
- Business: the metric the model exists to move.
- Alerting with thresholds someone has agreed to, and a documented response for each alert.

34.5 Retraining

- **Scheduled** (weekly/monthly): simple, predictable, and usually adequate.
- **Triggered** by drift or a performance drop: efficient but needs a trustworthy trigger and guard against thrashing.
- **Continual/online:** rarely necessary and easy to get wrong - feedback loops can make a model train on the consequences of its own predictions.
- Always validate a retrained model against the current production model on the same holdout before promoting it, and never promote automatically without that gate.

34.6 Testing machine learning systems

Test type	Example
Data validation	Schema, ranges, null rates, class balance (Great Expectations, Pandera)
Unit tests on features	A known input produces the expected feature value, including edge cases
Model behavioural tests	Invariance (irrelevant change leaves the prediction alone), directional expectation (income up -> risk down), minimum functionality on a curated set
Regression tests	Performance on a frozen golden set does not drop below a threshold
Slice tests	Per-subgroup metrics stay within bounds
Integration tests	The full pipeline runs end to end on a small sample in CI
Load tests	Latency under peak concurrency, with realistic payloads

34.7 Cost and carbon

- Track cost per 1,000 predictions and per training run. Quantization, batching, caching, distillation and a smaller model are the levers, roughly in that order of return.
- Cache aggressively: identical or near-identical requests are common, and semantic caching works well for LLM workloads.
- Right-size the hardware - many production models run happily on CPU, and a GPU that is idle 90% of the time is pure cost.
- Report energy or estimated emissions for large training runs; it is increasingly expected in papers and in corporate reporting.

Exercises

1. Take a model you have trained and wrap it in a container with a prediction endpoint, input validation and structured logging.
2. Implement PSI-based drift monitoring on a feature and simulate a drift event to verify the alert fires.
3. Write five behavioural tests for a model in your domain: two invariance, two directional, one minimum functionality.

35. Doing Research and Reading the Literature

35.1 Reading a paper efficiently

- **Pass 1 (5 minutes):** title, abstract, figures, tables, conclusion. Decide whether to continue.
- **Pass 2 (30 minutes):** introduction, method, and the experimental setup. Write down, in your own words, the one idea and the one experiment that supports it.
- **Pass 3 (hours):** re-derive the equations, check the ablations, and ask what is missing - which baseline is absent, which hyperparameter budget was unequal, which dataset would break it.
- Keep notes in a searchable form with a one-line summary per paper. In a year you will remember the idea and not the title.

Questions to ask of any empirical claim

- Is the baseline tuned as carefully as the proposed method?
- Are results averaged over seeds, with variance reported?
- Is the comparison at equal compute, equal parameters, or equal wall clock - and does the choice flatter the method?
- Is there an ablation isolating the claimed contribution?
- Could the gain come from the training recipe rather than the architecture?
- Is the test set possibly contaminated by the training data?
- Is the effect size larger than the noise, and is it practically meaningful?

35.2 Running an experiment that survives scrutiny

- Write the hypothesis and the decision rule **before** running: 'if X improves validation F1 by more than 0.5 points across 5 seeds, adopt it'.
- Change one thing at a time. A run that changes three things teaches you nothing when it improves.
- Keep a fixed, tuned baseline and re-run it whenever the codebase changes.
- Budget compute equally across compared methods, and say what the budget was.
- Log everything automatically; a result you cannot reproduce is a rumour.
- Report negative results to yourself honestly - the discipline of recording what failed is what stops you repeating it in six months.

35.3 Where the field is heading

- **Efficiency as the primary axis:** quantization, sparsity, distillation and better architectures are where most practical progress now lands, because inference cost dominates total cost of ownership.
- **Long context and memory:** state-space models, hybrid attention, and retrieval as an architectural component rather than a bolt-on.
- **Multimodality by default:** one model over text, image, audio and video, with tokenisers per modality.
- **Post-training and reasoning:** more of the capability gain now comes from what happens after pretraining - preference optimisation, verifiable-reward RL, and inference-time compute.
- **Agents and tool use:** models that act, with the attendant problems of reliability, evaluation and security.
- **On-device and private learning:** small capable models, personalised locally, with federated and differentially private updates.
- **Science of deep learning:** scaling laws, mechanistic interpretability, and a slowly improving theoretical account of why any of this generalises.

HOW TO STAY CURRENT WITHOUT DROWNING

Follow a small number of sources: two or three researchers' feeds, one curated newsletter, and the proceedings of NeurIPS/ICML/ICLR/CVPR/ACL skimmed once per cycle. Read one paper properly per week rather than twenty abstracts per day. Reimplement one method per quarter - implementation is the only reading comprehension test that works.

APPENDICES

Appendices

A compact mathematical reference, a glossary of every term used in this book, and a study roadmap with resources.

Appendix A. Mathematical Reference

Linear algebra

```
Dot product          a . b = SUM_i a_i b_i
Matrix product       (AB)_ij = SUM_k A_ik B_kj      (m,k)(k,n) -> (m,n)
Transpose            (AB)^T = B^T A^T
Inverse              (AB)^-1 = B^-1 A^-1
Trace                tr(AB) = tr(BA),   tr(A) = SUM_i lambda_i
Determinant          det(AB) = det(A)det(B),  det(A) = PROD_i lambda_i
Norms                ||x||_1, ||x||_2, ||x||_inf
Orthogonal Q         Q^T Q = I,  ||Qx|| = ||x||
Eigen                A v = lambda v;  symmetric A = Q L Q^T
SVD                  A = U S V^T  (always exists)
PSD                  x^T A x >= 0 for all x  <=>  all eigenvalues >= 0
```

Matrix calculus (denominator layout)

```
d(a^T x)/dx      = a
d(x^T A x)/dx    = (A + A^T) x   = 2Ax if A symmetric
d(||x||^2)/dx    = 2x
d(||Ax - b||^2)/dx = 2 A^T (Ax - b)
d(tr(AB))/dA     = B^T
d(log det A)/dA  = A^-T
Chain rule       dz/dx = (dz/dy)(dy/dx)
```

Probability

```
Bayes          P(H|D) = P(D|H)P(H)/P(D)
Expectation    E[aX + bY] = aE[X] + bE[Y]   (always)
Variance       Var[X] = E[X^2] - E[X]^2
               Var[aX] = a^2 Var[X]
               Var[X+Y] = Var[X]+Var[Y] if independent
Covariance     Cov[X,Y] = E[XY] - E[X]E[Y]
Gaussian       p(x) = (1/sqrt(2 pi s^2)) exp( -(x-mu)^2 / (2 s^2) )
CLT            mean of n iid samples -> N(mu, sigma^2/n)
Jensen         f convex => f(E[X]) <= E[f(X)]
```

Information theory

```
                    H(p)      = -SUM p log p
               H(p,q)    = -SUM p log q          (cross-entropy)
                KL(p||q)  = SUM p log(p/q) >= 0   (0 iff p = q)
                          H(p,q)    = H(p) + KL(p||q)
              I(X;Y)    = H(X) - H(X|Y) = KL( p(x,y) || p(x)p(y) )
```

Key derivatives used in this book

```
                sigmoid s(z)          s'(z) = s(z)(1 - s(z))
                    tanh(z)               1 - tanh^2(z)
                  ReLU(z)               1 if z > 0 else 0
                  GELU(z)               Phi(z) + z phi(z)
                softmax_i wrt z_j     p_i(delta_ij - p_j)
          CE(softmax) wrt z     p - y            <- the identity to remember
             BCE(sigmoid) wrt z    p - y            <- the same identity
                        MSE wrt y_hat         2(y_hat - y)/n
```

Complexity cheat-sheet

Operation	Cost
Dense layer forward, batch n	O(n * d_in * d_out)
Backward pass	About 2x the forward cost
Conv layer	O(K^2 * C_in * C_out * H * W)
Self-attention	O(n^2 d) time, O(n^2) memory naively
Normal equations	O(n d^2 + d^3)
SVD of (n,d)	O(n d min(n,d))
k-NN query, brute force	O(n d)
Decision tree training	O(n d log n)
k-means iteration	O(n k d)

Useful numbers

Quantity	Value
FP32 / FP16 / INT8 bytes per value	4 / 2 / 1
Adam training memory	~16 bytes per parameter (fp32) before activations
Cross-entropy at random init, K classes	log K (2.30 for K=10, 6.91 for K=1000)
Bootstrap sample coverage	63.2% of rows appear at least once
Chinchilla-optimal tokens per parameter	~20
KV cache bytes	2 * layers * kv_heads * head_dim * seq * batch * bytes_per_value
Rule of thumb, samples needed	10-100x the number of free parameters for classical models; far less with pretraining

Appendix B. Glossary

Term	Definition
Activation function	Non-linearity applied to a neuron's weighted sum; without it, depth adds nothing.
AdamW	Adam with decoupled weight decay; the default optimiser for Transformers.
Attention	A mechanism that computes a weighted average of values, where the weights come from query-key similarity.
Autoregressive	Generating a sequence one element at a time, each conditioned on all previous ones.
Backpropagation	Reverse-mode automatic differentiation applied to a neural network.
Bagging	Training models on bootstrap resamples and averaging them to cut variance.
Batch normalisation	Normalising activations using statistics of the current mini-batch, with learned scale and shift.
Bias (statistical)	Systematic error from a model class too simple to represent the truth.
Bias (parameter)	The additive constant term in a linear or convolutional layer.
Boosting	Sequentially adding weak models, each correcting the current ensemble's errors.
Calibration	Agreement between predicted probabilities and observed frequencies.
Capacity	How complex a function a model class can represent.
Catastrophic forgetting	Loss of previously learned ability when fine-tuning on new data.
Checkpointing (gradient)	Recomputing activations in the backward pass to save memory.
Contrastive learning	Training representations by pulling matching pairs together and pushing others apart.
Convolution	A weight-sharing local operation that slides a kernel over the input.
Cross-entropy	The standard classification loss; equals KL divergence from the label distribution up to a constant.
Cross-validation	Repeated train/validate splits used to estimate generalisation.
Diffusion model	A generative model that learns to reverse a gradual noising process.
Distillation	Training a small student to imitate a large teacher's outputs.
Distribution shift	The deployment data distribution differing from the training one.
Dropout	Randomly zeroing units during training as a regulariser.
Early stopping	Halting training when validation performance stops improving.
Embedding	A learned dense vector representing a discrete item.
Ensemble	A combination of several models' predictions.
Epoch	One full pass over the training set.
Feature	One measured input variable.
Fine-tuning	Continuing training of a pretrained model on a target task.
FLOPs	Floating-point operations; a compute measure that is a poor proxy for latency.
Generalisation	Performance on data not used for training.
Gradient descent	Iteratively stepping parameters against the gradient of the loss.
Gradient clipping	Rescaling gradients whose norm exceeds a threshold.
Hyperparameter	A setting chosen before training rather than learned from the training loss.
Inductive bias	The assumptions a model makes that let it prefer some solutions over others.
Inference	Running a trained model to obtain predictions.

Term	Definition
KV cache	Stored keys and values from previous tokens, so generation does not recompute them.
Label smoothing	Softening one-hot targets to reduce overconfidence.
Latent variable	An unobserved variable inferred by the model.
Layer normalisation	Normalising across features within one sample; batch-independent.
Learning rate	The step size in gradient descent; the most important hyperparameter.
Logit	An unbounded score before sigmoid or softmax.
LoRA	Low-rank adaptation: training small low-rank updates to frozen weights.
Loss function	The scalar measure of error that training minimises.
Mixed precision	Training with 16-bit compute and 32-bit master weights.
MLP	Multilayer perceptron; a stack of fully connected layers.
Momentum	Accumulating a velocity of past gradients to smooth and accelerate descent.
Non-parametric	A model whose complexity grows with the data, e.g. k-NN.
Overfitting	Fitting noise in the training data, so test error rises.
Parameter	A number learned from data during training.
Perplexity	exp(cross-entropy); the standard language-model metric.
Pruning	Removing weights, channels or blocks from a trained network.
Quantization	Representing weights and activations with fewer bits.
RAG	Retrieval-augmented generation: inserting retrieved documents into a model's prompt.
Receptive field	The region of input that influences one output unit.
Regularisation	Any change intended to reduce test error rather than training error.
Reinforcement learning	Learning a policy from rewards received while acting.
Residual connection	y = x + F(x); preserves gradient flow through depth.
RoPE	Rotary positional embedding; encodes position by rotating queries and keys.
Self-supervised learning	Supervised learning on labels derived automatically from the data itself.
Semi-supervised learning	Using labelled and unlabelled data together.
SGD	Stochastic gradient descent: updating from mini-batch gradients.
Softmax	Turning a vector of logits into a probability distribution.
Sparsity	The fraction of parameters or activations that are zero.
Straight-through estimator	Treating a non-differentiable forward op as the identity in the backward pass.
Supervised learning	Learning a mapping from labelled input-output pairs.
Tensor	An n-dimensional array, with autograd support in deep-learning frameworks.
Tokenisation	Splitting text into subword units a model can embed.
Transfer learning	Reusing knowledge from one task to improve another.
Transformer	An architecture built from self-attention and position-wise feedforward blocks with residual connections.
Underfitting	The model is too simple; both training and test error are high.
Unsupervised learning	Finding structure in data with no labels.
Validation set	Held-out data used to choose hyperparameters and stop training.
Vanishing gradient	Gradient magnitudes shrinking exponentially with depth, stalling early layers.

Term	Definition
Variance (statistical)	Sensitivity of the fitted model to which training sample was drawn.
Weight decay	Shrinking weights towards zero each step; L2 regularisation, decoupled in AdamW.
Zero-shot	Performing a task with no task-specific training examples.

Appendix C. Study Roadmap, Projects and Resources

A 24-week study plan

Weeks	Focus	Deliverable
1-2	Python, NumPy, pandas, plotting; Chapters 1-2	Load a dataset, clean it, plot five informative figures
3-4	Chapters 3-4: generalisation, splits, leakage, features	A leak-free pipeline with cross-validation
5-6	Chapters 5-7: linear and logistic regression, regularisation	Both implemented from scratch, matching scikit-learn
7-8	Chapters 8-11: k-NN, trees, SVM, ensembles	A tuned gradient-boosting model on a real tabular dataset
9-10	Chapters 12-13: unsupervised learning, evaluation	A clustering study and a full evaluation report with calibration
11-13	Chapters 14-17: MLP, backprop, activations, optimisers	Backpropagation in NumPy, 97%+ on MNIST
14-15	Chapters 18-20: normalisation, regularisation, debugging	A CNN on CIFAR-10 above 90% with a documented recipe
16-17	Chapter 21: CNNs and transfer learning	Fine-tune a pretrained backbone on your own images
18-20	Chapters 22-24: sequences, Transformers, LLMs	A character-level Transformer trained from scratch, plus a RAG demo
21-22	Chapters 25-27: generative and self-supervised models	A VAE and a small diffusion model on 32x32 images
23-24	Chapters 28-31, 34: efficiency and deployment	Quantize, prune and deploy one model to a device or an API, with measured latency

Portfolio projects worth building

- **End-to-end tabular:** a leak-free pipeline, tuned gradient boosting, SHAP explanations, calibrated probabilities, a threshold chosen from costs, and a deployed API. This single project demonstrates most of Parts I-II.
- **Vision transfer:** collect a few thousand of your own images, fine-tune a pretrained backbone, evaluate per class, and ship it to a phone with INT8 quantization.
- **From-scratch Transformer:** a character-level model trained on a text corpus you care about, with your own attention implementation.
- **Sensor / time-series:** windowing, leakage-safe group splits, a 1D-CNN or GRU baseline, and an on-device deployment with measured energy.
- **Compression study:** take one model, apply distillation, pruning and quantization, and produce the accuracy/size/latency Pareto curve on real hardware.
- **RAG assistant:** over documents you own, with an evaluation set you wrote and honest accuracy numbers.

Books

Book	Best for
Hands-On Machine Learning (Geron)	The best practical first book; scikit-learn and Keras
An Introduction to Statistical Learning (James et al.)	Classical ML with clear statistics; free PDF
The Elements of Statistical Learning (Hastie et al.)	The rigorous companion to the above

Book	Best for
Deep Learning (Goodfellow, Bengio, Courville)	Foundational theory; free online
Dive into Deep Learning (Zhang et al.)	Free, interactive, code-first, kept current
Pattern Recognition and Machine Learning (Bishop)	The Bayesian perspective
Probabilistic Machine Learning (Murphy)	Comprehensive modern reference in two volumes
Reinforcement Learning: An Introduction (Sutton & Barto)	The RL text; free online
Designing Machine Learning Systems (Huyen)	Production and MLOps
Efficient Deep Learning / TinyML literature	Quantization, pruning and edge deployment

Courses, tools and venues

- **Courses:** Andrew Ng's Machine Learning Specialization and Deep Learning Specialization; fast.ai Practical Deep Learning; Stanford CS231n (vision), CS224n (NLP), CS234 (RL); MIT 6.5940 (efficient deep learning); Hugging Face courses (NLP, diffusion, RL).
- **Libraries:** NumPy, pandas, scikit-learn, PyTorch, JAX, LightGBM/XGBoost/CatBoost, Hugging Face transformers/datasets/peft, Optuna, ONNX Runtime, ExecuTorch, TFLite, vLLM.
- **Data and practice:** Kaggle, OpenML, UCI, Hugging Face Datasets, Papers with Code.
- **Venues to skim:** NeurIPS, ICML, ICLR (general); CVPR, ICCV, ECCV (vision); ACL, EMNLP (language); MLSys (systems); arXiv cs.LG and cs.CV for preprints.

THE LAST PIECE OF ADVICE

Depth beats breadth. One project finished end to end - data collected, model trained, honestly evaluated, deployed, monitored, and improved after it failed in production - teaches more than twenty tutorials. The field will keep producing new architectures; the habits in Chapters 3, 13 and 20 will still be what separates a working system from a good validation score.